

Composition Is Where Obfuscation Robustness Breaks

ANONYMOUS AUTHOR(S)

Fine-tuning a code model on obfuscated programs raises its accuracy on those programs by roughly twenty points. We ask what that number is made of, and we ask it in three regimes rather than one: compositions of the transforms the model was tuned on, an obfuscator family it has never seen, and the same programs queried in the reverse direction — given a return value, produce a call that yields it. Across eight models and four lineages, the regimes disagree. On seen compositions every adaptation works and the differences are within five points. Outside them the ordering inverts. A single adapter trained on the pooled data of all six training conditions at once — the obvious way to spend the data — is below every arm assembled from the same conditions’ specialists on the unseen family, on all eight models, and below an adapter trained on unobfuscated code alone on six: training on more transforms made the model worse at a transform it had not seen. Every arm that fits the forward task pays for it backwards, and the failures are not malformed answers but the gold *forward* answer, returned to a question that asked for something else. Weight-space merging of the same specialists is the exception: at the floor of that failure mode on all eight models, above the untuned model on the reverse task on seven, and never significantly below it in any regime on any model — at a cost of three to six forward points where the specialists are the right fit. We conclude that adaptation to obfuscated code buys recombination of what the training distribution covered, that monolithic training buys it at the price of everything outside, and that composing specialists in weight space is the cheapest way found here to avoid paying it.

1 Introduction

Code obfuscation is widely used to make software difficult to inspect while preserving its intended functionality. In benign settings, developers employ obfuscation to protect intellectual property, implementation details, and proprietary business logic. In adversarial settings, however, malware authors routinely exploit the same techniques to conceal malicious behavior and frustrate program analysis and reverse engineering. Common transformations include identifier renaming, control-flow restructuring, dead-code insertion, expression rewriting, call indirection, etc. Although such transformations are intended to preserve program semantics, they can substantially obscure how those semantics are expressed in the source code, thereby increasing the difficulty of understanding the behavior of malware code [4, 5, 20, 23].

This difficulty is well established for human analysts. Nguyen *et al.* [17], for example, showed that code obfuscation significantly reduces humans’ accuracy in reasoning about program outputs. Such findings are particularly important for malware analysis, where analysts must often infer the behavior of code whose surface structure has been deliberately engineered to resist comprehension.

Large language models (LLMs) offer a promising alternative for automating such reasoning. Models have demonstrated strong capabilities in code generation, summarization, vulnerability detection, and program understanding [2, 6, 9, 14, 18]. However, most evaluations are performed on naturally written programs, where meaningful identifiers, familiar control structures, and common coding idioms provide abundant lexical and structural cues. Obfuscated programs remove or distort many of these cues while retaining the underlying computation, creating a useful stress test of whether an LLM actually reasons about program behavior or instead depends on surface regularities.

Recent studies suggest that this distinction matters. Rong *et al.* [?] showed that semantics-preserving obfuscations can cause substantial degradation in LLM program reasoning, including

program-output prediction. Similarly, Nikiema *et al.* [19] reported a statistically significant decline in model performance as the complexity of obfuscation increases. Together, these findings indicate that this limitation is especially problematic for security applications: an AI-assisted malware analyzer should not cease to understand a malicious computation merely due to obfuscation.

An important challenge, however, has received considerably less attention: *real obfuscated programs rarely contain only one transformation*. Obfuscators can combine multiple techniques so that, for example, renamed identifiers are embedded inside flattened control flow, rewritten expressions, and additional irrelevant code. Such *stacked obfuscation* creates a fundamentally harder deployment setting. Success on each transformation independently does not necessarily imply success when those transformations occur together. Consequently, simply demonstrating that a model can be adapted to individual obfuscations is insufficient. For practical malware analysis, the learned capability must also *compose*.

This observation naturally suggests several ways to compose knowledge learned from individual transformations. One could train a specialist for each obfuscation and use a *router* to select the appropriate specialist at inference time. Alternatively, one could combine the specialists in parameter space through *model merging*. A third, and seemingly more direct, strategy is to increase *training breadth* by fine-tuning a single model on examples from many individual obfuscations, hoping that knowledge learned separately will recombine when transformations are stacked. We evaluate these three strategies, together with a clean-code control and a KL-consistency objective that ties the student to its own behaviour on the unobfuscated parent, across eight models, and we evaluate them in three regimes rather than one: compositions of the transformations seen in training, an obfuscator family held out entirely, and the same programs queried in the reverse direction. The regimes disagree, and the disagreement is the result. On seen compositions every adaptation works and the choice between them is worth at most five points. Outside them the ordering inverts. Training breadth — the obvious way to spend the data — is the worst performer on the unseen family, below every arm assembled from per-transform specialists on all eight models and below an adapter trained on unobfuscated code alone on six. Broader exposure improves recombination within the training distribution while making the model *less* robust outside it.

Our analysis reveals why. The apparent robustness obtained from obfuscation-specific adaptation does not primarily arise from learning the semantics that remain invariant under transformation. Instead, models learn regularities associated with the *surface artifacts* left by particular obfuscators. Evidence for this behavior appears across multiple analyses: adaptation transfers poorly between distinct mechanisms within the same obfuscation family; the benefit of broad training is substantially stronger when recognizable identifier cues remain available; and improvements disappear when the same programs are queried in a different reasoning direction. Thus, stacking exposes a deeper problem than merely insufficient training coverage: obfuscation-specific fine-tuning can teach the model how particular transformations *look* without teaching it to consistently recover the behavior that those transformations preserve.

This diagnosis suggests a different principle for robust adaptation. Rather than attempting to compose transformation-specific knowledge, we tie learning to something that is invariant across all such transformations: the model’s behavior on the corresponding *unobfuscated program*. For every obfuscated training program x_{obf} , we retain its clean parent x_{clean} . A teacher model, tuned for reasoning over clean code, processes x_{clean} , while the student processes x_{obf} . In addition to the ordinary task loss, we minimize the KL divergence between their output distributions:

$$\mathcal{L} = \text{CE}(y \mid x_{\text{obf}}) + \lambda D_{\text{KL}}(p_{\text{teacher}}(\cdot \mid x_{\text{clean}}) \parallel p_{\text{student}}(\cdot \mid x_{\text{obf}})). \quad (1)$$

The first term exposes the student to obfuscated code, whereas the second explicitly encourages it to preserve the behavioral response of a model reasoning over the semantics-equivalent clean

program. Importantly, this objective does not require recognizing which obfuscations are present, selecting among specialists, merging transformation-specific parameters, or recovering the original source code. Instead, it changes the learning target itself: rather than tying adaptation to the surface patterns of individual transformations, it ties the student to the clean-code behavior that those transformations should preserve.

Across the two out-of-distribution regimes a single arm behaves differently from the rest. Weight-space merging of the per-transform specialists gives up three to six points on seen compositions, where fitting the training distribution pays, and gives up nothing anywhere else: it is above the untuned model on the reverse task on seven of eight models and is never significantly below it in any regime on any model. The mechanism is visible in what the other arms do when they fail backwards. Their failures are not malformed answers but the gold *forward* answer — the program’s return value, correctly computed, offered in reply to a question that asked for an input. Every single-adapter arm does this at a measurable rate; both merges sit at the floor of it on all eight models. Monolithic training fits the task as it was posed, these transformations and this direction, and composing separately-trained specialists in weight space does not produce that fit.

We state the bound plainly. Merging is an existing technique, applied here without modification; the contribution is not the algorithm but the evaluation that separates the arms and the finding that composition survives what monolithic training does not. Merging does not create a new ability to read obfuscation it has never seen — on the unseen family it is competitive with, not superior to, the router system of the same specialists — and it is not the best forward arm anywhere. What it does is avoid paying, outside the training distribution, for a fit inside it.

Contributions. This paper makes the following main contributions:

1. A three-regime evaluation of obfuscation adaptation, and the disagreement between the regimes. We evaluate six adaptations on eight models across four lineages in three regimes: compositions of seen transformations, a held-out obfuscator family, and the reverse task (given a return value, produce a call that yields it). On seen compositions the adaptations are within five points of one another; on the other two the ordering inverts. Reporting only the first regime, as prior work does, supports the opposite conclusion to reporting all three.

2. Monolithic training is the worst use of the data outside the training distribution. A single adapter trained on the pooled data of all six training conditions is below every arm assembled from per-condition specialists on the unseen family, on all eight models — and below an adapter trained on unobfuscated code alone on six. The comparison is data-matched by construction: the composed arms are built from specialists trained on exactly the rows breadth trains on, at the same rank, from the same base model.

3. Adaptation locks the direction it was trained in, and we measure it directly. Every single-adapter arm loses backward accuracy on the same programs it gained forward accuracy on. The failures are not parse errors: the most common ungradable backward reply is exactly the gold forward answer. We report this collapse rate per arm per model; the untuned models and both merges sit at its floor, and the highest rate belongs to the objective whose extra loss term most directly optimises for the forward answer.

4. Weight-space merging as the arm that never breaks, with its price stated. Merging the same specialists is above the untuned model on the reverse task on seven of eight models and is never significantly below it in any regime on any model, at a cost of three to six forward points where fitting the training distribution pays, one adapter, and no inference-time machinery. It is an existing technique used unmodified; the claim is about what the evaluation reveals, not about the algorithm.

2 Background

2.1 Obfuscating Transformations

An obfuscating transform rewrites a program's surface while preserving its observable behaviour: for every input, the transformed program returns what the original returned. The standard taxonomy separates *lexical* transforms, which rewrite identifiers and strip the names and annotations a reader relies on, from *control* transforms, which restructure the flow the computation takes, and *data* transforms, which change how values are represented and computed [4, 5]. The distinction matters here because it predicts what a model must recover. A lexical transform destroys evidence the model may have been relying on but leaves the computation's shape intact; a control transform leaves the identifiers and changes the shape; a data transform changes the arithmetic itself. An adaptation that generalises across all three has learned something about the computation. One that generalises within a family has learned that family's surface.

Two properties of the class are what make it a usable instrument. First, the transforms *compose*: an obfuscator may rename identifiers inside flattened control flow inside rewritten expressions, and real ones do. Composition is not a harder version of the same test but a different one, since a model may handle each part alone and still fail the stack. Second, because the transforms preserve semantics by construction, the ground truth is computable from the clean parent and is independent of any transform — so a grader can be exact, and a claim of invariance is falsifiable rather than a matter of judgement.

2.2 Behaviour Prediction as a Probe of Program Understanding

We evaluate program understanding as behaviour prediction rather than as generation or recovery. Given a program and an input, *output prediction* asks what it returns; given a program and a return value, *input prediction* asks for a call that produces it. Both are standard, and current code models are markedly imperfect at both even on unobfuscated code [8]. The pairing is what we use: the two directions share a program, a gold value and a grader, so an adaptation that improves one and damages the other has not become better at reading the program. Input prediction also has a property the forward direction lacks — inversion is many-to-one, so grading it requires executing the produced call rather than matching it, which we do throughout.

We never train on, or evaluate, recovery of the original source. That is the clean separation from the deobfuscation lineage: a model that can undo a transform tells us about the transform, whereas a model that answers correctly on the transformed program tells us about its understanding of the computation.

2.3 Low-Rank Adaptation

Fine-tuning a code model on a new distribution is now routinely done with low-rank adaptation [11], which freezes the base weights W and learns a low-rank update $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and $r \ll d$, applied to each target module. The practical consequence for this paper is not efficiency but *modularity*: an adapter is a small, separable object that can be trained per transform and combined with others afterwards, and the *same* architecture, rank and target modules can be held fixed while only the training data or the combination rule changes. Every arm below is a LoRA at the same rank on the same base model, so differences between them are differences in how the data was used, not in capacity.

3 Merging Obfuscation Specialists

We train one low-rank specialist per obfuscating transform and combine the specialists into a single adapter before inference. The resulting system occupies the same design point as a router

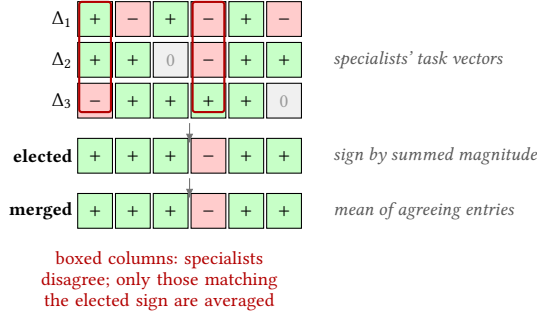


Fig. 1. **Sign-consensus merging, on six parameters of three specialists.** Each specialist contributes a task vector — the update its own training produced. Averaging these directly lets disagreeing updates cancel. TIES instead trims small entries, *elects* a sign per parameter by summing signed magnitudes across specialists, and averages only the entries that match it, so a parameter two specialists push positive and one pushes negative is merged from the two (boxed columns) [24]. DARE-TIES prepends a random drop of a fraction of each vector's entries with rescaling of the survivors, preserving the update in expectation [25]. Nothing is trained: both are arithmetic on weights already learned. The rule is shown here on task tensors, as it is defined; Figure 2 shows where it is actually applied for LoRA adapters, and why that is not the same operation.

over experts — per-transform specialists, composed — but resolves the composition offline, so the deployed artefact is one adapter of the base architecture, requiring no gate, no expert bank, and no additional computation at inference time. This section states the algorithm exactly as we run it.

3.1 Specialists

For each trainable condition $c \in \mathcal{C}$ we train one LoRA adapter on that condition's rows alone, holding the base weights W_0 frozen. Adapter c is the pair (A_c, B_c) with $A_c \in \mathbb{R}^{r \times k}$, $B_c \in \mathbb{R}^{d \times r}$, and it applies the update

$$\Delta_c = s_c B_c A_c, \quad s_c = \frac{\alpha}{r}, \quad (2)$$

where s_c is the LoRA scaling. All specialists share a rank, an α , a target-module set, an optimiser and a step budget, so they differ only in training data. We merge the $n = |\mathcal{C}|$ specialists for $\mathcal{C} = \{\text{L0, L1b, L1r, L2, S1, S2}\}$, which is exactly the set of conditions pooled training sees.

3.2 Sign-Consensus Merging

TIES [24] and DARE [25] were both introduced for *task vectors*: the difference $\theta_{\text{ft}} - \theta_{\text{pre}}$ between a fine-tuned and a pre-trained model, in the sense of task arithmetic [13]. Given a set of task tensors $\{T_c\}_{c=1}^n$ and combining weights $\{w_c\}$, both proceed in three stages.

1. *Sparsify*. Each tensor is reduced to a fraction ρ of its entries. TIES keeps the largest by magnitude,

$$\widehat{T}_c = T_c \odot \mathbf{1} \left[|T_c| \geq \text{topk}_\rho(|T_c|) \right], \quad (3)$$

while DARE drops entries at random and rescales the survivors so the tensor is preserved in expectation,

$$\widehat{T}_c = \frac{1}{\rho} T_c \odot M_c, \quad M_c \sim \text{Bernoulli}(\rho). \quad (4)$$

2. *Elect a sign.* A single sign is chosen per coordinate by summing the sparsified tensors and taking the sign of the sum, so that coordinates with large-magnitude support dominate:

$$\gamma = \text{sign}\left(\sum_{c=1}^n \widehat{T}_c\right), \quad m_c = 1[\text{sign}(\widehat{T}_c) = \gamma]. \quad (5)$$

This is the total variant of the majority-sign rule, which weights by summed magnitude; the alternative frequency variant counts signs instead. We use total throughout.

3. *Disjoint merge.* Only the entries agreeing with the elected sign contribute, and the sum is normalised by how many specialists contributed at that coordinate rather than by n :

$$T^* = \frac{\sum_{c=1}^n w_c \widehat{T}_c \odot m_c}{\max(\sum_{c=1}^n m_c, 1)}. \quad (6)$$

A coordinate on which every specialist opposes the consensus receives zero rather than an undefined value, which is what the clamp in the denominator provides. TIES applies Eq. 3; DARE-TIES applies Eq. 4; both then apply Eqs. 5–6. Neither involves any training.

3.3 Application to LoRA Adapters

The procedure above is defined on task tensors, and a LoRA adapter does not store one: it stores the factors (A_c, B_c) whose product is the update. Our implementation, which follows the standard PEFT path for LoRA, therefore treats the two factor sets as *separate* collections of task tensors and merges each independently:

$$A_* = \text{Merge}(\{A_c\}, \{\sqrt{w_c s_c}\}), \quad B_* = \text{Merge}(\{B_c\}, \{\sqrt{w_c s_c}\}), \quad (7)$$

where Merge is Eqs. 3–6 and the combining weight is split as a square root across the two sides so that the product recovers $w_c s_c$. The merged adapter is (A_*, B_*) , written at the original rank r with $\alpha = r$ so that its own scaling is unity and the factor already carries the specialists' scaling. It is served by the same multi-LoRA path as any single specialist.

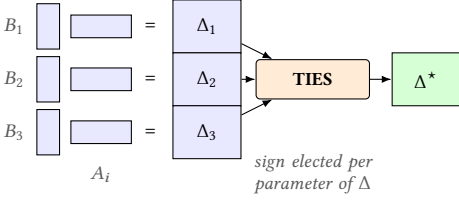
Equation 7 is *not* equivalent to applying Merge to the assembled updates $\{\Delta_c\}$, and the inequivalence is structural. Each entry of an update is a sum over the rank,

$$\Delta_c[m, n] = s_c \sum_{t=1}^r B_c[m, t] A_c[t, n], \quad (8)$$

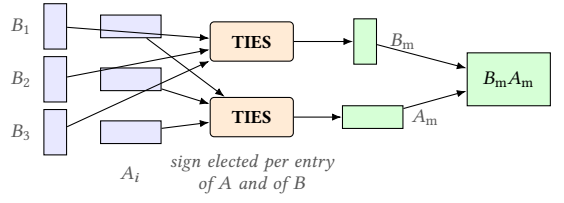
so no entry of A_c or B_c corresponds to a single coordinate of Δ_c : an entry of B_c participates in an entire row of the update and an entry of A_c in an entire column. The sign elected in Eq. 5 is therefore a statement about a factor coordinate, not about a parameter of the update, and the two merges compute different quantities. Figure 2 contrasts them. The gap is not a rounding effect: for random factors with $r = 4$, $n = 3$ and $\rho = 0.5$, $B_* A_*$ and $\text{Merge}(\{B_c A_c\})$ have a cosine similarity of 0.40 and disagree in sign on 39% of coordinates.

We merge in factor space because the alternative introduces a confound. Recovering a rank- r adapter from a merged Δ^* requires a truncated SVD, which changes the adapter's rank and introduces reconstruction error *before* the merging rule is evaluated; the deployed artefact would no longer match the base architecture, and any measured effect would combine the merge with its re-extraction. Factor-space merging preserves the rank exactly. The consequence for interpretation is that our results characterise sign-consensus merging *as it is applied to LoRA adapters in practice*, not the published task-vector algorithm in its original setting; whether the two differ in downstream accuracy is an open question these experiments do not address.

(a) update space — TIES as published



(b) factor space — what PEFT applies to LoRA



cosine 0.40, 39% of entries differ in sign

$$\Delta[m, n] = \sum_{t=1}^r B[m, t] A[t, n]: \text{one entry of the update draws on a whole column of } B \text{ and row of } A, \text{ so "agree on this parameter" is not the same statement in the two spaces}$$

Fig. 2. **Sign election in update space and in factor space are different operations.** (a) The published algorithm forms each specialist’s update $\Delta_i = B_i A_i$ and elects a sign per *parameter of the update*. (b) The standard LoRA implementation, and the one we use, applies the same rule independently to the A factors and to the B factors and multiplies the results, so signs are elected per *factor entry*. Because each entry of Δ is a sum over the rank, no factor entry corresponds to one parameter of the update, and the two merges do not coincide: on random factors at $r=4$ with three specialists and density 0.5 their outputs have cosine similarity 0.40 and disagree in sign on 39% of entries. We use (b) because the alternative — assembling Δ , merging, and re-extracting by SVD — changes the adapter’s rank and adds reconstruction error before the merging rule is evaluated.

3.4 Configuration

We report two variants, TIES (Eq. 3) and DARE-TIES (Eq. 4), both with density $\rho = 0.5$, the total majority-sign rule, and uniform weights $w_c = 1/n$. Non-uniform weighting is an additional degree of freedom; holding the weights uniform preserves the comparison against pooled training — which likewise weights its constituent conditions equally by construction — as a comparison of composition strategy rather than of tuning effort. Density was selected by a sweep over $\rho \in \{0.3, 0.5, 0.7\}$ on a 1.5 B pilot model outside the evaluation panel and applied unchanged to all eight panel models, so it is neither fitted to the panel nor tuned for it. DARE-TIES is stochastic through Eq. 4; we fix its seed and record it with the merged adapter, and each merge is performed once in float32.

4 Alternative Adaptation Strategies

Figure 3 places the merge among the five other ways one might spend the same budget of obfuscated training data. The first two do not adapt weights at all and bound the problem; the next three are the standard single-adapter answers; the last composes. Every arm is a LoRA of the same rank on the same target modules of the same base model, so what differs is the data and the combination rule, not the capacity. We give each enough detail to be reimplemented; exact hyperparameters are in Section 5.

Untuned. The instruction-tuned checkpoint, prompted directly, with no weight change. It is the reference every other arm is measured against, and on the backward task it is a surprisingly strong one.

In-context. The same checkpoint with one solved example prepended to the prompt [1]. The example is a single frozen seven-line program written for the purpose — not drawn from the corpus — shown *unobfuscated* whatever condition is being evaluated, with its gold value obtained by executing it through the same sandbox as every other item. It is identical for every item, model and condition, and its identity is recorded in the prompt-template hash, so no result here depends

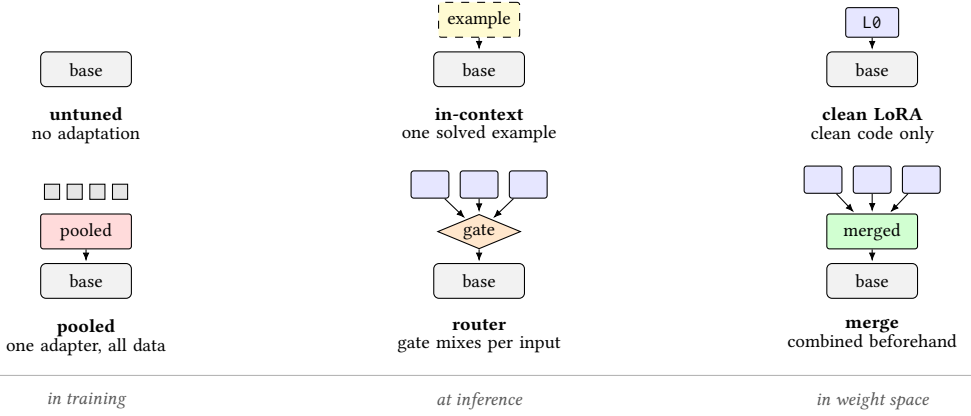


Fig. 3. **The six arms.** All are the same base model at the same LoRA rank, differing only in how the training data is used and where per-transform knowledge is combined. Top row: arms that never see an obfuscated program in training — *untuned*, *in-context*, and the *clean LoRA* control that separates learning the task from learning the transforms. Bottom row: the three ways to spend the obfuscated data, ordered by *when* the specialists are combined — *pooled* collapses them during training, *router* composes them per input at inference and must keep every expert resident, and *merge* composes them in weight space beforehand, leaving one adapter with no gate and no run-time cost. The KL-consistency arm is *pooled* with an added consistency term (Section 4).

on example selection. Because the example is clean code, this arm sees no obfuscated program at any point: it isolates whether the *answer format*, rather than the obfuscation, is what the untuned model is failing at, and it is the weakest intervention in the study by design.

Clean-code LoRA. An adapter trained on unobfuscated programs only, with the same prompt, optimiser and step count as every other adapter. It is the control that separates *learning the task* from *learning the transforms*: it sees the prompt format and the answer distribution but never an obfuscated program, so whatever it gains is not transform knowledge. It is also the teacher for the KL arm below.

Pooled training. One adapter trained on the union of every training condition, shuffled. This is the obvious way to spend the data and the one most likely to be done in practice: it needs no per-transform bookkeeping, no gate and no merge step. Because every composed arm below is built from specialists trained on exactly these rows, pooled training is the data-matched control for all of them.

KL-consistency training. Pooled training plus a distillation term [10] that ties the student’s answer to its own behaviour on the clean parent. For an obfuscated program x_{obf} with clean parent x_{clean} and gold answer y ,

$$\mathcal{L} = \text{CE}(y \mid x_{\text{obf}}) + \lambda D_{\text{KL}}(p_{\text{teacher}}(\cdot \mid x_{\text{clean}}) \parallel p_{\text{student}}(\cdot \mid x_{\text{obf}})), \quad (9)$$

where the teacher is the frozen clean-code LoRA above, the KL is taken over the answer tokens only, and the student never sees the parent. Three properties make it a fair comparison rather than an extra knob. It trains on exactly the rows pooled training uses, for the same number of steps. At $\lambda = 0$ it reduces to pooled training identically, so pooled training is not an approximate control but an exact one. And the teacher is fixed across all conditions, so the term adds no transform-specific supervision — it only asks that the same computation, written two ways, be answered the same way.

Router system. One adapter per transform, kept separate, with a learned gate that mixes their low-rank updates per token and per layer. The gate reads the decoder layer’s input hidden state h , so one gate serves every target projection in that layer, and scores experts by attention against a learned key k_e per expert:

$$a = \text{softmax}_e \left(\frac{(W_q \text{LayerNorm}(h)) \cdot k_e}{\sqrt{d_r} \tau} \right), \quad (10)$$

with router width d_r and a learned temperature τ per layer. Only the gate is trained; the experts and the base model stay frozen, so the arm adds a small number of parameters relative to the expert bank. Because a learned gate can be worth nothing over no gate at all, we report three controls alongside it — the same gate argmaxed to a hard route, a fixed uniform gate, and a gate frozen at random initialisation — which separate what the *mixture* is worth from what the *routing* is worth. This arm is the most expressive of the six and the most expensive: every expert must be resident at inference, and the gate runs at every layer for every token.

5 Evaluation Design

5.1 Behaviour Prediction Tasks and Grading

Both tasks are posed on the program *as the model is shown it* — the obfuscated variant, never the clean parent — and both are decoded greedily with a constrained, no-chain-of-thought answer format. The two are graded by different procedures, which we state separately because the asymmetry is not incidental: forward answers are unique and backward answers are not.

Output prediction (forward). The prompt gives the program and a call, and asks for the value the call returns. The reply is graded by normalised exact match against the gold value, which is obtained by executing the *clean parent* on the same arguments, so the reference is independent of any transform. Normalisation is deliberately minimal and fixed in advance: surrounding whitespace is stripped, a code fence or pair of backticks is unwrapped, and a single trailing period is removed. No containment or substring matching is used — an earlier audit of a substring grader on this corpus found roughly 3% false positives, for instance 927 matching inside 9273. Where the value parses as a literal in both the prediction and the gold, comparison is structural with a floating-point tolerance of 10^{-6} ; otherwise it is on the normalised strings. We additionally record *raw exact match* — byte equality after whitespace stripping alone, with no normalisation — so the sensitivity of any effect to the normalisation rules can be checked.

Input prediction (backward). The prompt gives the program and a return value, and asks for a call that produces it. Grading is by *execution*, not by matching: the predicted call is run against the obfuscated program in the project’s sandbox and is correct if the canonical representation of its return value equals the gold value. Matching the gold *arguments* is not required, because a return value generally has many pre-images and finding a different valid input is a success rather than an error. We nonetheless report the stricter rate at which the gold arguments are recovered (*exact-argument* accuracy) alongside the execution rate throughout, because the two differ substantially and the gap is itself a finding (Section 6).

Three details of the backward grader matter for reproduction. The reply must be a single call name(arg, . . .) whose arguments are literals; prose, expressions such as range(3), variable references and multi-line replies are recorded as format failures rather than repaired, since hunting for a call inside prose would conceal exactly the failure mode we want to measure. Arguments are literal-checked in the parent process before anything reaches the sandbox, so an unparseable prediction is a format failure and never an execution error. And the function *name* before the

parenthesis is not graded: under identifier renaming the entry point is renamed and the prompt states the new name, so a model that writes the original name has still identified the input; the call is executed against the item’s true entry point.

Format failures. Both graders report a format-failure rate, and the constrained answer format is what makes that rate meaningful. A cell whose rate exceeds 0.25 is excluded from every mean and marked rather than hidden; Section 5 gives the two marks and what distinguishes them.

5.2 Scope of Transformations Studied

Single transforms fall into three families (Section 2): *identifier* transforms (L1b, L1r, L2), *structural* transforms (S1, S2, and S2’s two halves S3 and S4), and an *encoding* family (X1 and its halves) that is held out of all training. Each single transform is applied to the clean parent and never stacked during training; stacks are a separate namespace, evaluated only. Python throughout; a JavaScript corpus exists but could not be regenerated on our cluster (Section 9).

5.3 Model Panel

Table 1 lists the panel: eight instruct models from four lineages (Meta, BigCode, Google, IBM), code-specialised and general, 7B to 34B, plus one pretrained checkpoint used only for the one-shot baseline. Every adapter is a LoRA with the same rank and target modules on every model, and one prompt builder serves training and evaluation alike.

Table 1. **The model panel.** Four lineages, code-specialised and general instruct models, 7B to 34B. *Chat template: system* accepts a system role; *merged* folds the system text into the user turn; *plain* has no template and is used for the one-shot baseline only. Parameter counts are nominal. Every adapter is a LoRA with the same rank and target modules on every model.

Model	Checkpoint	Lineage	Type	Params (B)	Layers	Hidden	Chat template
CodeLlama-7B [21]	codellama/CodeLlama-7b-Instruct-hf	Meta	coder	6.7	32	4096	system
CodeLlama-13B [21]	codellama/CodeLlama-13b-Instruct-hf	Meta	coder	13.0	40	5120	system
CodeLlama-34B [21]	codellama/CodeLlama-34b-Instruct-hf	Meta	coder	33.7	48	8192	system
Llama-3.1-8B [15]	meta-llama/llama-3.1-8B-Instruct	Meta	instruct	8.0	32	4096	system
StarCoder2-15B [16]	bigcode/starcoder2-15b-instruct-v0.1	BigCode	coder	15.5	40	6144	merged
Gemma-3-12B [7]	google/gemma-3-12b-it	Google	instruct	12.2	48	3840	system
CodeGemma-7B [3]	google/codegemma-7b-it	Google	coder	8.5	28	3072	merged
Granite-3.1-8B [12]	ibm-granite/granite-3.1-8b-instruct	IBM	instruct	8.2	40	4096	system
Llama-3.1-8B (base) [15]	meta-llama/llama-3.1-8B	Meta	pretrained	8.0	32	4096	plain

5.4 Corpus and Splits

Table 2 summarises the corpus. The split is by program, never by row; three input cases per program are fixed and their gold outputs executed from the clean parent, so the grader is independent of any transform. Coverage differs by condition because structural transforms decline some programs by design and the held-out family needs enough encodable sites; every paired contrast is computed on the programs common to the cells it compares, fixed before evaluation.

5.5 Single Transformations

Table 3 defines the single transforms. The identifier family destroys the evidence a reader relies on and leaves the computation’s shape intact; the structural family leaves identifiers and changes the shape; the encoding family changes how values are represented. Within the structural family, S3 and S4 are the two halves of S2 — one the reader must reason about, one it can ignore — and exist as separate transforms so that composition can be checked against a known whole.

Table 2. **The dataset.** One Python corpus, split by program so no obfuscated variant of a training program can reach evaluation.

Property	Value
Base programs (Python)	2231: apps 1584, cruxeval 543, humaneval 104
Split	by program, seed 17: train 1563 / val 111 / held-out 557
Input cases per program	3, gold output executed from the clean parent
Program length	mean 8.4 LOC, max 57
Held-out eval, clean	557 programs $\times$ 3 cases = 1671 items
Coverage per condition	Table 3 and Appendix A; structural transforms decline some programs by design, the unseen family needs ≥ 3 encodable sites
Training rows per adapter	one condition's train split $\times$ 3 cases (4689); breadth pools the five seen conditions
Paired contrasts	on the programs common to every cell in the contrast, fixed before evaluation; program-clustered bootstrap, 2,000 resamples
JavaScript	168 held-out programs (504 items), transferred intact; not regenerable on the cluster
Grading	strict normalised exact match, no containment; the backward task executes the produced call against the clean parent

5.6 Stacked Composites

Composites are a separate namespace from the ladder: parts are applied to the clean parent left to right, and order matters, so C_L1r_S1 and C_S1_L1r are different conditions — the second renames the state variables that control-flow flattening introduces. We evaluate sixteen. Six are depth-2 stacks of seen transforms, including an order-control pair and one positive control (C_S4_S3) that reconstructs S2 by composing its two halves, which is how we check that composition behaves as the ladder predicts. Four go deeper on seen transforms: three at depth 3 and one at depth 4. The remaining six contain the held-out family — three at depth 2, two more using half of it (arithmetic or strings alone), and one at depth 3 — and are the conditions no arm was trained for in any part. Each composite inflates program length, by $5\times$ to $22\times$ over the clean parent, and a per-composite cap bounds that growth; coverage differs between them because structural transforms decline some programs by design, so every paired contrast runs on the programs common to the cells it compares, fixed before evaluation. The full per-composite definitions and item counts are in `configs/conditions_composite.yaml`.

5.7 Held-Out Family and Quarantine Protocol

The paper's discriminator is an obfuscator family no arm is trained on. Its stimuli live under a quarantine that no training path can read: the single training entry point refuses anything outside the training tree or carrying the family's labels, a lint pass rejects raw reads that bypass it, the generator that produces the family refuses to run without an explicit flag and writes read-only, and a content scan over every training file checks for the family's markers regardless of labels. The family itself, H1, was evaluated exactly twice — one frozen pilot pass and one final pass — under a pre-registered discipline with every read logged, and its budget is spent.

Every unseen-family number in this paper is therefore measured on X1, a trainable sibling built for exactly this purpose: it reproduces H1's difficulty at $r = 0.9992$ (mean absolute difference 0.48 points) on arms trained on neither, and transfer from X1 to H1 is lossless (0.08 points) for arms trained on X1. No number from H1 selects, tunes or ranks anything.

Table 3. **Single transforms.** Each is applied to the clean parent and never stacked. *Cap* is the maximum growth factor over the parent above which a program is declined rather than clipped; *programs / items* is held-out coverage. H1 is the quarantined held-out obfuscator whose evaluation budget is spent; X1 is its trainable sibling and carries every unseen-family number.

Code	Family	Transform	Trainable	Cap	Programs / items	Role
L0	none	normalised original: comments and docstrings stripped, 4-space indent	yes	1.2×	557 / 1671	clean reference
L1b	identifier	adversarial renaming of all bindings incl. the entry function, from semantic-inversion tables (max→min) and an unrelated domain vocabulary	yes	2.0×	553 / 1659	ladder
L1r	identifier	random hex renaming of all bindings (v_a3f2, f_9c1e)	yes	2.0×	557 / 1671	ladder
L2	identifier	sequential minification of all bindings (a, b, ..., aa) plus type-annotation stripping	yes	1.5×	557 / 1671	ladder
S1	structural	control-flow flattening of the entry function into a dispatch loop with randomised state ids and shuffled cases	yes	6.0×	416 / 1248	ladder
S2	structural	opaque predicates (computed always-true/false guards) plus never-called dead helpers	yes	4.0×	556 / 1668	ladder
S3	structural	dead-code insertion only: 1–2 never-called module-level helpers (must be ignored)	yes	4.0×	557 / 1671	stacks only
S4	structural	opaque predicates only: 1–3 computed guards at statement boundaries (must be reasoned about)	yes	4.0×	556 / 1668	stacks only
X1	encoding	every string literal through an inline XOR decoder, every + − ^ through a type-guarded algebraic identity, int literals expanded	yes	12.0×	405 / 1215	unseen family
X1m	encoding	X1’s arithmetic half only (mechanisms: [mba])	yes	12.0×	351 / 1053	mechanism ablation
X1s	encoding	X1’s string half only (mechanisms: [str]); site bar 1	yes	12.0×	246 / 738	mechanism ablation
H1	heldout	string encoding (base64 / RC4 string array) plus guarded mixed-boolean-arithmetic rewriting	never	12.0×	–	quarantined

5.8 System Instantiation

Section 4 describes what each arm is; this is how it is instantiated. Every system is a LoRA of rank 32 ($\alpha = 64$) on the same target modules of the same base model, evaluated on still-obfuscated code, so the arms differ only in what they were trained on and how they are combined.

Clean LoRA (tuned_L0) is trained on unobfuscated programs; one *specialist* per seen transform; *breadth* (mono_all) on the pooled data of all six training conditions — the five seen transforms and clean code — 26,841 rows in 1,257 steps. The *KL-consistency* objective (cons_lam3) is Eq. 1 at $\lambda = 3$, a plateau from a sweep on the trainable grid, with a frozen clean-code-tuned adapter as teacher; the student never sees the parent, and it trains on exactly breadth’s rows and steps, so at $\lambda = 0$ it

reduces to breadth identically. The *router* (*mole_router*) mixes eight specialists with a learned gate, reported alongside its argmax, a fixed uniform gate and a gate frozen at random initialisation.

Merges combine the six adapters for $\{L0, L1b, L1r, L2, S1, S2\}$ — precisely the conditions breadth pools — with uniform weights, density 0.5, and sign election by summed signed magnitude. We report TIES and DARE-TIES. Density was selected by a sweep over $\{0.3, 0.5, 0.7\}$ on a 1.5 B pilot model outside the panel and applied unchanged to all eight, so it is not fitted to the panel and not tuned for it either. Merging is done in LoRA space (Section 3) and the merged adapter is written at the same rank, so it is served exactly as any other adapter is.

Two arms exist but are excluded from the comparison tables. *Family* (tuned_X1) is trained on the held-out family’s trainable sibling: it is the ceiling for that family and is reported only as that, because on exactly the conditions that matter most it is the one arm for which the held-out family is not held out. *DARE-linear* is a third merge variant we ran and do not discuss; it is in the appendix grids.

5.9 Measurement and Reporting

Every contrast is a paired difference on the program subset common to its cells, with a program-clustered bootstrap of 2,000 resamples; bold intervals exclude zero. Decision rules were pre-registered before submission and are reported by their verdicts even when inconclusive. A cell whose format-failure rate exceeds 0.25 enters no mean, and is marked rather than hidden. Such a cell is not always a broken prompt contract: decomposing the gated cells shows that many of them are replies that are exactly the gold *forward* answer — on CodeGemma-7B’s KL-consistency arm 96 % of them, on StarCoder2-15B’s tuned arms 79–91 % — which is the forward-locking effect of Section 6, not a formatting failure. We therefore distinguish the two in every table: $\ddagger$ marks a gated cell whose replies are predominantly the gold forward answer and $\dagger$ one whose replies are genuinely unparseable. Gating removes cells non-randomly, concentrating in the backward direction and on weaker models, so backward means rest on fewer cells than forward ones. Untuned format failure measures adherence to the prompt contract, not capability, so screening base models by it is unsound.

6 Results

Table 4 is the whole study on one page: six adaptations, eight models, and the three regimes that separate them. Read down a block and the models agree with each other; read *across* the blocks and the bold moves. On stacks built from transforms the adapters were trained on, the best arm is the KL-consistency arm on five models, the router system on two and breadth on one. On the held-out family the router system takes six and the KL-consistency arm two; backwards the merge takes six. On the held-out family the bold moves to the router system on six of eight, and on the reverse task to the merge on six — both assembled from per-transform specialists. No arm wins everywhere, and the one that never loses is not the one that wins most often.

6.1 Adaptation Succeeds on Seen Compositions

Stacking is where adaptation was supposed to break, and it does not. Every arm roughly doubles the untuned model on depth-2, -3 and -4 compositions of the seen transforms: 0.143 to 0.32–0.37 on CodeLlama-7B, 0.161 to 0.39–0.45 on CodeLlama-34B, and so on down the panel. The spread among tuned arms is three to six points, two to five times smaller than the gap to the untuned model, and a single-adapter arm sits at the top of it on six of the eight models. Composition per se is not the difficulty: a model that has seen identifier renaming and control-flow flattening separately handles them together.

Table 4. **Every adaptation on every model, in the three regimes that separate them.** Mean exact-match accuracy over the conditions in each block, equally weighted; the same numbers as the bold rows of the per-model tables in Appendix A. *seen stacks*: depth-2/3/4 compositions of the seen transforms; seven of its ten conditions include S3 or S4, which the mixture has experts for and *breadth* and *merge* do not. — no arm was trained on it. *backward*: input prediction, graded by execution. *base* is the untuned model, *clean* a LoRA tuned on unobfuscated code, *breadth* one LoRA tuned on all six training conditions at once, *KL* the KL-consistency objective, *router* a learned-gate mixture of eight per-condition specialists, and *TIES* / *DARE* the two sign-consensus merges of the six specialists that breadth pools — *DARE* additionally applies the random drop of Eq. 4. *breadth* and both merges are exactly data-matched; *mix* carries two further experts (S3, S4) that neither is trained on. **Bold** is the best non-base arm in that block; **red** is below base. ‘—’: no cell in the block survived the 0.25 format gate. **Read the bold across the blocks**: on seen stacks it falls on the single-adaptor arms, and on the two out-of-distribution blocks it moves to the arms composed from per-transform specialists.

model	seen stacks						unseen family						backward (input pred.)								
	base	clean	breadth	KL	router	TIES	DARE	base	clean	breadth	KL	router	TIES	DARE	base	clean	breadth	KL	router	TIES	DARE
CodeLlama-7B	0.143	0.323	0.362	0.374	0.359	0.270	0.331	0.120	0.259	0.243	0.280	0.272	0.225	0.266	0.282	0.266	0.249	0.287	0.287	0.278	0.311
CodeLlama-13B	0.170	0.354	0.390	0.402	0.398	0.298	0.373	0.145	0.281	0.271	0.297	0.305	0.238	0.295	0.294	0.312	—	0.223	0.294	0.304	0.337
CodeLlama-34B	0.161	0.390	0.426	0.452	0.431	0.350	0.402	0.139	0.307	0.302	0.337	0.330	0.281	0.314	0.319	0.295	—	0.260	—	0.372	0.381
Llama-3.1-8B	0.173	0.323	0.363	0.383	0.387	0.288	0.352	0.135	0.243	0.233	0.260	0.279	0.211	0.263	0.322	0.282	0.264	0.273	0.296	0.358	0.361
StarCoder2-15B	—	0.422	0.449	0.475	0.452	0.363	0.417	—	0.339	0.332	0.349	0.358	0.286	0.344	0.361	0.335	0.295	0.336	0.350	0.354	0.376
Gemma-3-12B	—	0.383	0.431	0.445	0.436	0.351	0.402	0.170	0.290	0.276	0.296	0.309	0.256	0.295	0.372	0.345	0.385	0.355	0.406	0.388	0.403
CodeGemma-7B	0.155	0.343	0.391	0.391	0.400	0.254	0.351	0.149	0.266	0.271	0.262	0.294	0.193	0.267	0.218	0.303	0.284	—	0.284	0.249	0.287
Granite-3.1-8B	0.234	0.307	0.359	0.353	0.359	0.291	0.324	0.168	0.218	0.214	0.204	0.256	0.205	0.234	0.285	0.202	0.217	0.150	0.247	0.324	0.270

This is the regime prior work reports, and if the evaluation stopped here the conclusion would be that the choice of adaptation barely matters and the monolithic one is fine. The next two blocks are the reason it does not stop here.

6.2 Pooled Training Is the Worst Use of the Data on an Unseen Family

The *unseen family* block holds the held-out obfuscator and every stack containing it. No arm was trained on it, and the ordering changes: breadth — one adapter trained on the pooled data of all six training conditions — is below *every* arm assembled from those same conditions’ specialists, on all eight models.

Paired by program with a cluster bootstrap over 2,000 resamples, and measured on the held-out family itself, breadth is below the router system by 3.3–5.9 points on 8 of 8 models (significant on all 8), below the merge by 2.3–5.2 on 8 of 8 (significant on 7), and below a LoRA trained on *unobfuscated code alone* by 2.7–4.7 on 8 of 8 (significant on 6).

Measured over the whole block — the family plus the six stacks that contain it — the same orderings hold but the margins shrink: breadth is below the router system on 8 of 8 (significant on all 8, 2.5–4.6 points), below the merge on 7 of 8 (significant on 5, 1.8–3.0), and below clean-code tuning on 7 of 8 but significantly on only 2 (1.3–1.6). The dilution is itself informative. A stack that contains the unseen family also contains transforms breadth was trained on, and breadth handles those well; its deficit is concentrated where the input is purely unfamiliar and fades as the familiar returns. That is what memorising a transform looks like, and it is not what an invariance would predict.

The merge comparison is exactly data-matched: its six ingredient adapters are trained on precisely the six conditions breadth pools, at the same rank, from the same base model, so the only difference is whether those rows were learned in one set of weights or in six that are then composed. The router system is *not* data-matched — it carries two further specialists, on the transforms S3 and S4, that neither breadth nor the merge is trained on. Its advantage does not come from them: measured against the merge, it is at least as large on composites that contain neither S3 nor S4 (+2.8 to +5.3)

as on those that do (+2.2 to +4.7), on all eight models. We nonetheless treat the exactly-matched merge comparison as the load-bearing one.

An adapter that has never seen an obfuscated program generalises better to an unseen obfuscator than one trained on five of them. Training on more transforms made the model worse at a transform it had not seen — which is what transform memorisation predicts and what invariance would forbid.

6.3 Single Adapters Lose the Untrained Direction

The backward task asks for a call that produces a given return value, on the same programs. No arm is trained in this direction, so it measures what adaptation did to program understanding rather than to task fitting.

Every single-adapter arm goes red somewhere in the backward block, and the pattern is systematic rather than model-specific. Pooling every backward condition that clears the format gate and pairing by program, clean-code tuning is below the untuned model on six of eight models (significantly on four), breadth on four of the six where it is readable (significantly on four), and the KL-consistency arm on six (significantly on five, by as much as 13.2 points on Granite-3.1-8B). The merges are the exception: DARE-TIES is *above* the untuned model on seven of eight models (+3.0 to +6.4 points, significant on all seven) and null on the eighth, and TIES is significantly above on six, including the model where DARE-TIES is null. Neither merge is significantly below the untuned model on any model in any of the three blocks. The arm that wins the seen-stack block is the clearest loser here.

Two qualifications belong with that result rather than in a footnote. Backward is graded by execution — any call returning the gold value counts, because inversion is many-to-one — and 15 % of items have a trivial gold (0, None, [], ...) that a well-formed guess can hit. On Llama-3.1-8B, whose untuned backward score is the panel's second highest, that accounts for nearly all of its lead: by exact-argument recovery it is 0.086 against CodeLlama-7B's 0.084, the same model. The merge's advantage survives the stricter grade on that model; some other differences do not, and we report both gradings in Appendix A.

6.4 Direction Locking Explains the Backward Failures

The backward failures are not parse failures. When a single-adapter arm is asked for an input and produces something ungradable, the reply is most often *exactly the gold forward answer* — the return value of the program, correctly computed, in answer to a question that asked for something else. The arm did not lose the ability to read the program; it lost the ability to be asked a different question about it.

Measured as the share of backward replies that are exactly the gold forward answer, the untuned models sit at the floor (0.000–0.056) and so do both merges (0.000–0.002 across all eight models). Every single-adapter arm is above that floor on every model but one — the exception is clean-code tuning on Llama-3.1-8B, at 0.001 — and the panel maximum — 0.680, on CodeGemma-7B — belongs to the KL-consistency arm, which also holds the per-model maximum on five of the eight. Its extra loss term is a KL divergence to a clean-code teacher's answer-token distribution: the arm that most explicitly optimises for producing the forward answer is the arm that most often produces it when asked for something else.

This connects the two out-of-distribution blocks. Monolithic training fits the task as it was posed: these transforms, this direction. Composing separately-trained specialists in weight space does not produce that fit, which is why the merge gives up three to six points where the fit pays and keeps its competence where the fit does not transfer.

7 Discussion

7.1 Cross-View Aggregation of Program Reasoning

The forward and backward tasks are two views of one object: the same program, the same gold value, the same grader, asked from opposite ends. That is what makes their disagreement informative. An arm that improves the forward view and degrades the backward one on the same programs has not become better at reading those programs; it has become better at producing the answer it was trained to produce. Reading each view alone hides this — every adaptation in Table 4 looks like progress forward — and only aggregating across views exposes it.

The aggregate we propose is simple and strict: an adaptation should be at or above the untuned model in *every* view, on *every* model, before it is called an improvement in understanding rather than in task fit. Under that criterion exactly one arm in this study passes on all eight models, the sign-consensus merge (Section 6), and it does so without being the best arm in any single view. Two properties of this criterion are worth stating. It is conservative in the right direction: it can only remove arms from the winner’s list, never add one. And it costs nothing to apply beyond evaluating in more than one direction, which any behaviour-prediction benchmark can do, since the pairing is a property of the items rather than of the model.

7.2 Choosing an Adaptation Strategy

No arm dominates, and the table is more useful than a ranking. If the deployment distribution is known and fixed, the KL-consistency arm and the router system are the strongest forward arms and the three to six points are real. If it is not — if an obfuscator may be updated, transforms may be composed in unseen ways, or the model may be asked something other than what it was tuned on — the merge is the only arm in this study that is never significantly below the untuned model in any block on any model, and it costs one adapter and no inference-time machinery. The router system matches or beats it on the unseen family but needs eight adapters and a gate at inference, and its learned gate is worth nothing over a uniform one on the held-out family.

7.3 The Limits of Training Breadth

The intuition behind pooled training is that a model shown more transforms learns something more general. The results say the opposite in both out-of-distribution regimes. On the held-out family, pooled training is below every arm composed from the same data on all eight models and below an adapter that never saw an obfuscated program on six. Backwards, it is below the untuned model on four of the six models where it is readable. Neither is what an invariance would predict; both are what memorising a transform’s surface predicts, since the surface is exactly what an unseen obfuscator is free to change.

The point is not that breadth is useless — on seen compositions it is within a few points of the best arm — but that its gains are confined to the distribution that produced them, and that the same data, learned as separate specialists and composed afterwards, does not have that confinement. The difference between the two is not the data or the capacity; it is whether the conditions were learned in one set of weights or in six. Where the deployment distribution is known and fixed that distinction may not matter. Where an obfuscator can be updated, it is the whole question.

8 Related Work

8.1 Comprehension Under Obfuscation and Deobfuscation

Obfuscation has a standard taxonomy of lexical, control and data transforms [4, 5], and its effect on *human* comprehension is measured: Nguyen et al. show it significantly reduces analysts’ accuracy at predicting program outputs [17]. For language models, recent work reports the same degradation

under semantics-preserving transforms, including on output prediction [?], and a decline that grows with obfuscation complexity [19]. Our evaluation extends that line in two ways: composition, since prior work evaluates single transforms and real obfuscators stack them, and direction, since we ask the reverse question on the same programs.

We deliberately do not join the deobfuscation lineage, in which recovering the original source from its obfuscated form is the training objective itself [22]. That is a different question: a model that can undo a transform tells us about the transform, whereas a model that answers correctly on the transformed program tells us about its understanding of the computation. Every evaluation in this paper is on code that stays obfuscated.

8.2 Model Merging and Modular Adaptation

Low-rank adaptation [11] made per-task adapters cheap enough that combining them became a design question in its own right. Weight-space merging answers it without further training. Task arithmetic showed that the difference between fine-tuned and pre-trained weights is a vector that can be added and negated to steer behaviour [13]; TIES resolves interference between task vectors by trimming and sign election [24], and DARE shows that fine-tuning deltas are redundant enough to drop most entries and rescale the rest [25]. Both are used here unmodified. Our contribution to this literature is not a merging method but a comparison of *where* merging sits relative to pooled training and routing under distribution shift — and the finding that its advantage appears precisely where the training distribution stops, which the merging literature, evaluated in-distribution, does not exercise.

8.3 Direction Specificity in Code Models

Benchmarks that pose both input and output prediction on the same functions established that current code models are imperfect at both and that the two are not interchangeable [8]. We use that pairing as an instrument rather than a benchmark: because the two directions share a program and a gold value, an adaptation that improves one and damages the other has changed what it produces rather than what it understands. The forward-collapse measurement — the rate at which a model asked for an input returns the output — makes that specificity directly observable, and to our knowledge it has not previously been used to separate task fitting from program understanding in adapted code models.

9 Threats to Validity

The held-out family is a proxy. Every unseen-family number is measured on X1, a trainable sibling of the quarantined family H1, whose budget was spent under a pre-registered two-pass discipline; X1 reproduces H1’s difficulty at $r = 0.9992$ and transfer between them was lossless (0.08 points) for arms trained on the sibling. It is a single family pair; a second hard family with a different surface would settle whether “family” means “surface family”. **Coverage is not uniform.** Conditions cover different program sets; every contrast runs on a subset recorded before evaluation, and where the intersection moves an estimate we report both. **Eight models is a small sample for a claim about models;** the panel’s strongest non-lineage model is also the one an untuned capability screen would have discarded. **The backward grade is permissive.** Inversion is many-to-one, so a produced call is correct if it returns the gold value; 15 % of items have a trivial gold that a well-formed guess can hit, and on one model (Llama-3.1-8B) that accounts for almost the whole of an apparently strong untuned backward score. Appendix A reports exact-argument recovery alongside execution grading, and the merge’s advantage survives the stricter grade where we can measure it. **Format gating removes cells, not at random.** A cell whose format-failure rate exceeds 0.25 is excluded from every mean; those cells concentrate in the backward direction and on

the weaker models, so the backward block is read on fewer cells than the forward one and its means are correspondingly less stable. Gated cells are marked rather than hidden. **Merge density was tuned off-panel.** The DARE-TIES density of 0.5 was selected by a sweep on the 1.5 B pilot model and applied unchanged to all eight panel models; it is therefore not fitted to the panel, but neither is it tuned for it. **Instrument failures are reported, not omitted.** The causal attention instrument is inert (an identifier knockout moves the gold log-probability by under 0.5 % everywhere), so any attention account would be correlational; we do not make one. **Single language.** Every headline is Python; the JavaScript corpus exists but cannot be regenerated on our cluster.

10 Conclusion

Fine-tuning a code model on obfuscated programs roughly doubles its accuracy on them, and we asked what that gain is made of by evaluating in three regimes rather than one: compositions of the transforms the model saw, an obfuscator family it never saw, and the same programs queried in the reverse direction. On seen compositions every adaptation works and the choice between them is worth a few points. Outside them the ordering inverts. Training one adapter on everything — the obvious use of the data — is the worst arm on the unseen family and loses the untrained direction on most models; the failures are not malformed answers but the trained answer to the wrong question. Merging per-transform specialists in weight space, an existing technique applied without modification, is the one arm at or above the untuned model in every regime on every one of eight models, at a cost of a few forward points where the training distribution is the right fit.

We take three things from this. Evaluating adaptation in a single view can support the opposite conclusion to evaluating it in several, and the pairing of directions is a cheap way to add a view to any behaviour-prediction benchmark. Breadth of training data does not buy breadth of competence; how the data is combined does. And for a deployment where the obfuscator can change, the recommendation is the boring one: train specialists, merge them, and deploy one adapter.

11 Data Availability

Data/code is available in project's website [\[\]](#).

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.14165 <https://arxiv.org/abs/2005.14165>
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [3] CodeGemma Team, Google. 2024. CodeGemma: Open Code Models Based on Gemma. arXiv:2406.11409 [cs.CL] <https://arxiv.org/abs/2406.11409>
- [4] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A Taxonomy of Obfuscating Transformations. *Technical Report* (1997).
- [5] Christian Collberg, Clark Thomborson, and Douglas Low. 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '98). Association for Computing Machinery, New York, NY, USA, 184–196. doi:10.1145/268946.268962
- [6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [7] Gemma Team, Google DeepMind. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] <https://arxiv.org/abs/2503.19786>
- [8] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution. arXiv:2401.03065 [cs.SE] <https://arxiv.org/abs/2401.03065>
- [9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie LIU, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML] <https://arxiv.org/abs/1503.02531>
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] <https://arxiv.org/abs/2106.09685>
- [12] IBM Granite Team. 2024. Granite 3.1 Language Models. <https://huggingface.co/ibm-granite/granite-3.1-8b-instruct>. IBM Research.
- [13] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. 2023. Editing Models with Task Arithmetic. In *International Conference on Learning Representations (ICLR)*. arXiv:2212.04089 <https://arxiv.org/abs/2212.04089>
- [14] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158
- [15] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [16] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz

- Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 [cs.SE] <https://arxiv.org/abs/2402.19173>
- [17] Anh H. N. Nguyen, Jack Le, Ilse Lahnstein Coronado, and Tien N. Nguyen. 2026. The Effect of Code Obfuscation on Human Program Comprehension. arXiv:2603.07668 [cs.SE] <https://arxiv.org/abs/2603.07668>
- [18] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. arXiv:2203.13474 [cs.LG] <https://arxiv.org/abs/2203.13474>
- [19] Serge Lionel Nikiema, Jordan Samhi, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Code Barrier: What LLMs Actually Understand? arXiv:2504.10557 [cs.SE] <https://arxiv.org/abs/2504.10557>
- [20] Philip O’Kane, Sakir Sezer, and Kieran McLaughlin. 2011. Obfuscation: The Hidden Malware. *IEEE Security and Privacy* 9, 5 (2011), 41–47. doi:10.1109/MSP.2011.98
- [21] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jeremy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Defossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL]
- [22] Baptiste Rozière, Marie-Anne Lachaux, Marc Szafraniec, and Guillaume Lample. 2021. DOBF: A Deobfuscation Pre-Training Objective for Programming Languages. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2102.07492 <https://arxiv.org/abs/2102.07492>
- [23] Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. 2019. An Observational Investigation of Reverse Engineers’ Process and Mental Models. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems (Glasgow, Scotland Uk) (CHI EA ’19)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3290607.3313040
- [24] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. TIES-Merging: Resolving Interference When Merging Models. arXiv:2306.01708 [cs.LG] <https://arxiv.org/abs/2306.01708>
- [25] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch. arXiv:2311.03099 [cs.CL] <https://arxiv.org/abs/2311.03099>

A Per-model absolute tables

Table 4 reports each adaptation as a mean over a block of conditions. This appendix gives the cells those means are taken over: every method against every obfuscation, one table per model, for the eight panel models. Each cell reads *output prediction / input prediction*. Bold rows are the block means, and an accuracy is coloured where it sits above or below the untuned model in the same direction; a red number carrying a dagger is red for the other reason, its format-failure rate is above 0.25 and the accuracy beneath it is not a fair reading of the arm. CodeLlama-7B (Table 5) is the one model on which every arm exists.

Table 5. **Every method against every obfuscation, CodeLlama-7B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.257 / 0.285	0.302 / -	0.427 / 0.324	0.413 / 0.276	0.422 / 0.297	0.429 / 0.305	0.366 / 0.277	0.426 / 0.329
L1b	0.198 / 0.265	0.255 / -	0.358 / 0.251	0.388 / 0.279	0.402 / 0.294	0.381 / 0.300	0.299 / 0.270	0.373 / 0.297
L1r	0.207 / 0.315	0.265 / -	0.378 / 0.327	0.387 / 0.276	0.395 / 0.293	0.383 / 0.312	0.323 / 0.327	0.381 / 0.352
L2	0.202 / 0.282	0.272 / -	0.382 / 0.324	0.378 / 0.276	0.398 / 0.275	0.386 / 0.328	0.317 / 0.268	0.375 / 0.342
S1	0.168 / 0.261	0.216 / -	0.379 / 0.188†	0.389 / 0.262	0.400 / 0.280	0.395 / 0.255	0.322 / 0.266	0.376 / 0.308
S2	0.193 / 0.330	0.215† / -	0.389 / 0.209†	0.403 / 0.294	0.415 / 0.332	0.421 / 0.293	0.340 / 0.326	0.392 / 0.357
X1	0.119 / 0.257	0.154 / -	0.269 / 0.221†	0.231 / 0.237	0.286 / 0.281	0.279 / 0.206†	0.234 / 0.255	0.276 / 0.280
C_L1b_S1	0.141 / 0.218	0.153 / -	0.279 / 0.163†	0.345 / 0.242	0.360 / 0.270	0.325 / 0.222	0.245 / 0.240	0.302 / 0.280
C_L1r_S1	0.131 / 0.301	0.179 / -	0.294 / 0.209†	0.350 / 0.255	0.356 / 0.270	0.346 / 0.227†	0.252 / 0.294	0.301 / 0.316
C_S1_L1r	0.135 / 0.282	0.168 / -	0.285 / 0.205	0.326 / 0.267	0.349 / 0.285	0.318 / 0.242	0.240 / 0.282	0.301 / 0.291
C_L2_S4	0.176 / 0.291	0.239 / -	0.355 / 0.231†	0.385 / 0.260	0.392 / 0.283	0.383 / 0.309	0.298 / 0.291	0.363 / 0.349
C_L1r_S3	0.172 / 0.296	0.201 / -	0.360 / 0.267	0.380 / 0.278	0.387 / 0.315	0.391 / 0.303	0.306 / 0.301	0.368 / 0.350
C_S4_S3	0.186 / 0.315	0.223† / -	0.392 / 0.201†	0.403 / 0.282	0.422 / 0.333	0.415 / 0.287	0.336 / 0.306	0.398 / 0.352
C3_L1r_S3_S4	0.150 / 0.303	0.181 / -	0.340 / 0.191†	0.377 / 0.268	0.380 / 0.302	0.375 / 0.249†	0.296 / 0.296	0.349 / 0.336
C3_S1_S3_S4	0.128 / 0.292	0.160 / -	0.354 / 0.166†	0.369 / 0.234	0.385 / 0.313	0.377 / 0.193†	0.274 / 0.275	0.340 / 0.321
C3_L1r_S1_S4	0.126 / 0.299	0.167 / -	0.289 / 0.163†	0.353 / 0.229	0.354 / 0.278	0.325 / 0.181†	0.239 / 0.289	0.296 / 0.314
C4_L1r_S1_S3_S4	0.089 / 0.282	0.122 / -	0.278 / 0.183†	0.336 / 0.217	0.356 / 0.288	0.334 / 0.176†	0.214 / 0.274	0.290 / 0.296
C_L1r_X1	0.102 / 0.253	0.121 / -	0.184 / 0.210	0.199 / 0.200	0.222 / 0.250	0.189 / 0.176†	0.171 / 0.256	0.193 / 0.255
C_X1_S1	0.104 / 0.251	0.131 / -	0.274 / 0.139†	0.230 / 0.201	0.267 / 0.276	0.291 / 0.144†	0.223 / 0.244	0.290 / 0.285
C_S2_X1	0.145 / 0.311	0.184 / -	0.313 / 0.153†	0.285 / 0.217	0.332 / 0.305	0.326 / 0.209†	0.265 / 0.302	0.317 / 0.326
C_L1r_X1m	0.112 / 0.250	0.116 / -	0.220 / 0.220	0.232 / 0.223	0.249 / 0.250	0.227 / 0.192†	0.190 / 0.236	0.215 / 0.253
C_S1_X1s	0.151 / 0.277	0.206 / -	0.334 / 0.144†	0.302 / 0.195	0.355 / 0.288	0.350 / 0.168†	0.301 / 0.253	0.338 / 0.286
C3_L1r_S1_X1	0.104 / 0.263	0.127 / -	0.219 / 0.139†	0.222 / 0.154†	0.250 / 0.231	0.242 / 0.097†	0.191 / 0.258	0.232 / 0.288
single obfuscation	0.194 / 0.291	0.252 ⁴ / -	0.377 / 0.301 ³	0.389 / 0.277	0.402 / 0.295	0.393 / 0.297	0.320 / 0.291	0.380 / 0.331
held-out family	0.119 / 0.257	0.154 / -	0.269 / -	0.231 / 0.237	0.286 / 0.281	0.279 / -	0.234 / 0.255	0.276 / 0.280
depth 2, all seen	0.157 / 0.284	0.188 ⁵ / -	0.327 / 0.236 ²	0.365 / 0.264	0.378 / 0.293	0.363 / 0.273 ⁵	0.280 / 0.286	0.339 / 0.323
depth 3, all seen	0.135 / 0.298	0.169 / -	0.328 / -	0.366 / 0.244	0.373 / 0.298	0.359 / -	0.270 / 0.287	0.328 / 0.324
depth 4, all seen	0.089 / 0.282	0.122 / -	0.278 / -	0.336 / 0.217	0.356 / 0.288	0.334 / -	0.214 / 0.274	0.290 / 0.296
depth 2, unseen inside	0.123 / 0.268	0.152 / -	0.265 / 0.215 ²	0.249 / 0.207	0.285 / 0.274	0.277 / -	0.230 / 0.258	0.271 / 0.281
depth 3, unseen inside	0.104 / 0.263	0.127 / -	0.219 / -	0.222 / -	0.250 / 0.231	0.242 / -	0.191 / 0.258	0.232 / 0.288

Table 6. **Every method against every obfuscation, CodeLlama-13B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.252 / 0.285	0.278 [†] / -	0.469 / 0.269	0.446 / 0.066 [†]	0.462 / 0.231	0.477 / 0.232 [†]	0.408 / 0.284	0.477 / 0.339
L1b	0.222 / 0.277	0.220 [†] / -	0.397 / 0.285	0.423 / 0.066 [†]	0.438 / 0.253	0.426 / 0.304	0.338 / 0.287	0.400 / 0.318
L1r	0.225 / 0.327	0.233 [†] / -	0.415 / 0.347	0.420 / 0.051 [†]	0.439 / 0.229	0.428 / 0.274	0.374 / 0.333	0.425 / 0.366
L2	0.228 / 0.279	0.244 [†] / -	0.407 / 0.232 [†]	0.417 / 0.026 [†]	0.432 / 0.206	0.424 / 0.152 [†]	0.365 / 0.289	0.421 / 0.318
S1	0.213 / 0.304	0.216 / -	0.408 / 0.341	0.413 / 0.059 [†]	0.435 / 0.199	0.435 / 0.294	0.368 / 0.327	0.435 / 0.363
S2	0.184 / 0.293	0.187 [†] / -	0.422 / 0.317	0.445 / 0.038 [†]	0.469 / 0.238	0.453 / 0.288 [†]	0.353 / 0.317	0.441 / 0.365
X1	0.149 / 0.258	0.141 [†] / -	0.294 / 0.274	0.254 / 0.016 [†]	0.292 / 0.219	0.313 / 0.246	0.241 / 0.272	0.306 / 0.298
C_L1b_S1	0.183 / 0.253	0.159 / -	0.318 / 0.326	0.367 / 0.059 [†]	0.375 / 0.201	0.370 / 0.298	0.288 / 0.281	0.329 / 0.341
C_L1r_S1	0.175 / 0.325	0.175 / -	0.318 / 0.360	0.371 / 0.070 [†]	0.376 / 0.203	0.371 / 0.303	0.287 / 0.333	0.356 / 0.368
C_S1_L1r	0.172 / 0.323	0.172 / -	0.315 / 0.351	0.359 / 0.086 [†]	0.363 / 0.230	0.370 / 0.310	0.277 / 0.344	0.341 / 0.376
C_L2_S4	0.200 / 0.274	0.187 [†] / -	0.389 / 0.186 [†]	0.422 / 0.020 [†]	0.432 / 0.168 [†]	0.421 / 0.132 [†]	0.329 / 0.284	0.404 / 0.321
C_L1r_S3	0.186 / 0.305	0.194 [†] / -	0.407 / 0.361	0.409 / 0.041 [†]	0.434 / 0.220	0.428 / 0.325	0.335 / 0.311	0.417 / 0.366
C_S4_S3	0.192 / 0.293	0.211 [†] / -	0.424 / 0.326	0.436 / 0.026 [†]	0.459 / 0.239	0.464 / 0.288 [†]	0.355 / 0.306	0.443 / 0.363
C3_L1r_S3_S4	0.166 / 0.310	0.169 [†] / -	0.382 / 0.337	0.410 / 0.032 [†]	0.429 / 0.224	0.416 / 0.283 [†]	0.316 / 0.314	0.394 / 0.353
C3_S1_S3_S4	0.147 / 0.295	0.158 / -	0.367 / 0.329	0.408 / 0.035 [†]	0.423 / 0.235	0.419 / 0.312	0.300 / 0.300	0.387 / 0.347
C3_L1r_S1_S4	0.163 / 0.316	0.156 / -	0.314 / 0.353	0.367 / 0.061 [†]	0.367 / 0.211	0.362 / 0.327	0.268 / 0.324	0.346 / 0.370
C4_L1r_S1_S3_S4	0.117 / 0.310	0.117 / -	0.300 / 0.333	0.353 / 0.049 [†]	0.360 / 0.222	0.363 / 0.332	0.230 / 0.320	0.320 / 0.353
C_L1r_X1	0.123 / 0.281	0.115 [†] / -	0.199 / 0.254	0.213 / 0.016 [†]	0.236 / 0.170 [†]	0.227 / 0.187 [†]	0.181 / 0.283	0.217 / 0.266
C_X1_S1	0.138 / 0.289	0.135 / -	0.295 / 0.264	0.270 / 0.007 [†]	0.307 / 0.201 [†]	0.320 / 0.233	0.243 / 0.293	0.310 / 0.319
C_S2_X1	0.160 / 0.271	0.167 [†] / -	0.335 / 0.279	0.334 / 0.008 [†]	0.356 / 0.204 [†]	0.360 / 0.217 [†]	0.281 / 0.305	0.353 / 0.329
C_L1r_X1m	0.138 / 0.291	0.127 [†] / -	0.246 / 0.259	0.248 / 0.020 [†]	0.279 / 0.179 [†]	0.267 / 0.199 [†]	0.200 / 0.274	0.251 / 0.283
C_S1_X1s	0.179 / 0.301	0.166 / -	0.364 / 0.290	0.345 / 0.025 [†]	0.360 / 0.199 [†]	0.390 / 0.267	0.311 / 0.309	0.376 / 0.317
C3_L1r_S1_X1	0.128 / 0.304	0.107 [†] / -	0.233 / 0.293	0.234 / 0.026 [†]	0.249 / 0.185 [†]	0.254 / 0.203 [†]	0.207 / 0.305	0.251 / 0.313
single obfuscation	0.215 / 0.296	0.216 ¹ / -	0.410 / 0.323 ⁴	0.423 / -	0.442 / 0.225	0.433 / 0.291 ³	0.359 / 0.311	0.424 / 0.346
held-out family	0.149 / 0.258	- / -	0.294 / 0.274	0.254 / -	0.292 / 0.219	0.313 / 0.246	0.241 / 0.272	0.306 / 0.298
depth 2, all seen	0.185 / 0.295	0.168 ³ / -	0.362 / 0.345 ⁵	0.394 / -	0.407 / 0.219 ⁵	0.404 / 0.309 ⁴	0.312 / 0.310	0.381 / 0.356
depth 3, all seen	0.159 / 0.307	0.157 ² / -	0.354 / 0.340	0.395 / -	0.406 / 0.223	0.399 / 0.320 ²	0.295 / 0.313	0.375 / 0.357
depth 4, all seen	0.117 / 0.310	0.117 / -	0.300 / 0.333	0.353 / -	0.360 / 0.222	0.363 / 0.332	0.230 / 0.320	0.320 / 0.353
depth 2, unseen inside	0.148 / 0.286	0.151 ² / -	0.288 / 0.269	0.282 / -	0.307 / -	0.313 / 0.250 ²	0.243 / 0.293	0.301 / 0.303
depth 3, unseen inside	0.128 / 0.304	- / -	0.233 / 0.293	0.234 / -	0.249 / -	0.254 / -	0.207 / 0.305	0.251 / 0.313

Table 7. **Every method against every obfuscation, CodeLlama-34B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.254 [†] / 0.363	0.178 [†] / -	0.520 / 0.260	0.494 / 0.200 [†]	0.525 / 0.284	0.526 / 0.239 [†]	0.484 / 0.396	0.527 / 0.382
L1b	0.205 / 0.327	0.077 [†] / -	0.417 / 0.253	0.469 / 0.174 [†]	0.494 / 0.287	0.472 / 0.259 [†]	0.401 / 0.369	0.429 / 0.360
L1r	0.220 / 0.339	0.117 [†] / -	0.469 / 0.255	0.457 / 0.135 [†]	0.487 / 0.256	0.481 / 0.170 [†]	0.420 / 0.378	0.467 / 0.373
L2	0.231 / 0.357	0.109 [†] / -	0.466 / 0.280	0.465 / 0.179 [†]	0.478 / 0.276	0.475 / 0.200 [†]	0.403 / 0.388	0.463 / 0.378
S1	0.222 / 0.323	0.079 [†] / -	0.472 / 0.305	0.467 / 0.178 [†]	0.496 / 0.264	0.484 / 0.241 [†]	0.419 / 0.377	0.456 / 0.390
S2	0.191 / 0.350	0.040 [†] / -	0.484 / 0.354	0.484 / 0.219 [†]	0.525 / 0.306	0.509 / 0.293 [†]	0.439 / 0.417	0.484 / 0.437
X1	0.135 / 0.303	0.050 [†] / -	0.325 / 0.253	0.298 / 0.118 [†]	0.334 / 0.249	0.344 / 0.114 [†]	0.299 / 0.352	0.328 / 0.338
C_L1b_S1	0.156 / 0.257	0.033 [†] / -	0.322 / 0.272	0.399 / 0.144 [†]	0.421 / 0.258	0.390 / 0.221 [†]	0.295 / 0.345	0.331 / 0.358
C_L1r_S1	0.184 / 0.304	0.063 [†] / -	0.351 / 0.285	0.393 / 0.132 [†]	0.427 / 0.200	0.415 / 0.145 [†]	0.332 / 0.353	0.362 / 0.375
C_S1_L1r	0.180 / 0.292	0.053 [†] / -	0.367 / 0.307	0.405 / 0.142 [†]	0.425 / 0.216	0.387 / 0.163 [†]	0.324 / 0.360	0.377 / 0.371
C_L2_S4	0.191 / 0.344	0.045 [†] / -	0.431 / 0.291	0.454 / 0.170 [†]	0.478 / 0.273	0.452 / 0.243 [†]	0.380 / 0.386	0.436 / 0.384
C_L1r_S3	0.169 / 0.333	0.041 [†] / -	0.451 / 0.327	0.454 / 0.167 [†]	0.482 / 0.256	0.471 / 0.197 [†]	0.407 / 0.403	0.453 / 0.419
C_S4_S3	0.199 / 0.352	0.041 [†] / -	0.482 / 0.345	0.487 / 0.218 [†]	0.521 / 0.301	0.505 / 0.276 [†]	0.435 / 0.418	0.488 / 0.442
C3_L1r_S3_S4	0.148 / 0.325	0.017 [†] / -	0.416 / 0.330	0.452 / 0.182 [†]	0.473 / 0.251	0.449 / 0.201 [†]	0.374 / 0.394	0.425 / 0.427
C3_S1_S3_S4	0.128 / 0.330	0.042 [†] / -	0.428 / 0.323	0.454 / 0.208 [†]	0.472 / 0.288	0.469 / 0.235 [†]	0.366 / 0.398	0.442 / 0.413
C3_L1r_S1_S4	0.147 / 0.291	0.041 [†] / -	0.338 / 0.311	0.390 / 0.121 [†]	0.412 / 0.199	0.391 / 0.134 [†]	0.307 / 0.358	0.354 / 0.381
C4_L1r_S1_S3_S4	0.105 / 0.283	0.033 [†] / -	0.316 / 0.333	0.372 / 0.164 [†]	0.413 / 0.214	0.380 / 0.175 [†]	0.281 / 0.379	0.351 / 0.386
C_L1r_X1	0.115 / 0.275	0.044 [†] / -	0.226 / 0.240	0.249 / 0.060 [†]	0.278 / 0.162 [†]	0.263 / 0.030 [†]	0.214 / 0.304	0.236 / 0.299
C_X1_S1	0.125 / 0.298	0.064 [†] / -	0.317 / 0.275	0.290 / 0.089 [†]	0.339 / 0.216 [†]	0.342 / 0.091 [†]	0.297 / 0.352	0.317 / 0.345
C_S2_X1	0.176 / 0.378	0.099 [†] / -	0.372 / 0.366	0.355 / 0.196 [†]	0.403 / 0.288	0.395 / 0.178 [†]	0.336 / 0.398	0.382 / 0.414
C_L1r_X1m	0.128 / 0.269	0.052 [†] / -	0.261 / 0.230	0.275 / 0.077 [†]	0.294 / 0.158 [†]	0.296 / 0.030 [†]	0.240 / 0.309	0.279 / 0.308
C_S1_X1s	0.176 / 0.346	0.077 [†] / -	0.390 / 0.302	0.382 / 0.141 [†]	0.415 / 0.268	0.397 / 0.115 [†]	0.353 / 0.385	0.387 / 0.418
C3_L1r_S1_X1	0.118 / 0.304	0.048 [†] / -	0.256 / 0.296	0.262 / 0.084 [†]	0.297 / 0.188 [†]	0.271 / 0.021 [†]	0.230 / 0.344	0.267 / 0.357
single obfuscation	0.214 / 0.339	- / -	0.462 / 0.289	0.469 / -	0.496 / 0.278	0.484 / -	0.416 / 0.386	0.460 / 0.388
held-out family	0.135 / 0.303	- / -	0.325 / 0.253	0.298 / -	0.334 / 0.249	0.344 / -	0.299 / 0.352	0.328 / 0.338
depth 2, all seen	0.180 / 0.314	- / -	0.401 / 0.305	0.432 / -	0.459 / 0.251	0.437 / -	0.362 / 0.377	0.408 / 0.391
depth 3, all seen	0.141 / 0.315	- / -	0.394 / 0.322	0.432 / -	0.452 / 0.246	0.436 / -	0.349 / 0.384	0.407 / 0.407
depth 4, all seen	0.105 / 0.283	- / -	0.316 / 0.333	0.372 / -	0.413 / 0.214	0.380 / -	0.281 / 0.379	0.351 / 0.386
depth 2, unseen inside	0.144 / 0.313	- / -	0.313 / 0.283	0.310 / -	0.346 / 0.278 ²	0.338 / -	0.288 / 0.350	0.320 / 0.357
depth 3, unseen inside	0.118 / 0.304	- / -	0.256 / 0.296	0.262 / -	0.297 / -	0.271 / -	0.230 / 0.344	0.267 / 0.357

Table 8. **Every method against every obfuscation, Llama-3.1-8B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.258 / 0.409	0.320 / -	0.448 / 0.335	0.426 / 0.320	0.445 / 0.332	0.475 / 0.338	0.398 / 0.434	0.462 / 0.414
L1b	0.222 / 0.371	0.276 / -	0.361 / 0.297	0.387 / 0.280	0.407 / 0.285	0.422 / 0.318	0.336 / 0.389	0.386 / 0.383
L1r	0.242 / 0.373	0.284 / -	0.387 / 0.312	0.387 / 0.290	0.405 / 0.297	0.407 / 0.311	0.348 / 0.388	0.398 / 0.372
L2	0.215 / 0.376	0.278 / -	0.391 / 0.297	0.386 / 0.284	0.409 / 0.294	0.425 / 0.311	0.353 / 0.384	0.396 / 0.364
S1	0.213 / 0.375	0.256 / -	0.395 / 0.325	0.391 / 0.285	0.413 / 0.291	0.428 / 0.316	0.335 / 0.417	0.412 / 0.411
S2	0.205 / 0.378	0.272 / -	0.413 / 0.345	0.413 / 0.326	0.445 / 0.325	0.456 / 0.366	0.361 / 0.419	0.437 / 0.435
X1	0.132 / 0.263	0.161 / -	0.249 / 0.267	0.222 / 0.211	0.255 / 0.228	0.280 / 0.258	0.217 / 0.314	0.266 / 0.311
C_L1b_S1	0.152 / 0.287	0.185 / -	0.279 / 0.253	0.338 / 0.274	0.368 / 0.258	0.369 / 0.306	0.252 / 0.315	0.307 / 0.352
C_L1r_S1	0.162 / 0.297	0.220 / -	0.303 / 0.281	0.352 / 0.280	0.363 / 0.283	0.366 / 0.293	0.273 / 0.340	0.328 / 0.350
C_S1_L1r	0.159 / 0.303	0.208 / -	0.277 / 0.269	0.338 / 0.274	0.354 / 0.274	0.347 / 0.293	0.247 / 0.348	0.306 / 0.356
C_L2_S4	0.205 / 0.320	0.257 / -	0.349 / 0.275	0.386 / 0.269	0.401 / 0.286	0.413 / 0.313	0.311 / 0.358	0.383 / 0.388
C_L1r_S3	0.198 / 0.342	0.266 / -	0.367 / 0.297	0.382 / 0.299	0.398 / 0.303	0.407 / 0.320	0.334 / 0.375	0.387 / 0.375
C_S4_S3	0.214 / 0.367	0.269 / -	0.421 / 0.333	0.411 / 0.313	0.444 / 0.329	0.459 / 0.363	0.357 / 0.410	0.437 / 0.417
C3_L1r_S3_S4	0.183 / 0.309	0.233 / -	0.329 / 0.271	0.374 / 0.303	0.391 / 0.300	0.399 / 0.317	0.306 / 0.363	0.370 / 0.372
C3_S1_S3_S4	0.160 / 0.353	0.220 / -	0.359 / 0.311	0.376 / 0.289	0.393 / 0.281	0.415 / 0.311	0.308 / 0.398	0.377 / 0.408
C3_L1r_S1_S4	0.160 / 0.302	0.208 / -	0.279 / 0.270	0.342 / 0.262	0.358 / 0.271	0.353 / 0.276	0.260 / 0.344	0.319 / 0.341
C4_L1r_S1_S3_S4	0.140 / 0.281	0.191 / -	0.263 / 0.234	0.335 / 0.280	0.358 / 0.279	0.342 / 0.274	0.235 / 0.324	0.310 / 0.340
C_L1r_X1	0.100 / 0.218	0.118 / -	0.165 / 0.220	0.181 / 0.193	0.207 / 0.239	0.204 / 0.230	0.147 / 0.248	0.184 / 0.275
C_X1_S1	0.130 / 0.291	0.179 / -	0.271 / 0.283	0.225 / 0.210	0.253 / 0.220	0.294 / 0.269	0.233 / 0.341	0.296 / 0.332
C_S2_X1	0.170 / 0.323	0.212 / -	0.296 / 0.288	0.284 / 0.229	0.317 / 0.241	0.336 / 0.284	0.263 / 0.362	0.317 / 0.350
C_L1r_X1m	0.115 / 0.232	0.148 / -	0.184 / 0.210	0.216 / 0.217	0.232 / 0.245	0.238 / 0.225	0.162 / 0.276	0.217 / 0.274
C_S1_X1s	0.193 / 0.357	0.231 / -	0.341 / 0.287	0.299 / 0.205	0.335 / 0.212	0.362 / 0.276	0.287 / 0.392	0.349 / 0.376
C3_L1r_S1_X1	0.105 / 0.268	0.156 / -	0.194 / 0.234	0.207 / 0.188	0.224 / 0.194	0.238 / 0.241	0.170 / 0.298	0.215 / 0.305
single obfuscation	0.219 / 0.375	0.273 / -	0.390 / 0.315	0.393 / 0.293	0.416 / 0.299	0.427 / 0.324	0.347 / 0.399	0.406 / 0.393
held-out family	0.132 / 0.263	0.161 / -	0.249 / 0.267	0.222 / 0.211	0.255 / 0.228	0.280 / 0.258	0.217 / 0.314	0.266 / 0.311
depth 2, all seen	0.182 / 0.319	0.234 / -	0.332 / 0.284	0.368 / 0.285	0.388 / 0.289	0.394 / 0.315	0.296 / 0.358	0.358 / 0.373
depth 3, all seen	0.168 / 0.321	0.221 / -	0.322 / 0.284	0.364 / 0.285	0.380 / 0.284	0.389 / 0.301	0.291 / 0.369	0.355 / 0.374
depth 4, all seen	0.140 / 0.281	0.191 / -	0.263 / 0.234	0.335 / 0.280	0.358 / 0.279	0.342 / 0.274	0.235 / 0.324	0.310 / 0.340
depth 2, unseen inside	0.142 / 0.284	0.177 / -	0.252 / 0.258	0.241 / 0.211	0.269 / 0.231	0.287 / 0.257	0.218 / 0.324	0.273 / 0.321
depth 3, unseen inside	0.105 / 0.268	0.156 / -	0.194 / 0.234	0.207 / 0.188	0.224 / 0.194	0.238 / 0.241	0.170 / 0.298	0.215 / 0.305

Table 9. **Every method against every obfuscation, StarCoder2-15B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.000 [†] / 0.366	0.008 [†] / -	0.542 / 0.411	0.529 / 0.311	0.532 / 0.399	0.546 / 0.435	0.484 / 0.423	0.539 / 0.452
L1b	0.000 [†] / 0.216 [†]	0.000 [†] / -	0.431 / 0.316	0.487 / 0.301	0.510 / 0.395	0.489 / 0.382	0.366 / 0.356	0.426 / 0.364
L1r	0.000 [†] / 0.282 [†]	0.000 [†] / -	0.483 / 0.354	0.485 / 0.273	0.509 / 0.369	0.492 / 0.368	0.414 / 0.391	0.476 / 0.423
L2	0.000 [†] / 0.355	0.002 [†] / -	0.479 / 0.391	0.486 / 0.297	0.503 / 0.379	0.493 / 0.406	0.401 / 0.408	0.466 / 0.421
S1	0.000 [†] / 0.235 [†]	0.010 [†] / -	0.508 / 0.361	0.487 / 0.257 [†]	0.501 / 0.364	0.510 / 0.377	0.451 / 0.407	0.487 / 0.421
S2	0.000 [†] / 0.238 [†]	0.023 [†] / -	0.512 / 0.359	0.514 / 0.300	0.539 / 0.384	0.533 / 0.404	0.440 / 0.416	0.518 / 0.440
X1	0.000 [†] / 0.235 [†]	0.009 [†] / -	0.347 / 0.254 [†]	0.320 / 0.177 [†]	0.343 / 0.281	0.353 / 0.261 [†]	0.283 / 0.290	0.343 / 0.328
C_L1b_S1	0.000 [†] / 0.122 [†]	0.001 [†] / -	0.360 / 0.254 [†]	0.416 / 0.211 [†]	0.456 / 0.317	0.417 / 0.297	0.321 / 0.303	0.354 / 0.324
C_L1r_S1	0.000 [†] / 0.165 [†]	0.000 [†] / -	0.393 / 0.301	0.423 / 0.214 [†]	0.440 / 0.303	0.419 / 0.305	0.344 / 0.360	0.389 / 0.388
C_S1_L1r	0.000 [†] / 0.136 [†]	0.000 [†] / -	0.377 / 0.311	0.406 / 0.183 [†]	0.431 / 0.303	0.418 / 0.309	0.346 / 0.367	0.387 / 0.374
C_L2_S4	0.000 [†] / 0.215 [†]	0.005 [†] / -	0.454 / 0.342	0.470 / 0.260 [†]	0.508 / 0.353	0.485 / 0.378	0.381 / 0.384	0.438 / 0.395
C_L1r_S3	0.000 [†] / 0.243 [†]	0.000 [†] / -	0.473 / 0.300	0.483 / 0.282	0.504 / 0.368	0.490 / 0.323	0.390 / 0.375	0.459 / 0.383
C_S4_S3	0.000 [†] / 0.243 [†]	0.028 [†] / -	0.511 / 0.368	0.516 / 0.301	0.539 / 0.386	0.525 / 0.389	0.442 / 0.420	0.506 / 0.448
C3_L1r_S3_S4	0.000 [†] / 0.182 [†]	0.004 [†] / -	0.448 / 0.294	0.484 / 0.242 [†]	0.503 / 0.339	0.475 / 0.315	0.377 / 0.374	0.436 / 0.386
C3_S1_S3_S4	0.000 [†] / 0.191 [†]	0.019 [†] / -	0.476 / 0.337	0.465 / 0.235 [†]	0.493 / 0.333	0.480 / 0.345	0.393 / 0.370	0.464 / 0.406
C3_L1r_S1_S4	0.000 [†] / 0.096 [†]	0.002 [†] / -	0.370 / 0.293	0.417 / 0.209 [†]	0.439 / 0.270	0.413 / 0.297	0.332 / 0.360	0.376 / 0.353
C4_L1r_S1_S3_S4	0.000 [†] / 0.104 [†]	0.001 [†] / -	0.357 / 0.289	0.412 / 0.206 [†]	0.437 / 0.273	0.401 / 0.271	0.301 / 0.341	0.358 / 0.358
C_L1r_X1	0.000 [†] / 0.123 [†]	0.002 [†] / -	0.248 / 0.165 [†]	0.274 / 0.165 [†]	0.286 / 0.205 [†]	0.278 / 0.168 [†]	0.215 / 0.241	0.259 / 0.262
C_X1_S1	0.000 [†] / 0.194 [†]	0.022 [†] / -	0.357 / 0.259 [†]	0.322 / 0.127 [‡]	0.342 / 0.243 [†]	0.385 / 0.238 [†]	0.295 / 0.302	0.371 / 0.355
C_S2_X1	0.000 [†] / 0.184 [†]	0.014 [†] / -	0.403 / 0.261 [†]	0.375 / 0.196 [†]	0.397 / 0.268 [†]	0.413 / 0.276 [†]	0.341 / 0.366	0.403 / 0.384
C_L1r_X1m	0.000 [†] / 0.179 [†]	0.003 [†] / -	0.307 / 0.191 [†]	0.323 / 0.178 [†]	0.338 / 0.234	0.328 / 0.180 [†]	0.252 / 0.271	0.317 / 0.267
C_S1_X1s	0.000 [†] / 0.258 [†]	0.044 [†] / -	0.434 / 0.291 [†]	0.408 / 0.176 [†]	0.434 / 0.286 [†]	0.443 / 0.272 [†]	0.378 / 0.345	0.427 / 0.384
C3_L1r_S1_X1	0.000 [†] / 0.153 [†]	0.002 [†] / -	0.278 / 0.191 [†]	0.298 / 0.107 [‡]	0.302 / 0.167 [†]	0.306 / 0.146 [†]	0.241 / 0.285	0.286 / 0.323
single obfuscation	- / 0.355 ¹	- / -	0.482 / 0.356	0.492 / 0.293 ⁴	0.512 / 0.378	0.503 / 0.387	0.414 / 0.395	0.475 / 0.414
held-out family	- / -	- / -	0.347 / -	0.320 / -	0.343 / 0.281	0.353 / -	0.283 / 0.290	0.343 / 0.328
depth 2, all seen	- / -	- / -	0.428 / 0.325 ⁵	0.452 / 0.291 ²	0.480 / 0.338	0.459 / 0.334	0.371 / 0.368	0.422 / 0.385
depth 3, all seen	- / -	- / -	0.431 / 0.308	0.455 / -	0.478 / 0.314	0.456 / 0.319	0.367 / 0.368	0.425 / 0.382
depth 4, all seen	- / -	- / -	0.357 / 0.289	0.412 / -	0.437 / 0.273	0.401 / 0.271	0.301 / 0.341	0.358 / 0.358
depth 2, unseen inside	- / -	- / -	0.350 / -	0.341 / -	0.359 / 0.234 ¹	0.369 / -	0.296 / 0.305	0.355 / 0.330
depth 3, unseen inside	- / -	- / -	0.278 / -	0.298 / -	0.302 / -	0.306 / -	0.241 / 0.285	0.286 / 0.323

Table 10. **Every method against every obfuscation, Gemma-3-12B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.334 [†] / 0.453	0.340 [†] / -	0.520 / 0.440	0.507 / 0.447	0.502 / 0.355	0.530 / 0.474	0.481 / 0.483	0.523 / 0.481
L1b	0.262 [†] / 0.405	0.274 [†] / -	0.441 / 0.353	0.475 / 0.427	0.488 / 0.309 [†]	0.481 / 0.436	0.399 / 0.424	0.464 / 0.425
L1r	0.273 [†] / 0.420	0.283 [†] / -	0.459 / 0.394	0.463 / 0.411	0.472 / 0.280 [†]	0.477 / 0.433	0.419 / 0.437	0.474 / 0.440
L2	0.269 [†] / 0.403	0.281 [†] / -	0.456 / 0.378	0.471 / 0.405	0.478 / 0.295 [†]	0.481 / 0.424	0.414 / 0.432	0.465 / 0.420
S1	0.269 [†] / 0.424	0.289 [†] / -	0.456 / 0.375	0.457 / 0.421	0.483 / 0.256 [†]	0.496 / 0.446	0.420 / 0.443	0.479 / 0.435
S2	0.290 [†] / 0.443	0.308 [†] / -	0.500 / 0.444	0.503 / 0.457	0.504 / 0.277 [†]	0.527 / 0.486	0.457 / 0.466	0.513 / 0.484
X1	0.152 / 0.333	0.183 / -	0.310 / 0.305	0.263 / 0.333	0.289 / 0.199 [†]	0.314 / 0.351	0.256 / 0.345	0.311 / 0.371
C_L1b_S1	0.195 [†] / 0.349	0.197 [†] / -	0.333 / 0.317	0.406 / 0.366	0.420 / 0.214 [†]	0.414 / 0.375	0.307 / 0.353	0.359 / 0.396
C_L1r_S1	0.200 [†] / 0.356	0.204 [†] / -	0.336 / 0.319	0.389 / 0.379	0.416 / 0.203 [†]	0.398 / 0.386	0.317 / 0.364	0.362 / 0.386
C_S1_L1r	0.181 [†] / 0.337	0.193 [†] / -	0.334 / 0.319	0.393 / 0.379	0.414 / 0.207 [†]	0.389 / 0.403	0.295 / 0.375	0.338 / 0.405
C_L2_S4	0.249 [†] / 0.387	0.262 [†] / -	0.438 / 0.381	0.460 / 0.416	0.469 / 0.222 [†]	0.459 / 0.451	0.391 / 0.412	0.452 / 0.430
C_L1r_S3	0.260 [†] / 0.427	0.274 [†] / -	0.438 / 0.402	0.462 / 0.423	0.469 / 0.254 [†]	0.461 / 0.445	0.403 / 0.436	0.450 / 0.443
C_S4_S3	0.297 [†] / 0.434	0.308 [†] / -	0.491 / 0.438	0.506 / 0.462	0.503 / 0.274 [†]	0.518 / 0.493	0.457 / 0.445	0.496 / 0.472
C3_L1r_S3_S4	0.232 [†] / 0.398	0.256 [†] / -	0.419 / 0.386	0.454 / 0.420	0.467 / 0.223 [†]	0.455 / 0.446	0.385 / 0.417	0.440 / 0.441
C3_S1_S3_S4	0.220 [†] / 0.387	0.256 [†] / -	0.425 / 0.354	0.452 / 0.419	0.465 / 0.240 [†]	0.482 / 0.456	0.377 / 0.408	0.449 / 0.430
C3_L1r_S1_S4	0.176 [†] / 0.347	0.191 [†] / -	0.320 / 0.302	0.390 / 0.376	0.415 / 0.181 [†]	0.395 / 0.376	0.303 / 0.359	0.341 / 0.377
C4_L1r_S1_S3_S4	0.170 [†] / 0.329	0.172 [†] / -	0.302 / 0.293	0.394 / 0.364	0.413 / 0.180 [†]	0.383 / 0.383	0.275 / 0.336	0.332 / 0.370
C_L1r_X1	0.102 / 0.258	0.126 / -	0.195 / 0.242	0.206 / 0.279	0.224 / 0.121 [†]	0.213 / 0.279	0.180 / 0.277	0.203 / 0.274
C_X1_S1	0.172 / 0.348	0.193 / -	0.317 / 0.282	0.271 / 0.331	0.296 / 0.132 [†]	0.333 / 0.360	0.283 / 0.359	0.312 / 0.375
C_S2_X1	0.201 / 0.360	0.222 / -	0.368 / 0.339	0.349 / 0.357	0.364 / 0.210 [†]	0.389 / 0.402	0.299 / 0.373	0.368 / 0.395
C_L1r_X1m	0.108 [†] / 0.273	0.115 [†] / -	0.220 / 0.269	0.235 / 0.323	0.244 / 0.146 [†]	0.250 / 0.307	0.195 / 0.271	0.231 / 0.285
C_S1_X1s	0.255 / 0.412	0.275 / -	0.395 / 0.371	0.374 / 0.383	0.395 / 0.178 [†]	0.421 / 0.429	0.349 / 0.419	0.393 / 0.438
C3_L1r_S1_X1	0.136 / 0.279	0.147 [†] / -	0.225 / 0.226	0.236 / 0.270	0.258 / 0.115 [†]	0.245 / 0.297	0.228 / 0.298	0.246 / 0.300
single obfuscation	- / 0.419	- / -	0.462 / 0.389	0.474 / 0.424	0.485 / -	0.492 / 0.445	0.422 / 0.440	0.479 / 0.441
held-out family	0.152 / 0.333	0.183 / -	0.310 / 0.305	0.263 / 0.333	0.289 / -	0.314 / 0.351	0.256 / 0.345	0.311 / 0.371
depth 2, all seen	- / 0.382	- / -	0.395 / 0.363	0.436 / 0.404	0.448 / -	0.440 / 0.425	0.362 / 0.397	0.409 / 0.422
depth 3, all seen	- / 0.377	- / -	0.388 / 0.348	0.432 / 0.405	0.449 / -	0.444 / 0.426	0.355 / 0.395	0.410 / 0.416
depth 4, all seen	- / 0.329	- / -	0.302 / 0.293	0.394 / 0.364	0.413 / -	0.383 / 0.383	0.275 / 0.336	0.332 / 0.370
depth 2, unseen inside	0.182 [†] / 0.330	0.204 [†] / -	0.299 / 0.301	0.287 / 0.335	0.305 / -	0.321 / 0.356	0.261 / 0.340	0.301 / 0.353
depth 3, unseen inside	0.136 / 0.279	- / -	0.225 / 0.226	0.236 / 0.270	0.258 / -	0.245 / 0.297	0.228 / 0.298	0.246 / 0.300

Table 11. **Every method against every obfuscation, CodeGemma-7B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	TIES out / in	DARE out / in
L0	0.237 / 0.266	0.267 [†] / -	0.465 / 0.364	0.439 / 0.324	0.445 / 0.098 [‡]	0.474 / 0.323	0.359 / 0.302	0.465 / 0.329
L1b	0.185 / 0.268	0.209 [†] / -	0.382 / 0.309	0.425 / 0.298	0.435 / 0.067 [‡]	0.443 / 0.307	0.288 / 0.297	0.391 / 0.301
L1r	0.213 / 0.251	0.225 [†] / -	0.399 / 0.333	0.417 / 0.301	0.417 / 0.107 [‡]	0.426 / 0.294	0.320 / 0.268	0.411 / 0.303
L2	0.208 / 0.239	0.230 [†] / -	0.412 / 0.327	0.422 / 0.291	0.422 / 0.078 [‡]	0.440 / 0.287	0.307 / 0.251	0.406 / 0.299
S1	0.195 / 0.234	0.212 / -	0.405 / 0.357	0.415 / 0.337	0.421 / 0.125 [‡]	0.457 / 0.320	0.307 / 0.275	0.414 / 0.312
S2	0.198 / 0.270	0.242 [†] / -	0.450 / 0.372	0.429 / 0.351	0.448 / 0.126 [‡]	0.479 / 0.359	0.332 / 0.309	0.448 / 0.330
X1	0.143 / 0.190	0.157 / -	0.271 / 0.260	0.251 / 0.217	0.247 / 0.049 [‡]	0.301 / 0.241	0.208 / 0.226	0.274 / 0.264
C_L1b_S1	0.148 / 0.229	0.163 / -	0.286 / 0.274	0.372 / 0.276	0.368 / 0.089 [‡]	0.365 / 0.267	0.214 / 0.255	0.302 / 0.297
C_L1r_S1	0.144 / 0.170	0.144 / -	0.315 / 0.289	0.373 / 0.290	0.367 / 0.086 [‡]	0.364 / 0.278	0.229 / 0.240	0.318 / 0.271
C_S1_L1r	0.132 / 0.168	0.149 / -	0.288 / 0.284	0.363 / 0.316	0.361 / 0.099 [‡]	0.350 / 0.286	0.217 / 0.242	0.300 / 0.281
C_L2_S4	0.176 / 0.239	0.208 [†] / -	0.391 / 0.318	0.423 / 0.299	0.421 / 0.070 [‡]	0.431 / 0.314	0.289 / 0.274	0.391 / 0.295
C_L1r_S3	0.183 / 0.223	0.216 [†] / -	0.396 / 0.326	0.415 / 0.314	0.410 / 0.150 [†]	0.430 / 0.314	0.301 / 0.259	0.393 / 0.293
C_S4_S3	0.202 / 0.257	0.235 [†] / -	0.448 / 0.372	0.430 / 0.357	0.441 / 0.130 [‡]	0.475 / 0.354	0.349 / 0.300	0.445 / 0.348
C3_L1r_S3_S4	0.157 / 0.187	0.182 [†] / -	0.376 / 0.318	0.407 / 0.312	0.411 / 0.083 [‡]	0.427 / 0.312	0.276 / 0.251	0.372 / 0.294
C3_S1_S3_S4	0.158 / 0.231	0.185 / -	0.367 / 0.338	0.395 / 0.332	0.405 / 0.138 [‡]	0.430 / 0.338	0.270 / 0.263	0.378 / 0.306
C3_L1r_S1_S4	0.126 / 0.173	0.137 / -	0.289 / 0.274	0.367 / 0.286	0.366 / 0.064 [‡]	0.367 / 0.269	0.196 / 0.224	0.303 / 0.267
C4_L1r_S1_S3_S4	0.124 / 0.150	0.125 [†] / -	0.271 / 0.267	0.361 / 0.290	0.359 / 0.081 [‡]	0.360 / 0.279	0.198 / 0.215	0.303 / 0.270
C_L1r_X1	0.118 / 0.133 [†]	0.110 / -	0.194 / 0.208	0.212 / 0.174	0.197 / 0.036 [‡]	0.205 / 0.194	0.135 / 0.159	0.184 / 0.202
C_X1_S1	0.144 / 0.202	0.152 / -	0.285 / 0.303	0.261 / 0.209	0.247 / 0.034 [‡]	0.313 / 0.247	0.196 / 0.227	0.288 / 0.280
C_S2_X1	0.188 / 0.227	0.198 / -	0.344 / 0.300	0.323 / 0.241	0.326 / 0.031 [‡]	0.364 / 0.289	0.260 / 0.259	0.340 / 0.293
C_L1r_X1m	0.130 / 0.146 [†]	0.112 / -	0.214 / 0.231	0.256 / 0.211	0.234 / 0.049 [‡]	0.228 / 0.209	0.142 / 0.188	0.201 / 0.230
C_S1_X1s	0.186 / 0.230	0.210 / -	0.338 / 0.322	0.351 / 0.230	0.339 / 0.037 [‡]	0.393 / 0.260	0.255 / 0.246	0.358 / 0.302
C3_L1r_S1_X1	0.134 / 0.168	0.118 / -	0.218 / 0.230	0.241 / 0.140 [†]	0.246 / 0.013 [‡]	0.258 / 0.191	0.158 / 0.208	0.224 / 0.232
single obfuscation	0.200 / 0.252	0.212 ¹ / -	0.410 / 0.340	0.422 / 0.315	0.429 / -	0.449 / 0.313	0.311 / 0.280	0.414 / 0.309
held-out family	0.143 / 0.190	0.157 / -	0.271 / 0.260	0.251 / 0.217	0.247 / -	0.301 / 0.241	0.208 / 0.226	0.274 / 0.264
depth 2, all seen	0.164 / 0.214	0.152 ³ / -	0.354 / 0.310	0.396 / 0.308	0.395 / -	0.403 / 0.302	0.266 / 0.262	0.358 / 0.298
depth 3, all seen	0.147 / 0.197	0.161 ² / -	0.344 / 0.310	0.390 / 0.310	0.394 / -	0.408 / 0.306	0.248 / 0.246	0.351 / 0.289
depth 4, all seen	0.124 / 0.150	- / -	0.271 / 0.267	0.361 / 0.290	0.359 / -	0.360 / 0.279	0.198 / 0.215	0.303 / 0.270
depth 2, unseen inside	0.153 / 0.220 ³	0.156 / -	0.275 / 0.273	0.280 / 0.213	0.269 / -	0.301 / 0.240	0.198 / 0.216	0.274 / 0.261
depth 3, unseen inside	0.134 / 0.168	0.118 / -	0.218 / 0.230	0.241 / -	0.246 / -	0.258 / 0.191	0.158 / 0.208	0.224 / 0.232

Table 12. **Every method against every obfuscation, Granite-3.1-8B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 9); *router* is the learned-gate mixture of per-transform specialists; *TIES* and *DARE* are the two sign-consensus merges of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base	ICL	clean LoRA	breadth	KL	router	TIES	DARE
	out / in	out / in	out / in	out / in	out / in	out / in	out / in	out / in
L0	0.285 / 0.291	0.293 / -	0.430 / 0.206 [†]	0.416 / 0.261	0.396 / 0.117 [†]	0.438 / 0.268	0.383 / 0.384	0.433 / 0.304
L1b	0.267 / 0.255	0.267 / -	0.338 / 0.156 [†]	0.391 / 0.228	0.381 / 0.117 [†]	0.396 / 0.252	0.315 / 0.327	0.350 / 0.278
L1r	0.268 / 0.284	0.260 / -	0.375 / 0.209 [†]	0.383 / 0.268	0.378 / 0.150	0.391 / 0.280	0.353 / 0.372	0.384 / 0.299
L2	0.251 / 0.247	0.248 / -	0.372 / 0.209 [†]	0.387 / 0.236	0.377 / 0.133 [†]	0.400 / 0.244	0.342 / 0.339	0.378 / 0.294
S1	0.254 / 0.305	0.253 / -	0.346 / 0.196 [†]	0.379 / 0.220	0.358 / 0.099 [†]	0.403 / 0.260	0.323 / 0.355	0.371 / 0.284
S2	0.272 / 0.329	0.276 / -	0.386 / 0.182 [†]	0.415 / 0.244	0.394 / 0.098 [†]	0.422 / 0.278	0.358 / 0.390	0.402 / 0.315
X1	0.147 / 0.198 [†]	0.160 / -	0.216 / 0.123 [†]	0.207 / 0.156 [†]	0.188 / 0.048 [‡]	0.244 / 0.210	0.199 / 0.275	0.231 / 0.243
C_L1b_S1	0.219 / 0.216	0.191 / -	0.263 / 0.136 [†]	0.342 / 0.199	0.337 / 0.082 [†]	0.324 / 0.234	0.247 / 0.262	0.279 / 0.228
C_L1r_S1	0.204 / 0.284	0.187 / -	0.279 / 0.206	0.344 / 0.225	0.338 / 0.126 [†]	0.334 / 0.250	0.266 / 0.311	0.301 / 0.267
C_S1_L1r	0.224 / 0.262	0.187 / -	0.279 / 0.208	0.326 / 0.241	0.319 / 0.132 [†]	0.327 / 0.254	0.262 / 0.305	0.293 / 0.276
C_L2_S4	0.242 / 0.278	0.245 / -	0.334 / 0.182 [†]	0.376 / 0.225	0.376 / 0.117 [†]	0.384 / 0.242	0.319 / 0.342	0.351 / 0.287
C_L1r_S3	0.259 / 0.296	0.259 / -	0.350 / 0.160 [†]	0.382 / 0.240	0.376 / 0.145 [†]	0.378 / 0.279	0.324 / 0.344	0.365 / 0.296
C_S4_S3	0.270 / 0.334	0.282 / -	0.380 / 0.180 [†]	0.412 / 0.254	0.389 / 0.100 [†]	0.429 / 0.290	0.353 / 0.381	0.394 / 0.313
C3_L1r_S3_S4	0.257 / 0.331	0.245 / -	0.333 / 0.180 [†]	0.380 / 0.238	0.375 / 0.135 [†]	0.375 / 0.279	0.307 / 0.365	0.348 / 0.276
C3_S1_S3_S4	0.242 / 0.323	0.248 / -	0.316 / 0.181 [†]	0.372 / 0.214	0.358 / 0.062 [†]	0.380 / 0.273	0.315 / 0.353	0.342 / 0.289
C3_L1r_S1_S4	0.221 / 0.293	0.200 / -	0.279 / 0.192	0.333 / 0.223	0.330 / 0.111 [†]	0.327 / 0.246	0.271 / 0.317	0.287 / 0.262
C4_L1r_S1_S3_S4	0.203 / 0.305	0.205 / -	0.258 / 0.175 [†]	0.325 / 0.216	0.334 / 0.106 [†]	0.331 / 0.253	0.251 / 0.329	0.281 / 0.272
C_L1r_X1	0.136 / 0.165 [†]	0.124 / -	0.157 / 0.102 [†]	0.172 / 0.163	0.163 / 0.072 [†]	0.189 / 0.170	0.153 / 0.223	0.173 / 0.201
C_X1_S1	0.153 / 0.234	0.169 / -	0.222 / 0.156 [†]	0.203 / 0.161	0.186 / 0.034 [‡]	0.258 / 0.228	0.200 / 0.294	0.244 / 0.251
C_S2_X1	0.209 / 0.300	0.217 / -	0.263 / 0.151 [†]	0.252 / 0.150 [†]	0.240 / 0.043 [‡]	0.311 / 0.249	0.246 / 0.341	0.272 / 0.277
C_L1r_X1m	0.158 / 0.159 [†]	0.145 / -	0.188 / 0.117 [†]	0.204 / 0.180	0.183 / 0.091 [†]	0.226 / 0.199	0.173 / 0.242	0.211 / 0.196
C_S1_X1s	0.206 / 0.282	0.215 / -	0.298 / 0.141 [†]	0.267 / 0.173	0.269 / 0.037 [‡]	0.330 / 0.232	0.259 / 0.329	0.293 / 0.268
C3_L1r_S1_X1	0.167 / 0.259	0.147 / -	0.186 / 0.163 [†]	0.195 / 0.156	0.198 / 0.057 [‡]	0.233 / 0.219	0.204 / 0.282	0.217 / 0.239
single obfuscation	0.262 / 0.284	0.261 / -	0.363 / -	0.391 / 0.239	0.378 / 0.150 ¹	0.402 / 0.263	0.338 / 0.357	0.377 / 0.294
held-out family	0.147 / -	0.160 / -	0.216 / -	0.207 / -	0.188 / -	0.244 / 0.210	0.199 / 0.275	0.231 / 0.243
depth 2, all seen	0.236 / 0.278	0.225 / -	0.314 / 0.207 ²	0.364 / 0.231	0.356 / -	0.363 / 0.258	0.295 / 0.324	0.330 / 0.278
depth 3, all seen	0.240 / 0.316	0.231 / -	0.309 / 0.192 ¹	0.362 / 0.225	0.354 / -	0.361 / 0.266	0.298 / 0.345	0.326 / 0.276
depth 4, all seen	0.203 / 0.305	0.205 / -	0.258 / -	0.325 / 0.216	0.334 / -	0.331 / 0.253	0.251 / 0.329	0.281 / 0.272
depth 2, unseen inside	0.172 / 0.272 ³	0.174 / -	0.226 / -	0.220 / 0.169 ⁴	0.208 / -	0.263 / 0.216	0.206 / 0.286	0.239 / 0.239
depth 3, unseen inside	0.167 / 0.259	0.147 / -	0.186 / -	0.195 / 0.156	0.198 / -	0.233 / 0.219	0.204 / 0.282	0.217 / 0.239

B Per-model master grids

One table per panel model: every condition and every stack, forward and backward, with the raw accuracy, the change against the untuned model, and the percentage of the untuned model's clean-code accuracy. Cells marked *fmt* exceed the pre-registered format-failure gate (0.25) and enter no mean; the gate is applied per cell, which is stricter than the pooled registration. The untuned CodeLlama-34B sits at 0.254 on clean code, a hair over the bar, which gates its percentage column while its deltas stand. Rows marked '–' are cells not yet run at the time of generation.

C Routing and merging grids

Every routing and merging system on every condition and stack that has it, for the models on which those arms exist (routing: CodeLlama-7B; merging: CodeLlama-7B, Granite-3.1-8B, Llama-3.1-8B, StarCoder2-15B), with the same three numbers as Appendix B. The routing arms are MoLE mixtures of the eight per-transform experts; *uniform* and *random* gates separate the mixture's worth from the routing's. Every routing cell ran through the mixture engine.

D Supplementary tables

The contrast tables referenced from the research questions that are not the one table each question carries in the body: routing and merging on stacks (Table 26), merging across the panel (Table 27), the direction ratio (Table 28), the anchoring advantage by model (Table 29), and the cross-model summary in relative terms (Table 30).

Table 13. **Master grid, CodeLlama-7B.** Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base		base_1shot			tuned_L0			mono_all			cons_lam3			tuned_X1		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
<i>Forward: output prediction</i>																	
L0	0.257	100	0.302	+4.5	117	0.427	+17.0	166	0.413	+15.6	161	0.422	+16.5	164	0.413	+15.6	161
L1b	0.198	77	0.255	+5.7	99	0.358	+16.0	139	0.388	+19.0	151	0.402	+20.4	156	0.346	+14.8	135
L1r	0.207	81	0.265	+5.8	103	0.378	+17.1	147	0.387	+18.0	151	0.395	+18.8	154	0.360	+15.3	140
L2	0.202	79	0.272	+7.0	106	0.382	+18.0	149	0.378	+17.7	147	0.398	+19.6	155	0.354	+15.2	138
S1	0.168	66	0.216	+4.7	84	0.379	+21.0	147	0.389	+22.1	151	0.400	+23.2	156	0.370	+20.2	144
S2	0.193	75	fmt	fmt	fmt	0.389	+19.6	152	0.403	+20.9	157	0.415	+22.2	162	0.383	+19.0	149
X1	0.119	46	-	-	-	0.269	+14.9	105	0.231	+11.2	90	0.286	+16.6	111	0.316	+19.7	123
C_L1b_S1	0.141	55	-	-	-	0.279	+13.7	108	0.345	+20.4	134	0.360	+21.9	140	0.278	+13.6	108
C_L1r_S1	0.131	51	-	-	-	0.294	+16.3	114	0.350	+21.9	136	0.356	+22.5	139	0.296	+16.5	115
C_S1_L1r	0.135	53	-	-	-	0.285	+15.0	111	0.326	+19.2	127	0.349	+21.4	136	0.297	+16.2	116
C_L2_S4	0.176	68	-	-	-	0.355	+17.9	138	0.385	+20.9	150	0.392	+21.7	153	0.343	+16.7	134
C_L1r_S3	0.172	67	-	-	-	0.360	+18.8	140	0.380	+20.8	148	0.387	+21.5	151	0.350	+17.8	136
C_S4_S3	0.186	72	-	-	-	0.392	+20.6	153	0.403	+21.7	157	0.422	+23.6	164	0.382	+19.6	149
seen stacks, pooled (6)	0.157	61	-	-	-	0.327	+17.1	127	0.365	+20.8	142	0.378	+22.1	147	0.324	+16.8	126
C3_L1r_S3_S4	0.150	58	-	-	-	0.340	+19.0	132	0.377	+22.7	147	0.380	+23.0	148	0.343	+19.3	133
C3_S1_S3_S4	0.128	50	-	-	-	0.354	+22.6	138	0.369	+24.1	144	0.385	+25.7	150	0.343	+21.5	133
C3_L1r_S1_S4	0.126	49	-	-	-	0.289	+16.3	112	0.353	+22.7	137	0.354	+22.7	138	0.303	+17.7	118
C4_L1r_S1_S3_S4	0.089	35	-	-	-	0.278	+18.8	108	0.336	+24.7	131	0.356	+26.7	139	0.292	+20.3	114
depth-3/4 stacks, pooled (4)	0.123	48	-	-	-	0.315	+19.2	123	0.359	+23.5	140	0.369	+24.5	144	0.320	+19.7	125
C_L1r_X1	0.102	40	-	-	-	0.184	+8.2	72	0.199	+9.6	77	0.222	+12.0	87	0.254	+15.2	99
C_X1_S1	0.104	40	-	-	-	0.274	+17.0	107	0.230	+12.6	89	0.267	+16.3	104	0.326	+22.2	127
C_S2_X1	0.145	57	-	-	-	0.313	+16.8	122	0.285	+13.9	111	0.332	+18.6	129	0.347	+20.2	135
unseen-containing stacks, pooled (3)	0.117	46	-	-	-	0.257	+14.0	100	0.238	+12.1	93	0.274	+15.7	107	0.309	+19.2	120
C_L1r_X1m	0.112	44	-	-	-	0.220	+10.7	85	0.232	+12.0	90	0.249	+13.7	97	0.281	+16.9	110
C_S1_X1s	0.151	59	-	-	-	0.334	+18.3	130	0.302	+15.1	117	0.355	+20.4	138	0.341	+19.0	133
C3_L1r_S1_X1	0.104	40	-	-	-	0.219	+11.5	85	0.222	+11.8	86	0.250	+14.6	97	0.271	+16.7	106
<i>Backward: input prediction, graded by execution</i>																	
L0	0.285	100	-	-	-	0.324	+3.9	114	0.276	-0.8	97	0.297	+1.3	104	0.312	+2.7	109
L1b	0.265	93	-	-	-	0.251	-1.3	88	0.279	+1.4	98	0.294	+2.9	103	0.265	+0.1	93
L1r	0.315	111	-	-	-	0.327	+1.1	115	0.276	-3.9	97	0.293	-2.3	103	0.327	+1.1	115
L2	0.282	99	-	-	-	0.324	+4.2	114	0.276	-0.6	97	0.275	-0.7	97	0.324	+4.2	114
S1	0.261	92	-	-	-	fmt	fmt	fmt	0.262	+0.1	92	0.280	+1.9	98	0.280	+1.8	98
S2	0.330	116	-	-	-	fmt	fmt	fmt	0.294	-3.6	103	0.332	+0.2	117	0.330	+0.0	116
X1	0.257	90	-	-	-	fmt	fmt	fmt	0.237	-2.0	83	0.281	+2.5	99	0.291	+3.4	102
C_L1b_S1	0.218	76	-	-	-	fmt	fmt	fmt	0.242	+2.5	85	0.270	+5.3	95	0.246	+2.8	86
C_L1r_S1	0.301	106	-	-	-	fmt	fmt	fmt	0.255	-4.6	90	0.270	-3.1	95	0.300	-0.2	105
C_S1_L1r	0.282	99	-	-	-	0.205	-7.7	72	0.267	-1.5	94	0.285	+0.2	100	0.262	-2.0	92
C_L2_S4	0.291	102	-	-	-	fmt	fmt	fmt	0.260	-3.1	91	0.283	-0.8	99	0.315	+2.4	111
C_L1r_S3	0.296	104	-	-	-	0.267	-2.9	94	0.278	-1.9	97	0.315	+1.9	111	0.330	+3.4	116
C_S4_S3	0.315	111	-	-	-	fmt	fmt	fmt	0.282	-3.4	99	0.333	+1.7	117	0.324	+0.8	114
seen stacks, pooled (6)	0.284	100	-	-	-	fmt	fmt	fmt	0.264	-2.0	93	0.293	+0.9	103	0.296	+1.2	104
C3_L1r_S3_S4	0.303	106	-	-	-	fmt	fmt	fmt	0.268	-3.5	94	0.302	-0.1	106	0.306	+0.4	108
C3_S1_S3_S4	0.292	103	-	-	-	fmt	fmt	fmt	0.234	-5.8	82	0.313	+2.1	110	0.291	-0.1	102
C3_L1r_S1_S4	0.299	105	-	-	-	fmt	fmt	fmt	0.229	-7.0	80	0.278	-2.2	97	0.262	-3.7	92
C4_L1r_S1_S3_S4	0.282	99	-	-	-	fmt	fmt	fmt	0.217	-6.5	76	0.288	+0.6	101	0.273	-1.0	96
depth-3/4 stacks, pooled (4)	0.294	103	-	-	-	fmt	fmt	fmt	0.237	-5.7	83	0.295	+0.1	104	0.283	-1.1	99
C_L1r_X1	0.253	89	-	-	-	0.210	-4.3	74	0.200	-5.3	70	0.250	-0.2	88	0.245	-0.7	86
C_X1_S1	0.251	88	-	-	-	fmt	fmt	fmt	0.201	-5.0	71	0.276	+2.5	97	0.255	+0.4	90
C_S2_X1	0.311	109	-	-	-	fmt	fmt	fmt	0.217	-9.4	76	0.305	-0.5	107	0.280	-3.0	98
unseen-containing stacks, pooled (3)	0.271	95	-	-	-	fmt	fmt	fmt	0.206	-6.5	72	0.277	+0.6	97	0.260	-1.1	91
C_L1r_X1m	0.250	88	-	-	-	0.220	-2.9	77	0.223	-2.7	78	0.250	+0.0	88	0.233	-1.7	82
C_S1_X1s	0.277	97	-	-	-	fmt	fmt	fmt	0.195	-8.1	69	0.288	+1.1	101	0.270	-0.6	95
C3_L1r_S1_X1	0.263	92	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	0.231	-3.2	81	0.254	-0.9	89

Table 14. **Master grid, CodeLlama-13B.** Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base		base_1shot			tuned_L0			mono_all			cons_lam3			tuned_X1		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
<i>Forward: output prediction</i>																	
L0	0.252	100	fmt	fmt	fmt	0.469	+21.7	186	0.446	+19.3	177	0.462	+21.0	183	0.463	+21.1	184
L1b	0.222	88	fmt	fmt	fmt	0.397	+17.6	158	0.423	+20.1	168	0.438	+21.7	174	0.379	+15.7	150
L1r	0.225	89	fmt	fmt	fmt	0.415	+19.0	165	0.420	+19.5	167	0.439	+21.4	174	0.407	+18.1	161
L2	0.228	90	fmt	fmt	fmt	0.407	+17.8	161	0.417	+18.9	166	0.432	+20.4	171	0.401	+17.3	159
S1	0.213	85	0.216	+0.2	86	0.408	+19.5	162	0.413	+20.0	164	0.435	+22.1	172	0.410	+19.6	163
S2	0.184	73	fmt	fmt	fmt	0.422	+23.8	168	0.445	+26.0	176	0.469	+28.4	186	0.422	+23.8	168
X1	0.149	59	-	-	-	0.294	+14.5	117	0.254	+10.5	101	0.292	+14.3	116	0.353	+20.4	140
C_L1b_S1	0.183	72	-	-	-	0.318	+13.6	126	0.367	+18.4	146	0.375	+19.2	149	0.320	+13.7	127
C_L1r_S1	0.175	69	-	-	-	0.318	+14.4	126	0.371	+19.6	147	0.376	+20.1	149	0.342	+16.8	136
C_S1_L1r	0.172	68	-	-	-	0.315	+14.4	125	0.359	+18.8	142	0.363	+19.2	144	0.350	+17.8	139
C_L2_S4	0.200	79	-	-	-	0.389	+19.0	154	0.422	+22.2	167	0.432	+23.2	171	0.385	+18.5	153
C_L1r_S3	0.186	74	-	-	-	0.407	+22.0	161	0.409	+22.3	162	0.434	+24.8	172	0.399	+21.3	158
C_S4_S3	0.192	76	-	-	-	0.424	+23.2	168	0.436	+24.4	173	0.459	+26.7	182	0.433	+24.1	172
seen stacks, pooled (6)	0.185	73	-	-	-	0.362	+17.8	144	0.394	+21.0	156	0.407	+22.2	161	0.372	+18.7	147
C3_L1r_S3_S4	0.166	66	-	-	-	0.382	+21.6	152	0.410	+24.4	163	0.429	+26.3	170	0.382	+21.5	151
C3_S1_S3_S4	0.147	58	-	-	-	0.367	+22.0	146	0.408	+26.1	162	0.423	+27.6	168	0.384	+23.7	152
C3_L1r_S1_S4	0.163	65	-	-	-	0.314	+15.1	124	0.367	+20.4	146	0.367	+20.4	146	0.349	+18.6	138
C4_L1r_S1_S3_S4	0.117	46	-	-	-	0.300	+18.4	119	0.353	+23.6	140	0.360	+24.3	143	0.331	+21.5	131
depth-3/4 stacks, pooled (4)	0.148	59	-	-	-	0.341	+19.3	135	0.384	+23.6	152	0.395	+24.7	157	0.361	+21.3	143
C_L1r_X1	0.123	49	-	-	-	0.199	+7.7	79	0.213	+9.0	84	0.236	+11.3	93	0.284	+16.1	113
C_X1_S1	0.138	55	-	-	-	0.295	+15.7	117	0.270	+13.2	107	0.307	+16.8	122	0.341	+20.3	135
C_S2_X1	0.160	64	-	-	-	0.335	+17.4	133	0.334	+17.4	133	0.356	+19.6	141	0.376	+21.6	149
unseen-containing stacks, pooled (3)	0.140	56	-	-	-	0.276	+13.6	110	0.272	+13.2	108	0.300	+15.9	119	0.334	+19.3	132
C_L1r_X1m	0.138	55	-	-	-	0.246	+10.8	98	0.248	+11.0	98	0.279	+14.1	110	0.319	+18.2	127
C_S1_X1s	0.179	71	-	-	-	0.364	+18.5	144	0.345	+16.6	137	0.360	+18.1	143	0.368	+18.9	146
C3_L1r_S1_X1	0.128	51	-	-	-	0.233	+10.5	92	0.234	+10.6	93	0.249	+12.1	99	0.291	+16.3	116
<i>Backward: input prediction, graded by execution</i>																	
L0	0.285	100	-	-	-	0.269	-1.6	94	fmt	fmt	fmt	0.231	-5.4	81	fmt	fmt	fmt
L1b	0.277	97	-	-	-	0.285	+0.8	100	fmt	fmt	fmt	0.253	-2.5	89	fmt	fmt	fmt
L1r	0.327	115	-	-	-	0.347	+2.0	122	fmt	fmt	fmt	0.229	-9.8	80	fmt	fmt	fmt
L2	0.279	98	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	0.206	-7.2	72	fmt	fmt	fmt
S1	0.304	107	-	-	-	0.341	+3.7	120	fmt	fmt	fmt	0.199	-10.6	70	fmt	fmt	fmt
S2	0.293	103	-	-	-	0.317	+2.4	111	fmt	fmt	fmt	0.238	-5.5	84	fmt	fmt	fmt
X1	0.258	91	-	-	-	0.274	+1.6	96	fmt	fmt	fmt	0.219	-4.0	77	fmt	fmt	fmt
C_L1b_S1	0.253	89	-	-	-	0.326	+7.3	114	fmt	fmt	fmt	0.201	-5.2	71	fmt	fmt	fmt
C_L1r_S1	0.325	114	-	-	-	0.360	+3.5	126	fmt	fmt	fmt	0.203	-12.1	71	fmt	fmt	fmt
C_S1_L1r	0.323	113	-	-	-	0.351	+2.8	123	fmt	fmt	fmt	0.230	-9.3	81	fmt	fmt	fmt
C_L2_S4	0.274	96	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C_L1r_S3	0.305	107	-	-	-	0.361	+5.6	127	fmt	fmt	fmt	0.220	-8.5	77	fmt	fmt	fmt
C_S4_S3	0.293	103	-	-	-	0.326	+3.4	114	fmt	fmt	fmt	0.239	-5.3	84	fmt	fmt	fmt
seen stacks, pooled (6)	0.295	104	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C3_L1r_S3_S4	0.310	109	-	-	-	0.337	+2.7	118	fmt	fmt	fmt	0.224	-8.6	79	fmt	fmt	fmt
C3_S1_S3_S4	0.295	103	-	-	-	0.329	+3.5	116	fmt	fmt	fmt	0.235	-5.9	83	fmt	fmt	fmt
C3_L1r_S1_S4	0.316	111	-	-	-	0.353	+3.7	124	fmt	fmt	fmt	0.211	-10.5	74	fmt	fmt	fmt
C4_L1r_S1_S3_S4	0.310	109	-	-	-	0.333	+2.3	117	fmt	fmt	fmt	0.222	-8.9	78	fmt	fmt	fmt
depth-3/4 stacks, pooled (4)	0.308	108	-	-	-	0.338	+3.1	119	fmt	fmt	fmt	0.223	-8.5	78	fmt	fmt	fmt
C_L1r_X1	0.281	99	-	-	-	0.254	-2.6	89	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C_X1_S1	0.289	101	-	-	-	0.264	-2.5	93	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C_S2_X1	0.271	95	-	-	-	0.279	+0.8	98	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
unseen-containing stacks, pooled (3)	0.280	98	-	-	-	0.266	-1.4	93	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C_L1r_X1m	0.291	102	-	-	-	0.259	-3.1	91	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C_S1_X1s	0.301	106	-	-	-	0.290	-1.1	102	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt
C3_L1r_S1_X1	0.304	107	-	-	-	0.293	-1.0	103	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt

Table 15. **Master grid, CodeLlama-34B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base	acc	base_1shot	acc	Δ	tuned_L0	acc	Δ	mono_all	acc	Δ	cons_lam3	acc	Δ	tuned_X1	acc	Δ
<i>Forward: output prediction</i>																	
L0	fmt	fmt	fmt	fmt	fmt	0.520	fmt	fmt	0.494	fmt	fmt	0.525	fmt	fmt	0.502	fmt	fmt
L1b	0.205	fmt	fmt	fmt	fmt	0.417	+21.2	fmt	0.469	+26.4	fmt	0.494	+28.9	fmt	0.401	+19.6	fmt
L1r	0.220	fmt	fmt	fmt	fmt	0.469	+25.0	fmt	0.457	+23.8	fmt	0.487	+26.8	fmt	0.459	+24.0	fmt
L2	0.231	fmt	fmt	fmt	fmt	0.466	+23.5	fmt	0.465	+23.5	fmt	0.478	+24.7	fmt	0.456	+22.6	fmt
S1	0.222	fmt	fmt	fmt	fmt	0.472	+24.9	fmt	0.467	+24.5	fmt	0.496	+27.3	fmt	0.461	+23.9	fmt
S2	0.191	fmt	fmt	fmt	fmt	0.484	+29.3	fmt	0.484	+29.3	fmt	0.525	+33.5	fmt	0.478	+28.7	fmt
X1	0.135	fmt	-	-	-	0.325	+19.0	fmt	0.298	+16.3	fmt	0.334	+19.9	fmt	0.381	+24.5	fmt
C_L1b_S1	0.156	fmt	-	-	-	0.322	+16.6	fmt	0.399	+24.3	fmt	0.421	+26.5	fmt	0.336	+18.0	fmt
C_L1r_S1	0.184	fmt	-	-	-	0.351	+16.8	fmt	0.393	+21.0	fmt	0.427	+24.3	fmt	0.366	+18.2	fmt
C_S1_L1r	0.180	fmt	-	-	-	0.367	+18.8	fmt	0.405	+22.5	fmt	0.425	+24.6	fmt	0.379	+20.0	fmt
C_L2_S4	0.191	fmt	-	-	-	0.431	+24.1	fmt	0.454	+26.3	fmt	0.478	+28.7	fmt	0.431	+24.1	fmt
C_L1r_S3	0.169	fmt	-	-	-	0.451	+28.1	fmt	0.454	+28.5	fmt	0.482	+31.3	fmt	0.437	+26.8	fmt
C_S4_S3	0.199	fmt	-	-	-	0.482	+28.3	fmt	0.487	+28.7	fmt	0.521	+32.2	fmt	0.476	+27.7	fmt
<i>seen stacks, pooled (6)</i>	0.180	fmt	-	-	-	0.401	+22.1	fmt	0.432	+25.2	fmt	0.459	+27.9	fmt	0.404	+22.4	fmt
C3_L1r_S3_S4	0.148	fmt	-	-	-	0.416	+26.8	fmt	0.452	+30.4	fmt	0.473	+32.5	fmt	0.424	+27.5	fmt
C3_S1_S3_S4	0.128	fmt	-	-	-	0.428	+30.1	fmt	0.454	+32.7	fmt	0.472	+34.5	fmt	0.440	+31.2	fmt
C3_L1r_S1_S4	0.147	fmt	-	-	-	0.338	+19.2	fmt	0.390	+24.3	fmt	0.412	+26.5	fmt	0.359	+21.2	fmt
C4_L1r_S1_S3_S4	0.105	fmt	-	-	-	0.316	+21.1	fmt	0.372	+26.7	fmt	0.413	+30.8	fmt	0.354	+24.9	fmt
<i>depth-3/4 stacks, pooled (4)</i>	0.132	fmt	-	-	-	0.375	+24.3	fmt	0.417	+28.5	fmt	0.442	+31.1	fmt	0.394	+26.2	fmt
C_L1r_X1	0.115	fmt	-	-	-	0.226	+11.0	fmt	0.249	+13.3	fmt	0.278	+16.2	fmt	0.275	+16.0	fmt
C_X1_S1	0.125	fmt	-	-	-	0.317	+19.2	fmt	0.290	+16.5	fmt	0.339	+21.4	fmt	0.372	+24.7	fmt
C_S2_X1	0.176	fmt	-	-	-	0.372	+19.6	fmt	0.355	+17.9	fmt	0.403	+22.7	fmt	0.395	+21.9	fmt
<i>unseen-containing stacks, pooled (3)</i>	0.139	fmt	-	-	-	0.305	+16.6	fmt	0.298	+15.9	fmt	0.340	+20.1	fmt	0.348	+20.9	fmt
C_L1r_X1m	0.128	fmt	-	-	-	0.261	+13.3	fmt	0.275	+14.6	fmt	0.294	+16.5	fmt	0.317	+18.9	fmt
C_S1_X1s	0.176	fmt	-	-	-	0.390	+21.4	fmt	0.382	+20.6	fmt	0.415	+23.9	fmt	0.411	+23.5	fmt
C3_L1r_S1_X1	0.118	fmt	-	-	-	0.256	+13.8	fmt	0.262	+14.4	fmt	0.297	+17.9	fmt	0.293	+17.5	fmt
<i>Backward: input prediction, graded by execution</i>																	
L0	0.363	100	-	-	-	0.260	-10.3	72	fmt	fmt	fmt	0.284	-7.8	78	0.269	-9.4	74
L1b	0.327	90	-	-	-	0.253	-7.4	70	fmt	fmt	fmt	0.287	-4.0	79	0.227	-10.0	63
L1r	0.339	93	-	-	-	0.255	-8.4	70	fmt	fmt	fmt	0.256	-8.3	71	0.241	-9.8	67
L2	0.357	99	-	-	-	0.280	-7.7	77	fmt	fmt	fmt	0.276	-8.1	76	0.268	-8.9	74
S1	0.323	89	-	-	-	0.305	-1.8	84	fmt	fmt	fmt	0.264	-5.8	73	0.246	-7.7	68
S2	0.350	96	-	-	-	0.354	+0.4	98	fmt	fmt	fmt	0.306	-4.4	84	0.278	-7.2	77
X1	0.303	84	-	-	-	0.253	-5.0	70	fmt	fmt	fmt	0.249	-5.3	69	0.207	-9.6	57
C_L1b_S1	0.257	71	-	-	-	0.272	+1.5	75	fmt	fmt	fmt	0.258	+0.1	71	fmt	fmt	fmt
C_L1r_S1	0.304	84	-	-	-	0.285	-1.8	79	fmt	fmt	fmt	0.200	-10.4	55	0.254	-4.9	70
C_S1_L1r	0.292	80	-	-	-	0.307	+1.5	85	fmt	fmt	fmt	0.216	-7.6	60	0.264	-2.8	73
C_L2_S4	0.344	95	-	-	-	0.291	-5.3	80	fmt	fmt	fmt	0.273	-7.1	75	0.258	-8.6	71
C_L1r_S3	0.333	92	-	-	-	0.327	-0.7	90	fmt	fmt	fmt	0.256	-7.7	71	0.269	-6.5	74
C_S4_S3	0.352	97	-	-	-	0.345	-0.7	95	fmt	fmt	fmt	0.301	-5.1	83	0.276	-7.6	76
<i>seen stacks, pooled (6)</i>	0.314	86	-	-	-	0.305	-0.9	84	fmt	fmt	fmt	0.251	-6.3	69	fmt	fmt	fmt
C3_L1r_S3_S4	0.325	90	-	-	-	0.330	+0.5	91	fmt	fmt	fmt	0.251	-7.4	69	0.272	-5.3	75
C3_S1_S3_S4	0.330	91	-	-	-	0.323	-0.7	89	fmt	fmt	fmt	0.288	-4.2	79	0.270	-6.0	74
C3_L1r_S1_S4	0.291	80	-	-	-	0.311	+2.0	86	fmt	fmt	fmt	0.199	-9.2	55	0.271	-2.0	75
C4_L1r_S1_S3_S4	0.283	78	-	-	-	0.333	+5.0	92	fmt	fmt	fmt	0.214	-6.9	59	0.270	-1.3	75
<i>depth-3/4 stacks, pooled (4)</i>	0.307	85	-	-	-	0.324	+1.7	89	fmt	fmt	fmt	0.238	-6.9	66	0.271	-3.6	75
C_L1r_X1	0.275	76	-	-	-	0.240	-3.5	66	fmt	fmt	fmt	fmt	fmt	fmt	0.195	-8.0	54
C_X1_S1	0.298	82	-	-	-	0.275	-2.3	76	fmt	fmt	fmt	fmt	fmt	fmt	0.228	-7.0	63
C_S2_X1	0.378	104	-	-	-	0.366	-1.3	101	fmt	fmt	fmt	0.288	-9.1	79	0.233	-14.5	64
<i>unseen-containing stacks, pooled (3)</i>	0.317	87	-	-	-	0.294	-2.3	81	fmt	fmt	fmt	fmt	fmt	fmt	0.219	-9.8	60
C_L1r_X1m	0.269	74	-	-	-	0.230	-3.9	63	fmt	fmt	fmt	fmt	fmt	fmt	0.189	-8.0	52
C_S1_X1s	0.346	95	-	-	-	0.302	-4.4	83	fmt	fmt	fmt	0.268	-7.8	74	0.250	-9.6	69
C3_L1r_S1_X1	0.304	84	-	-	-	0.296	-0.8	82	fmt	fmt	fmt	fmt	fmt	fmt	0.237	-6.6	65

Table 16. **Master grid, Llama-3.1-8B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base		base	1shot		tuned_L0			mono_all			cons_lam3			tuned_X1		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
<i>Forward: output prediction</i>																	
L0	0.258	100	0.320	+6.2	124	0.448	+19.0	174	0.426	+16.8	165	0.445	+18.7	172	0.459	+20.1	178
L1b	0.222	86	0.276	+5.4	107	0.361	+13.9	140	0.387	+16.5	150	0.407	+18.5	158	0.369	+14.7	143
L1r	0.242	94	0.284	+4.3	110	0.387	+14.6	150	0.387	+14.5	150	0.405	+16.3	157	0.378	+13.7	147
L2	0.215	83	0.278	+6.3	108	0.391	+17.6	152	0.386	+17.1	150	0.409	+19.4	158	0.387	+17.2	150
S1	0.213	83	0.256	+4.3	99	0.395	+18.2	153	0.391	+17.7	151	0.413	+20.0	160	0.409	+19.6	158
S2	0.205	79	0.272	+6.7	106	0.413	+20.8	160	0.413	+20.8	160	0.445	+23.9	172	0.438	+23.3	170
X1	0.132	51	-	-	-	0.249	+11.7	96	0.222	+9.0	86	0.255	+12.4	99	0.332	+20.0	129
C_L1b_S1	0.152	59	-	-	-	0.279	+12.7	108	0.338	+18.7	131	0.368	+21.6	143	0.305	+15.3	118
C_L1r_S1	0.162	63	-	-	-	0.303	+14.1	118	0.352	+19.0	136	0.363	+20.1	141	0.314	+15.2	122
C_S1_L1r	0.159	62	-	-	-	0.277	+11.8	107	0.338	+17.9	131	0.354	+19.5	137	0.305	+14.6	118
C_L2_S4	0.205	79	-	-	-	0.349	+14.3	135	0.386	+18.1	149	0.401	+19.6	155	0.350	+14.5	136
C_L1r_S3	0.198	77	-	-	-	0.367	+16.9	142	0.382	+18.4	148	0.398	+19.9	154	0.369	+17.1	143
C_S4_S3	0.214	83	-	-	-	0.421	+20.7	163	0.411	+19.7	159	0.444	+23.0	172	0.431	+21.8	167
<i>seen stacks, pooled (6)</i>	0.182	70	-	-	-	0.332	+15.1	129	0.368	+18.6	142	0.388	+20.6	150	0.346	+16.4	134
C3_L1r_S3_S4	0.183	71	-	-	-	0.329	+14.6	128	0.374	+19.1	145	0.391	+20.8	152	0.363	+18.0	141
C3_S1_S3_S4	0.160	62	-	-	-	0.359	+19.8	139	0.376	+21.5	146	0.393	+23.2	152	0.378	+21.8	147
C3_L1r_S1_S4	0.160	62	-	-	-	0.279	+11.8	108	0.342	+18.2	133	0.358	+19.7	139	0.299	+13.9	116
C4_L1r_S1_S3_S4	0.140	54	-	-	-	0.263	+12.4	102	0.335	+19.6	130	0.358	+21.8	139	0.294	+15.4	114
<i>depth-3/4 stacks, pooled (4)</i>	0.161	62	-	-	-	0.308	+14.7	119	0.357	+19.6	138	0.375	+21.4	145	0.334	+17.3	129
C_L1r_X1	0.100	39	-	-	-	0.165	+6.4	64	0.181	+8.1	70	0.207	+10.6	80	0.257	+15.7	100
C_X1_S1	0.130	50	-	-	-	0.271	+14.1	105	0.225	+9.5	87	0.253	+12.3	98	0.300	+17.0	116
C_S2_X1	0.170	66	-	-	-	0.296	+12.6	115	0.284	+11.4	110	0.317	+14.7	123	0.360	+19.0	139
<i>unseen-containing stacks, pooled (3)</i>	0.134	52	-	-	-	0.244	+11.1	95	0.230	+9.6	89	0.259	+12.5	100	0.306	+17.2	118
C_L1r_X1m	0.115	45	-	-	-	0.184	+6.9	71	0.216	+10.1	84	0.232	+11.7	90	0.291	+17.6	113
C_S1_X1s	0.193	75	-	-	-	0.341	+14.8	132	0.299	+10.6	116	0.335	+14.2	130	0.355	+16.2	138
C3_L1r_S1_X1	0.105	41	-	-	-	0.194	+8.9	75	0.207	+10.2	80	0.224	+11.9	87	0.251	+14.6	97
<i>Backward: input prediction, graded by execution</i>																	
L0	0.409	100	-	-	-	0.335	-7.4	82	0.320	-9.0	78	0.332	-7.8	81	0.365	-4.4	89
L1b	0.371	91	-	-	-	0.297	-7.4	73	0.280	-9.2	68	0.285	-8.6	70	0.297	-7.5	72
L1r	0.373	91	-	-	-	0.312	-6.1	76	0.290	-8.3	71	0.297	-7.5	73	0.316	-5.7	77
L2	0.376	92	-	-	-	0.297	-7.8	73	0.284	-9.2	69	0.294	-8.1	72	0.319	-5.7	78
S1	0.375	92	-	-	-	0.325	-5.0	79	0.285	-9.0	70	0.291	-8.4	71	0.360	-1.5	88
S2	0.378	92	-	-	-	0.345	-3.4	84	0.326	-5.2	80	0.325	-5.3	79	0.366	-1.3	89
X1	0.263	64	-	-	-	0.267	+0.3	65	0.211	-5.3	51	0.228	-3.5	56	0.302	+3.9	74
C_L1b_S1	0.287	70	-	-	-	0.253	-3.4	62	0.274	-1.4	67	0.258	-2.9	63	0.297	+1.0	72
C_L1r_S1	0.297	73	-	-	-	0.281	-1.7	69	0.280	-1.8	68	0.283	-1.4	69	0.271	-2.6	66
C_S1_L1r	0.303	74	-	-	-	0.269	-3.4	66	0.274	-2.9	67	0.274	-2.9	67	0.290	-1.3	71
C_L2_S4	0.320	78	-	-	-	0.275	-4.6	67	0.269	-5.2	66	0.286	-3.4	70	0.309	-1.1	75
C_L1r_S3	0.342	84	-	-	-	0.297	-4.5	73	0.299	-4.3	73	0.303	-3.9	74	0.308	-3.4	75
C_S4_S3	0.367	90	-	-	-	0.333	-3.4	81	0.313	-5.4	76	0.329	-3.8	80	0.362	-0.5	88
<i>seen stacks, pooled (6)</i>	0.319	78	-	-	-	0.284	-3.5	70	0.285	-3.5	70	0.289	-3.1	71	0.306	-1.3	75
C3_L1r_S3_S4	0.309	75	-	-	-	0.271	-3.8	66	0.303	-0.5	74	0.300	-0.8	73	0.300	-0.9	73
C3_S1_S3_S4	0.353	86	-	-	-	0.311	-4.2	76	0.289	-6.3	71	0.281	-7.2	69	0.340	-1.3	83
C3_L1r_S1_S4	0.302	74	-	-	-	0.270	-3.2	66	0.262	-4.0	64	0.271	-3.1	66	0.270	-3.2	66
C4_L1r_S1_S3_S4	0.281	69	-	-	-	0.234	-4.7	57	0.280	-0.2	68	0.279	-0.2	68	0.259	-2.2	63
<i>depth-3/4 stacks, pooled (4)</i>	0.311	76	-	-	-	0.272	-4.0	66	0.284	-2.8	69	0.283	-2.8	69	0.292	-1.9	71
C_L1r_X1	0.218	53	-	-	-	0.220	+0.2	54	0.193	-2.6	47	0.239	+2.1	58	0.265	+4.7	65
C_X1_S1	0.291	71	-	-	-	0.283	-0.7	69	0.210	-8.1	51	0.220	-7.1	54	0.302	+1.1	74
C_S2_X1	0.323	79	-	-	-	0.288	-3.4	70	0.229	-9.3	56	0.241	-8.1	59	0.306	-1.6	75
<i>unseen-containing stacks, pooled (3)</i>	0.277	68	-	-	-	0.264	-1.3	64	0.211	-6.7	51	0.233	-4.4	57	0.291	+1.4	71
C_L1r_X1m	0.232	57	-	-	-	0.210	-2.2	51	0.217	-1.5	53	0.245	+1.3	60	0.258	+2.7	63
C_S1_X1s	0.357	87	-	-	-	0.287	-7.0	70	0.205	-15.2	50	0.212	-14.5	52	0.330	-2.7	81
C3_L1r_S1_X1	0.268	65	-	-	-	0.234	-3.4	57	0.188	-8.0	46	0.194	-7.4	47	0.243	-2.5	59

Table 17. **Master grid, StarCoder2-15B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base	acc	acc	base_1shot	%	tuned_L0	acc	%	mono_all	acc	%	cons_lam3	acc	%	tuned_X1	acc	%
<i>Forward: output prediction</i>																	
L0	fnt	fnt	fnt	fnt	fnt	0.542	fnt	fnt	0.529	fnt	fnt	0.532	fnt	fnt	0.513	fnt	fnt
L1b	fnt	fnt	fnt	fnt	fnt	0.431	fnt	fnt	0.487	fnt	fnt	0.510	fnt	fnt	0.402	fnt	fnt
L1r	fnt	fnt	fnt	fnt	fnt	0.483	fnt	fnt	0.485	fnt	fnt	0.509	fnt	fnt	0.448	fnt	fnt
L2	fnt	fnt	fnt	fnt	fnt	0.479	fnt	fnt	0.486	fnt	fnt	0.503	fnt	fnt	0.437	fnt	fnt
S1	fnt	fnt	fnt	fnt	fnt	0.508	fnt	fnt	0.487	fnt	fnt	0.501	fnt	fnt	0.470	fnt	fnt
S2	fnt	fnt	fnt	fnt	fnt	0.512	fnt	fnt	0.514	fnt	fnt	0.539	fnt	fnt	0.501	fnt	fnt
X1	fnt	fnt	-	-	-	0.347	fnt	fnt	0.320	fnt	fnt	0.343	fnt	fnt	0.381	fnt	fnt
C_L1b_S1	fnt	fnt	-	-	-	0.360	fnt	fnt	0.416	fnt	fnt	0.456	fnt	fnt	0.336	fnt	fnt
C_L1r_S1	fnt	fnt	-	-	-	0.393	fnt	fnt	0.423	fnt	fnt	0.440	fnt	fnt	0.377	fnt	fnt
C_S1_L1r	fnt	fnt	-	-	-	0.377	fnt	fnt	0.406	fnt	fnt	0.431	fnt	fnt	0.370	fnt	fnt
C_L2_S4	fnt	fnt	-	-	-	0.454	fnt	fnt	0.470	fnt	fnt	0.508	fnt	fnt	0.421	fnt	fnt
C_L1r_S3	fnt	fnt	-	-	-	0.473	fnt	fnt	0.483	fnt	fnt	0.504	fnt	fnt	0.435	fnt	fnt
C_S4_S3	fnt	fnt	-	-	-	0.511	fnt	fnt	0.516	fnt	fnt	0.539	fnt	fnt	0.490	fnt	fnt
<i>seen stacks, pooled (6)</i>	fnt	fnt	-	-	-	0.428	fnt	fnt	0.452	fnt	fnt	0.480	fnt	fnt	0.405	fnt	fnt
C3_L1r_S3_S4	fnt	fnt	-	-	-	0.448	fnt	fnt	0.484	fnt	fnt	0.503	fnt	fnt	0.413	fnt	fnt
C3_S1_S3_S4	fnt	fnt	-	-	-	0.476	fnt	fnt	0.465	fnt	fnt	0.493	fnt	fnt	0.459	fnt	fnt
C3_L1r_S1_S4	fnt	fnt	-	-	-	0.370	fnt	fnt	0.417	fnt	fnt	0.439	fnt	fnt	0.366	fnt	fnt
C4_L1r_S1_S3_S4	fnt	fnt	-	-	-	0.357	fnt	fnt	0.412	fnt	fnt	0.437	fnt	fnt	0.353	fnt	fnt
<i>depth-3/4 stacks, pooled (4)</i>	fnt	fnt	-	-	-	0.412	fnt	fnt	0.444	fnt	fnt	0.468	fnt	fnt	0.398	fnt	fnt
C_L1r_X1	fnt	fnt	-	-	-	0.248	fnt	fnt	0.274	fnt	fnt	0.286	fnt	fnt	0.288	fnt	fnt
C_X1_S1	fnt	fnt	-	-	-	0.357	fnt	fnt	0.322	fnt	fnt	0.342	fnt	fnt	0.401	fnt	fnt
C_S2_X1	fnt	fnt	-	-	-	0.403	fnt	fnt	0.375	fnt	fnt	0.397	fnt	fnt	0.420	fnt	fnt
<i>unseen-containing stacks, pooled (3)</i>	fnt	fnt	-	-	-	0.336	fnt	fnt	0.324	fnt	fnt	0.341	fnt	fnt	0.370	fnt	fnt
C_L1r_X1m	fnt	fnt	-	-	-	0.307	fnt	fnt	0.323	fnt	fnt	0.338	fnt	fnt	0.342	fnt	fnt
C_S1_X1s	fnt	fnt	-	-	-	0.434	fnt	fnt	0.408	fnt	fnt	0.434	fnt	fnt	0.435	fnt	fnt
C3_L1r_S1_X1	fnt	fnt	-	-	-	0.278	fnt	fnt	0.298	fnt	fnt	0.302	fnt	fnt	0.306	fnt	fnt
<i>Backward: input prediction, graded by execution</i>																	
L0	0.366	100	-	-	-	0.411	+4.4	112	0.311	-5.6	85	0.399	+3.2	109	0.393	+2.7	107
L1b	fnt	fnt	-	-	-	0.316	fnt	86	0.301	fnt	82	0.395	fnt	108	fnt	fnt	fnt
L1r	fnt	fnt	-	-	-	0.354	fnt	97	0.273	fnt	75	0.369	fnt	101	0.298	fnt	81
L2	0.355	97	-	-	-	0.391	+3.7	107	0.297	-5.7	81	0.379	+2.4	103	0.363	+0.8	99
S1	fnt	fnt	-	-	-	0.361	fnt	98	fnt	fnt	fnt	0.364	fnt	99	0.342	fnt	93
S2	fnt	fnt	-	-	-	0.359	fnt	98	0.300	fnt	82	0.384	fnt	105	fnt	fnt	fnt
X1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	0.281	fnt	77	fnt	fnt	fnt
C_L1b_S1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	0.317	fnt	87	fnt	fnt	fnt
C_L1r_S1	fnt	fnt	-	-	-	0.301	fnt	82	fnt	fnt	fnt	0.303	fnt	83	fnt	fnt	fnt
C_S1_L1r	fnt	fnt	-	-	-	0.311	fnt	85	fnt	fnt	fnt	0.303	fnt	83	fnt	fnt	fnt
C_L2_S4	fnt	fnt	-	-	-	0.342	fnt	93	fnt	fnt	fnt	0.353	fnt	96	0.290	fnt	79
C_L1r_S3	fnt	fnt	-	-	-	0.300	fnt	82	0.282	fnt	77	0.368	fnt	100	fnt	fnt	fnt
C_S4_S3	fnt	fnt	-	-	-	0.368	fnt	101	0.301	fnt	82	0.386	fnt	105	fnt	fnt	fnt
<i>seen stacks, pooled (6)</i>	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	0.338	fnt	92	fnt	fnt	fnt
C3_L1r_S3_S4	fnt	fnt	-	-	-	0.294	fnt	80	fnt	fnt	fnt	0.339	fnt	92	fnt	fnt	fnt
C3_S1_S3_S4	fnt	fnt	-	-	-	0.337	fnt	92	fnt	fnt	fnt	0.333	fnt	91	fnt	fnt	fnt
C3_L1r_S1_S4	fnt	fnt	-	-	-	0.293	fnt	80	fnt	fnt	fnt	0.270	fnt	74	fnt	fnt	fnt
C4_L1r_S1_S3_S4	fnt	fnt	-	-	-	0.289	fnt	79	fnt	fnt	fnt	0.273	fnt	74	fnt	fnt	fnt
<i>depth-3/4 stacks, pooled (4)</i>	fnt	fnt	-	-	-	0.304	fnt	83	fnt	fnt	fnt	0.304	fnt	83	fnt	fnt	fnt
C_L1r_X1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt
C_X1_S1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt
C_S2_X1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt
<i>unseen-containing stacks, pooled (3)</i>	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt
C_L1r_X1m	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	0.234	fnt	64	fnt	fnt	fnt
C_S1_X1s	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt
C3_L1r_S1_X1	fnt	fnt	-	-	-	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt	fnt

Table 18. **Master grid, Gemma-3-12B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base	acc	base_1shot	acc	Δ	tuned_L0	acc	Δ	mono_all	acc	Δ	cons_lam3	acc	Δ	tuned_X1	acc	Δ
<i>Forward: output prediction</i>																	
L0	fmt	fmt	fmt	fmt	fmt	0.520	fmt	fmt	0.507	fmt	fmt	0.502	fmt	fmt	0.506	fmt	fmt
L1b	fmt	fmt	fmt	fmt	fmt	0.441	fmt	fmt	0.475	fmt	fmt	0.488	fmt	fmt	0.402	fmt	fmt
L1r	fmt	fmt	fmt	fmt	fmt	0.459	fmt	fmt	0.463	fmt	fmt	0.472	fmt	fmt	0.447	fmt	fmt
L2	fmt	fmt	fmt	fmt	fmt	0.456	fmt	fmt	0.471	fmt	fmt	0.478	fmt	fmt	0.441	fmt	fmt
S1	fmt	fmt	fmt	fmt	fmt	0.456	fmt	fmt	0.457	fmt	fmt	0.483	fmt	fmt	0.452	fmt	fmt
S2	fmt	fmt	fmt	fmt	fmt	0.500	fmt	fmt	0.503	fmt	fmt	0.504	fmt	fmt	0.489	fmt	fmt
X1	0.152	fmt	-	-	-	0.310	+15.7	fmt	0.263	+11.0	fmt	0.289	+13.7	fmt	0.376	+22.3	fmt
C_L1b_S1	fmt	fmt	-	-	-	0.333	fmt	fmt	0.406	fmt	fmt	0.420	fmt	fmt	0.330	fmt	fmt
C_L1r_S1	fmt	fmt	-	-	-	0.336	fmt	fmt	0.389	fmt	fmt	0.416	fmt	fmt	0.344	fmt	fmt
C_S1_L1r	fmt	fmt	-	-	-	0.334	fmt	fmt	0.393	fmt	fmt	0.414	fmt	fmt	0.329	fmt	fmt
C_L2_S4	fmt	fmt	-	-	-	0.438	fmt	fmt	0.460	fmt	fmt	0.469	fmt	fmt	0.424	fmt	fmt
C_L1r_S3	fmt	fmt	-	-	-	0.438	fmt	fmt	0.462	fmt	fmt	0.469	fmt	fmt	0.423	fmt	fmt
C_S4_S3	fmt	fmt	-	-	-	0.491	fmt	fmt	0.506	fmt	fmt	0.503	fmt	fmt	0.481	fmt	fmt
<i>seen stacks, pooled (6)</i>	fmt	fmt	-	-	-	0.395	fmt	fmt	0.436	fmt	fmt	0.448	fmt	fmt	0.388	fmt	fmt
C3_L1r_S3_S4	fmt	fmt	-	-	-	0.419	fmt	fmt	0.454	fmt	fmt	0.467	fmt	fmt	0.417	fmt	fmt
C3_S1_S3_S4	fmt	fmt	-	-	-	0.425	fmt	fmt	0.452	fmt	fmt	0.465	fmt	fmt	0.431	fmt	fmt
C3_L1r_S1_S4	fmt	fmt	-	-	-	0.320	fmt	fmt	0.390	fmt	fmt	0.415	fmt	fmt	0.330	fmt	fmt
C4_L1r_S1_S3_S4	fmt	fmt	-	-	-	0.302	fmt	fmt	0.394	fmt	fmt	0.413	fmt	fmt	0.323	fmt	fmt
<i>depth-3/4 stacks, pooled (4)</i>	fmt	fmt	-	-	-	0.366	fmt	fmt	0.423	fmt	fmt	0.440	fmt	fmt	0.375	fmt	fmt
C_L1r_X1	0.102	fmt	-	-	-	0.195	+9.3	fmt	0.206	+10.4	fmt	0.224	+12.2	fmt	0.285	+18.3	fmt
C_X1_S1	0.172	fmt	-	-	-	0.317	+14.6	fmt	0.271	+10.0	fmt	0.296	+12.5	fmt	0.358	+18.6	fmt
C_S2_X1	0.201	fmt	-	-	-	0.368	+16.6	fmt	0.349	+14.8	fmt	0.364	+16.3	fmt	0.412	+21.1	fmt
<i>unseen-containing stacks, pooled (3)</i>	0.158	fmt	-	-	-	0.293	+13.5	fmt	0.275	+11.7	fmt	0.295	+13.7	fmt	0.352	+19.3	fmt
C_L1r_X1m	fmt	fmt	-	-	-	0.220	fmt	fmt	0.235	fmt	fmt	0.244	fmt	fmt	0.312	fmt	fmt
C_S1_X1s	0.255	fmt	-	-	-	0.395	+14.0	fmt	0.374	+12.0	fmt	0.395	+14.0	fmt	0.401	+14.7	fmt
C3_L1r_S1_X1	0.136	fmt	-	-	-	0.225	+8.9	fmt	0.236	+10.0	fmt	0.258	+12.2	fmt	0.260	+12.4	fmt
<i>Backward: input prediction, graded by execution</i>																	
L0	0.453	100	-	-	-	0.440	-1.3	97	0.447	-0.6	99	0.355	-9.8	78	0.469	+1.6	104
L1b	0.405	89	-	-	-	0.353	-5.2	78	0.427	+2.2	94	fmt	fmt	fmt	0.395	-1.0	87
L1r	0.420	93	-	-	-	0.394	-2.6	87	0.411	-0.9	91	fmt	fmt	fmt	0.428	+0.8	94
L2	0.403	89	-	-	-	0.378	-2.6	83	0.405	+0.2	89	fmt	fmt	fmt	0.431	+2.8	95
S1	0.424	94	-	-	-	0.375	-4.9	83	0.421	-0.2	93	fmt	fmt	fmt	0.461	+3.7	102
S2	0.443	98	-	-	-	0.444	+0.1	98	0.457	+1.4	101	fmt	fmt	fmt	0.454	+1.1	100
X1	0.333	74	-	-	-	0.305	-2.9	67	0.333	+0.0	74	fmt	fmt	fmt	0.393	+6.0	87
C_L1b_S1	0.349	77	-	-	-	0.317	-3.3	70	0.366	+1.7	81	fmt	fmt	fmt	0.370	+2.1	82
C_L1r_S1	0.356	79	-	-	-	0.319	-3.7	70	0.379	+2.2	84	fmt	fmt	fmt	0.397	+4.1	88
C_S1_L1r	0.337	74	-	-	-	0.319	-1.8	70	0.379	+4.1	84	fmt	fmt	fmt	0.407	+6.9	90
C_L2_S4	0.387	85	-	-	-	0.381	-0.6	84	0.416	+2.9	92	fmt	fmt	fmt	0.427	+4.0	94
C_L1r_S3	0.427	94	-	-	-	0.402	-2.6	89	0.423	-0.5	93	fmt	fmt	fmt	0.438	+1.1	97
C_S4_S3	0.434	96	-	-	-	0.438	+0.4	97	0.462	+2.8	102	fmt	fmt	fmt	0.469	+3.5	104
<i>seen stacks, pooled (6)</i>	0.382	84	-	-	-	0.363	-1.9	80	0.404	+2.2	89	fmt	fmt	fmt	0.418	+3.6	92
C3_L1r_S3_S4	0.398	88	-	-	-	0.386	-1.2	85	0.420	+2.2	93	fmt	fmt	fmt	0.447	+4.9	99
C3_S1_S3_S4	0.387	86	-	-	-	0.354	-3.3	78	0.419	+3.1	92	fmt	fmt	fmt	0.440	+5.2	97
C3_L1r_S1_S4	0.347	77	-	-	-	0.302	-4.5	67	0.376	+3.0	83	fmt	fmt	fmt	0.377	+3.0	83
C4_L1r_S1_S3_S4	0.329	73	-	-	-	0.293	-3.7	65	0.364	+3.5	80	fmt	fmt	fmt	0.399	+6.9	88
<i>depth-3/4 stacks, pooled (4)</i>	0.365	81	-	-	-	0.334	-3.2	74	0.395	+2.9	87	fmt	fmt	fmt	0.416	+5.0	92
C_L1r_X1	0.258	57	-	-	-	0.242	-1.6	53	0.279	+2.1	62	fmt	fmt	fmt	0.329	+7.2	73
C_X1_S1	0.348	77	-	-	-	0.282	-6.5	62	0.331	-1.7	73	fmt	fmt	fmt	0.410	+6.2	91
C_S2_X1	0.360	79	-	-	-	0.339	-2.0	75	0.357	-0.3	79	fmt	fmt	fmt	0.398	+3.8	88
<i>unseen-containing stacks, pooled (3)</i>	0.322	71	-	-	-	0.288	-3.4	64	0.322	+0.1	71	fmt	fmt	fmt	0.379	+5.7	84
C_L1r_X1m	0.273	60	-	-	-	0.269	-0.4	59	0.323	+5.0	71	fmt	fmt	fmt	0.349	+7.6	77
C_S1_X1s	0.412	91	-	-	-	0.371	-4.1	82	0.383	-3.0	84	fmt	fmt	fmt	0.448	+3.6	99
C3_L1r_S1_X1	0.279	62	-	-	-	0.226	-5.3	50	0.270	-0.9	60	fmt	fmt	fmt	0.337	+5.8	74

Table 19. **Master grid, CodeGemma-7B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base		base_1shot			tuned_L0			mono_all			cons_lam3			tuned_X1		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
<i>Forward: output prediction</i>																	
L0	0.237	100	fmt	fmt	fmt	0.465	+22.9	197	0.439	+20.2	186	0.445	+20.8	188	0.462	+22.6	195
L1b	0.185	78	fmt	fmt	fmt	0.382	+19.7	161	0.425	+24.0	180	0.435	+25.1	184	0.345	+16.0	146
L1r	0.213	90	fmt	fmt	fmt	0.399	+18.6	169	0.417	+20.4	176	0.417	+20.4	176	0.390	+17.7	165
L2	0.208	88	fmt	fmt	fmt	0.412	+20.4	174	0.422	+21.4	178	0.422	+21.4	178	0.392	+18.3	166
S1	0.195	82	0.212	+1.7	90	0.405	+21.0	171	0.415	+22.1	176	0.421	+22.6	178	0.423	+22.8	179
S2	0.198	84	fmt	fmt	fmt	0.450	+25.2	190	0.429	+23.1	181	0.448	+25.0	189	0.444	+24.6	188
X1	0.143	61	-	-	-	0.271	+12.8	115	0.251	+10.8	106	0.247	+10.4	104	0.348	+20.4	147
C_L1b_S1	0.148	63	-	-	-	0.286	+13.7	121	0.372	+22.3	157	0.368	+21.9	156	0.287	+13.9	121
C_L1r_S1	0.144	61	-	-	-	0.315	+17.2	133	0.373	+22.9	158	0.367	+22.3	155	0.304	+16.0	129
C_S1_L1r	0.132	56	-	-	-	0.288	+15.6	122	0.363	+23.1	154	0.361	+22.8	153	0.299	+16.7	127
C_L2_S4	0.176	75	-	-	-	0.391	+21.5	165	0.423	+24.7	179	0.421	+24.5	178	0.374	+19.8	158
C_L1r_S3	0.183	77	-	-	-	0.396	+21.3	168	0.415	+23.2	175	0.410	+22.7	173	0.375	+19.2	159
C_S4_S3	0.202	85	-	-	-	0.448	+24.7	189	0.430	+22.9	182	0.441	+23.9	186	0.446	+24.5	189
seen stacks, pooled (6)	0.164	69	-	-	-	0.354	+19.0	150	0.396	+23.2	167	0.395	+23.0	167	0.348	+18.4	147
C3_L1r_S3_S4	0.157	66	-	-	-	0.376	+21.8	159	0.407	+25.0	172	0.411	+25.4	174	0.361	+20.3	152
C3_S1_S3_S4	0.158	67	-	-	-	0.367	+20.9	155	0.395	+23.7	167	0.405	+24.7	171	0.392	+23.4	166
C3_L1r_S1_S4	0.126	53	-	-	-	0.289	+16.3	122	0.367	+24.1	155	0.366	+23.9	155	0.301	+17.5	127
C4_L1r_S1_S3_S4	0.124	52	-	-	-	0.271	+14.7	114	0.361	+23.7	153	0.359	+23.5	152	0.285	+16.1	120
depth-3/4 stacks, pooled (4)	0.141	60	-	-	-	0.326	+18.4	138	0.383	+24.1	162	0.385	+24.4	163	0.335	+19.3	141
C_L1r_X1	0.118	50	-	-	-	0.194	+7.6	82	0.212	+9.4	90	0.197	+7.9	83	0.245	+12.8	104
C_X1_S1	0.144	61	-	-	-	0.285	+14.0	120	0.261	+11.6	110	0.247	+10.3	105	0.339	+19.4	143
C_S2_X1	0.188	80	-	-	-	0.344	+15.6	146	0.323	+13.5	136	0.326	+13.8	138	0.379	+19.1	160
unseen-containing stacks, pooled (3)	0.150	63	-	-	-	0.274	+12.4	116	0.265	+11.5	112	0.257	+10.7	109	0.321	+17.1	136
C_L1r_X1m	0.130	55	-	-	-	0.214	+8.4	90	0.256	+12.5	108	0.234	+10.4	99	0.288	+15.8	122
C_S1_X1s	0.186	79	-	-	-	0.338	+15.2	143	0.351	+16.5	148	0.339	+15.3	143	0.386	+20.0	163
C3_L1r_S1_X1	0.134	57	-	-	-	0.218	+8.4	92	0.241	+10.6	102	0.246	+11.2	104	0.254	+11.9	107
<i>Backward: input prediction, graded by execution</i>																	
L0	0.266	100	-	-	-	0.364	+9.8	137	0.324	+5.8	122	fmt	fmt	fmt	0.256	-1.0	96
L1b	0.268	101	-	-	-	0.309	+4.1	116	0.298	+3.1	112	fmt	fmt	fmt	0.193	-7.4	73
L1r	0.251	94	-	-	-	0.333	+8.3	125	0.301	+5.0	113	fmt	fmt	fmt	0.225	-2.6	85
L2	0.239	90	-	-	-	0.327	+8.7	123	0.291	+5.1	109	fmt	fmt	fmt	0.220	-1.9	83
S1	0.234	88	-	-	-	0.357	+12.3	134	0.337	+10.3	127	fmt	fmt	fmt	0.231	-0.3	87
S2	0.270	102	-	-	-	0.372	+10.2	140	0.351	+8.0	132	fmt	fmt	fmt	0.248	-2.3	93
X1	0.190	72	-	-	-	0.260	+7.0	98	0.217	+2.7	82	fmt	fmt	fmt	0.235	+4.4	88
C_L1b_S1	0.229	86	-	-	-	0.274	+4.5	103	0.276	+4.7	104	fmt	fmt	fmt	fmt	fmt	fmt
C_L1r_S1	0.170	64	-	-	-	0.289	+11.9	109	0.290	+12.0	109	fmt	fmt	fmt	0.208	+3.8	78
C_S1_L1r	0.168	63	-	-	-	0.284	+11.6	107	0.316	+14.8	119	fmt	fmt	fmt	0.203	+3.4	76
C_L2_S4	0.239	90	-	-	-	0.318	+8.0	120	0.299	+6.0	112	fmt	fmt	fmt	0.238	-0.1	90
C_L1r_S3	0.223	84	-	-	-	0.326	+10.4	123	0.314	+9.1	118	fmt	fmt	fmt	0.223	-0.0	84
C_S4_S3	0.257	97	-	-	-	0.372	+11.5	140	0.357	+10.0	134	fmt	fmt	fmt	0.242	-1.5	91
seen stacks, pooled (6)	0.214	81	-	-	-	0.310	+9.6	117	0.308	+9.4	116	fmt	fmt	fmt	fmt	fmt	fmt
C3_L1r_S3_S4	0.187	70	-	-	-	0.318	+13.1	120	0.312	+12.5	118	fmt	fmt	fmt	0.221	+3.4	83
C3_S1_S3_S4	0.231	87	-	-	-	0.338	+10.7	127	0.332	+10.0	125	fmt	fmt	fmt	0.234	+0.3	88
C3_L1r_S1_S4	0.173	65	-	-	-	0.274	+10.0	103	0.286	+11.3	108	fmt	fmt	fmt	0.191	+1.8	72
C4_L1r_S1_S3_S4	0.150	56	-	-	-	0.267	+11.7	101	0.290	+14.0	109	fmt	fmt	fmt	0.177	+2.7	67
depth-3/4 stacks, pooled (4)	0.185	70	-	-	-	0.299	+11.4	113	0.305	+12.0	115	fmt	fmt	fmt	0.206	+2.0	77
C_L1r_X1	fmt	fmt	-	-	-	0.208	fmt	78	0.174	fmt	66	fmt	fmt	fmt	0.178	fmt	67
C_X1_S1	0.202	76	-	-	-	0.303	+10.1	114	0.209	+0.6	79	fmt	fmt	fmt	0.227	+2.5	86
C_S2_X1	0.227	85	-	-	-	0.300	+7.3	113	0.241	+1.4	91	fmt	fmt	fmt	fmt	fmt	fmt
unseen-containing stacks, pooled (3)	fmt	fmt	-	-	-	0.271	fmt	102	0.208	fmt	78	fmt	fmt	fmt	fmt	fmt	fmt
C_L1r_X1m	fmt	fmt	-	-	-	0.231	fmt	87	0.211	fmt	79	fmt	fmt	fmt	0.205	fmt	77
C_S1_X1s	0.230	86	-	-	-	0.322	+9.3	121	0.230	+0.1	87	fmt	fmt	fmt	0.243	+1.4	92
C3_L1r_S1_X1	0.168	63	-	-	-	0.230	+6.2	87	fmt	fmt	fmt	fmt	fmt	fmt	0.191	+2.3	72

Table 20. **Master grid, Granite-3.1-8B**. Every condition and every stack, forward (output prediction) and backward (input prediction). Per system: *acc* raw accuracy; Δ points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

Condition	untuned		ICL			clean LoRA			breadth			KL			family		
	base		base_1shot			tuned_L0			mono_all			cons_lam3			tuned_X1		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
<i>Forward: output prediction</i>																	
L0	0.285	100	0.293	+0.8	103	0.430	+14.5	151	0.416	+13.1	146	0.396	+11.1	139	0.413	+12.8	145
L1b	0.267	94	0.267	+0.0	94	0.338	+7.1	118	0.391	+12.5	137	0.381	+11.4	134	0.321	+5.4	113
L1r	0.268	94	0.260	-0.8	91	0.375	+10.8	132	0.383	+11.5	134	0.378	+11.1	133	0.376	+10.8	132
L2	0.251	88	0.248	-0.4	87	0.372	+12.0	130	0.387	+13.5	136	0.377	+12.6	132	0.366	+11.4	128
S1	0.254	89	0.253	-0.2	89	0.346	+9.1	121	0.379	+12.5	133	0.358	+10.4	126	0.350	+9.5	123
S2	0.272	96	0.276	+0.4	97	0.386	+11.4	136	0.415	+14.2	145	0.394	+12.1	138	0.398	+12.5	140
X1	0.147	51	-	-	-	0.216	+6.9	76	0.207	+6.0	73	0.188	+4.1	66	0.282	+13.5	99
C_L1b_S1	0.219	77	-	-	-	0.263	+4.4	92	0.342	+12.4	120	0.337	+11.8	118	0.261	+4.2	92
C_L1r_S1	0.204	72	-	-	-	0.279	+7.5	98	0.344	+14.0	121	0.338	+13.3	118	0.291	+8.7	102
C_S1_L1r	0.224	79	-	-	-	0.279	+5.4	98	0.326	+10.2	115	0.319	+9.5	112	0.291	+6.6	102
C_L2_S4	0.242	85	-	-	-	0.334	+9.2	117	0.376	+13.4	132	0.376	+13.4	132	0.341	+9.9	120
C_L1r_S3	0.259	91	-	-	-	0.350	+9.1	123	0.382	+12.3	134	0.376	+11.7	132	0.359	+10.1	126
C_S4_S3	0.270	95	-	-	-	0.380	+11.0	133	0.412	+14.2	144	0.389	+11.9	137	0.383	+11.3	134
<i>seen stacks, pooled (6)</i>	0.236	83	-	-	-	0.314	+7.8	110	0.364	+12.7	128	0.356	+12.0	125	0.321	+8.5	113
C3_L1r_S3_S4	0.257	90	-	-	-	0.333	+7.6	117	0.380	+12.3	133	0.375	+11.8	132	0.340	+8.3	119
C3_S1_S3_S4	0.242	85	-	-	-	0.316	+7.4	111	0.372	+13.1	131	0.358	+11.7	126	0.330	+8.9	116
C3_L1r_S1_S4	0.221	78	-	-	-	0.279	+5.7	98	0.333	+11.2	117	0.330	+10.9	116	0.283	+6.1	99
C4_L1r_S1_S3_S4	0.203	71	-	-	-	0.258	+5.5	90	0.325	+12.2	114	0.334	+13.1	117	0.270	+6.7	95
<i>depth-3/4 stacks, pooled (4)</i>	0.231	81	-	-	-	0.296	+6.6	104	0.352	+12.2	124	0.349	+11.9	123	0.306	+7.5	107
C_L1r_X1	0.136	48	-	-	-	0.157	+2.1	55	0.172	+3.6	60	0.163	+2.7	57	0.245	+11.0	86
C_X1_S1	0.153	54	-	-	-	0.222	+7.0	78	0.203	+5.0	71	0.186	+3.3	65	0.272	+12.0	96
C_S2_X1	0.209	73	-	-	-	0.263	+5.5	92	0.252	+4.4	89	0.240	+3.1	84	0.323	+11.4	113
<i>unseen-containing stacks, pooled (3)</i>	0.166	58	-	-	-	0.214	+4.8	75	0.209	+4.3	73	0.196	+3.1	69	0.280	+11.4	98
C_L1r_X1m	0.158	55	-	-	-	0.188	+3.0	66	0.204	+4.7	72	0.183	+2.6	64	0.279	+12.1	98
C_S1_X1s	0.206	72	-	-	-	0.298	+9.2	104	0.267	+6.1	94	0.269	+6.3	94	0.318	+11.2	111
C3_L1r_S1_X1	0.167	59	-	-	-	0.186	+1.8	65	0.195	+2.8	68	0.198	+3.1	70	0.238	+7.0	83
<i>Backward: input prediction, graded by execution</i>																	
L0	0.291	100	-	-	-	fmt	fmt	fmt	0.261	-3.0	90	fmt	fmt	fmt	0.272	-1.9	94
L1b	0.255	88	-	-	-	fmt	fmt	fmt	0.228	-2.7	79	fmt	fmt	fmt	0.211	-4.4	73
L1r	0.284	98	-	-	-	fmt	fmt	fmt	0.268	-1.7	92	0.150	-13.4	52	0.262	-2.3	90
L2	0.247	85	-	-	-	fmt	fmt	fmt	0.236	-1.1	81	fmt	fmt	fmt	0.222	-2.5	76
S1	0.305	105	-	-	-	fmt	fmt	fmt	0.220	-8.6	75	fmt	fmt	fmt	0.236	-6.9	81
S2	0.329	113	-	-	-	fmt	fmt	fmt	0.244	-8.5	84	fmt	fmt	fmt	0.266	-6.2	92
X1	fmt	fmt	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	0.207	fmt	71
C_L1b_S1	0.216	74	-	-	-	fmt	fmt	fmt	0.199	-1.8	68	fmt	fmt	fmt	0.194	-2.2	67
C_L1r_S1	0.284	98	-	-	-	0.206	-7.8	71	0.225	-5.9	77	fmt	fmt	fmt	0.234	-5.0	80
C_S1_L1r	0.262	90	-	-	-	0.208	-5.4	72	0.241	-2.2	83	fmt	fmt	fmt	0.242	-2.0	83
C_L2_S4	0.278	96	-	-	-	fmt	fmt	fmt	0.225	-5.3	78	fmt	fmt	fmt	0.213	-6.5	73
C_L1r_S3	0.296	102	-	-	-	fmt	fmt	fmt	0.240	-5.6	83	fmt	fmt	fmt	0.251	-4.5	86
C_S4_S3	0.334	115	-	-	-	fmt	fmt	fmt	0.254	-8.0	87	fmt	fmt	fmt	0.254	-8.0	87
<i>seen stacks, pooled (6)</i>	0.278	96	-	-	-	fmt	fmt	fmt	0.231	-4.8	79	fmt	fmt	fmt	0.231	-4.7	80
C3_L1r_S3_S4	0.331	114	-	-	-	fmt	fmt	fmt	0.238	-9.3	82	fmt	fmt	fmt	0.235	-9.6	81
C3_S1_S3_S4	0.323	111	-	-	-	fmt	fmt	fmt	0.214	-10.9	74	fmt	fmt	fmt	0.242	-8.1	83
C3_L1r_S1_S4	0.293	101	-	-	-	0.192	-10.0	66	0.223	-6.9	77	fmt	fmt	fmt	0.238	-5.5	82
C4_L1r_S1_S3_S4	0.305	105	-	-	-	fmt	fmt	fmt	0.216	-8.9	74	fmt	fmt	fmt	0.239	-6.5	82
<i>depth-3/4 stacks, pooled (4)</i>	0.313	108	-	-	-	fmt	fmt	fmt	0.223	-9.0	77	fmt	fmt	fmt	0.239	-7.4	82
C_L1r_X1	fmt	fmt	-	-	-	fmt	fmt	fmt	0.163	fmt	56	fmt	fmt	fmt	0.190	fmt	65
C_X1_S1	0.234	80	-	-	-	fmt	fmt	fmt	0.161	-7.3	55	fmt	fmt	fmt	0.242	+0.8	83
C_S2_X1	0.300	103	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	0.235	-6.5	81
<i>unseen-containing stacks, pooled (3)</i>	fmt	fmt	-	-	-	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	fmt	0.222	fmt	76
C_L1r_X1m	fmt	fmt	-	-	-	fmt	fmt	fmt	0.180	fmt	62	fmt	fmt	fmt	0.188	fmt	65
C_S1_X1s	0.282	97	-	-	-	fmt	fmt	fmt	0.173	-10.9	59	fmt	fmt	fmt	0.234	-4.8	81
C3_L1r_S1_X1	0.259	89	-	-	-	fmt	fmt	fmt	0.156	-10.3	54	fmt	fmt	fmt	0.226	-3.3	78

Table 21. **Routing, CodeLlama-7B.** MoLE mixtures of the eight per-transform experts on every condition and stack that has them: *router* = trained gate, *hard router* = its argmax, *uniform* = fixed uniform gate, *random* = gate frozen at random initialisation. Per system: raw accuracy, Δ points against the untuned model on the same items, and % of the untuned model’s clean-code accuracy. Every cell ran through the mixture engine.

Condition	untuned			router			hard router			uniform			random		
	base	acc	%	mole_router	acc	Δ	mole_hardrouter	acc	Δ	mole_uniform	acc	Δ	mole_random	acc	Δ
L0	0.258	100		0.429	+17.1	167	0.428	+17.0	167	0.428	+16.9	166	0.433	+17.5	169
L1b	0.196	76		0.381	+18.5	148	0.384	+18.8	149	0.371	+17.5	144	0.375	+17.9	146
L1r	0.205	80		0.383	+17.7	149	0.381	+17.6	148	0.390	+18.4	152	0.391	+18.6	152
L2	0.198	77		0.386	+18.8	150	0.384	+18.6	149	0.392	+19.3	152	0.393	+19.5	153
S1	0.167	65		0.395	+22.8	154	0.393	+22.6	153	0.395	+22.8	154	0.394	+22.7	153
S2	0.191	74		0.421	+23.0	164	0.421	+22.9	164	0.415	+22.3	161	0.411	+22.0	160
X1	0.118	46		0.279	+16.1	109	0.283	+16.5	110	0.283	+16.6	110	0.283	+16.5	110
C_L1b_S1	0.141	55		0.325	+18.4	126	0.324	+18.3	126	0.310	+16.8	121	0.312	+17.1	121
C_L1r_S1	0.131	51		0.346	+21.5	135	0.339	+20.8	132	0.318	+18.7	124	0.320	+18.9	125
C_S1_L1r	0.135	53		0.318	+18.4	124	0.324	+18.9	126	0.302	+16.8	118	0.303	+16.8	118
C_L2_S4	0.176	68		0.383	+20.8	149	0.379	+20.3	148	0.380	+20.4	148	0.379	+20.3	147
C_L1r_S3	0.172	67		0.391	+21.9	152	0.389	+21.7	151	0.383	+21.1	149	0.380	+20.8	148
C_S4_S3	0.185	72		0.415	+22.9	161	0.413	+22.7	161	0.409	+22.4	159	0.409	+22.4	159
C3_L1r_S3_S4	0.150	58		0.375	+22.5	146	0.376	+22.6	146	0.365	+21.5	142	0.363	+21.3	141
C3_S1_S3_S4	0.128	50		0.377	+24.8	147	0.378	+24.9	147	0.367	+23.8	143	0.367	+23.9	143
C3_L1r_S1_S4	0.126	49		0.325	+19.9	126	0.326	+20.0	127	0.306	+18.0	119	0.305	+17.9	119
C4_L1r_S1_S3_S4	0.089	35		0.334	+24.5	130	0.336	+24.7	131	0.296	+20.7	115	0.298	+20.9	116
C_L1r_X1	0.102	40		0.189	+8.7	74	0.186	+8.4	72	0.194	+9.2	76	0.190	+8.8	74
C_X1_S1	0.104	40		0.291	+18.7	113	0.286	+18.2	111	0.293	+18.9	114	0.295	+19.1	115
C_S2_X1	0.145	56		0.326	+18.1	127	0.328	+18.3	128	0.329	+18.4	128	0.329	+18.4	128
C_L1r_X1m	0.114	44		0.227	+11.3	88	0.224	+11.0	87	0.221	+10.7	86	0.221	+10.6	86
C_S1_X1s	0.148	58		0.350	+20.2	136	0.348	+20.0	135	0.344	+19.6	134	0.343	+19.5	134
C3_L1r_S1_X1	0.102	40		0.242	+14.0	94	0.245	+14.3	95	0.234	+13.2	91	0.239	+13.8	93

Table 22. **Merging, CodeLlama-7B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

Condition	untuned			TIES			DARE-TIES			DARE-linear			L0-anch. TIES			L0-anch. DARE-TIES		
	base	acc	%	merge_ties	acc	Δ	merge_dare_ties	acc	Δ	merge_dare_linear	acc	Δ	l0merge_ties	acc	Δ	l0merge_dare_ties	acc	Δ
L0	0.257	100		0.366	+11.0	143	0.426	+16.9	166	fnt fnt	fnt		0.394	+13.7	153	0.413	+15.6	161
L1b	0.197	77		0.299	+10.1	116	0.373	+17.6	145	fnt fnt	fnt		0.331	+13.4	129	0.349	+15.1	136
L1r	0.207	81		0.323	+11.6	126	0.381	+17.4	148	fnt fnt	fnt		0.338	+13.1	132	0.352	+14.5	137
L2	0.202	79		0.317	+11.5	123	0.375	+17.4	146	fnt fnt	fnt		0.337	+13.5	131	0.355	+15.3	138
S1	0.168	66		0.322	+15.4	125	0.376	+20.8	146	fnt fnt	fnt		0.342	+17.3	133	0.365	+19.6	142
S2	0.193	75		0.340	+14.7	132	0.392	+19.9	153	fnt fnt	fnt		0.365	+17.2	142	0.379	+18.5	147
X1	0.119	46		0.234	+11.4	91	0.276	+15.7	107	fnt fnt	fnt		-	-	-	-	-	-
C_L1b_S1	0.141	55		0.245	+10.4	95	0.302	+16.0	117	fnt fnt	fnt		0.269	+12.8	105	0.280	+13.9	109
C_L1r_S1	0.131	51		0.255	+12.4	99	0.301	+17.0	117	fnt fnt	fnt		0.277	+14.6	108	0.283	+15.2	110
C_S1_L1r	0.135	53		0.245	+11.0	95	0.301	+16.6	117	fnt fnt	fnt		0.273	+13.8	106	0.282	+14.7	110
C_L2_S4	0.176	68		0.299	+12.3	116	0.363	+18.7	141	fnt fnt	fnt		0.325	+14.9	126	0.349	+17.3	136
C_L1r_S3	0.172	67		0.307	+13.5	120	0.368	+19.6	143	fnt fnt	fnt		0.326	+15.4	127	0.338	+16.6	131
C_S4_S3	0.186	72		0.341	+15.5	133	0.398	+21.2	155	fnt fnt	fnt		0.367	+18.1	143	0.385	+19.9	150
C3_L1r_S3_S4	0.150	58		0.298	+14.8	116	0.349	+19.9	136	fnt fnt	fnt		0.313	+16.3	122	0.328	+17.8	128
C3_S1_S3_S4	0.128	50		0.271	+14.3	106	0.340	+21.1	132	fnt fnt	fnt		0.295	+16.6	115	0.315	+18.7	123
C3_L1r_S1_S4	0.126	49		0.240	+11.4	94	0.296	+17.0	115	fnt fnt	fnt		0.272	+14.6	106	0.283	+15.7	110
C4_L1r_S1_S3_S4	0.089	35		0.217	+12.8	85	0.290	+20.0	113	fnt fnt	fnt		0.243	+15.4	95	0.256	+16.7	100
C_L1r_X1	0.102	40		0.171	+6.9	67	0.193	+9.1	75	fnt fnt	fnt		-	-	-	-	-	-
C_X1_S1	0.104	40		0.223	+12.0	87	0.290	+18.6	113	fnt fnt	fnt		-	-	-	-	-	-
C_S2_X1	0.145	57		0.265	+12.0	103	0.317	+17.1	123	fnt fnt	fnt		-	-	-	-	-	-
C_L1r_X1m	0.112	44		0.190	+7.8	74	0.215	+10.3	84	fnt fnt	fnt		-	-	-	-	-	-
C_S1_X1s	0.151	59		0.301	+15.0	117	0.338	+18.8	132	fnt fnt	fnt		-	-	-	-	-	-
C3_L1r_S1_X1	0.104	40		0.191	+8.7	74	0.232	+12.8	90	fnt fnt	fnt		-	-	-	-	-	-

Table 23. **Merging, Granite-3.1-8B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

Condition	untuned		TIES			DARE-TIES			DARE-linear			L0-anch. TIES			L0-anch. DARE-TIES		
	base		merge_ties			merge_dare_ties			merge_dare_linear			l0merge_ties			l0merge_dare_ties		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
L0	0.285	100	0.383	+9.8	134	0.433	+14.8	152	fmt	fmt	fmt	-	-	-	-	-	-
L1b	0.266	93	0.315	+4.9	110	0.350	+8.4	123	fmt	fmt	fmt	-	-	-	-	-	-
L1r	0.268	94	0.353	+8.5	124	0.384	+11.6	135	fmt	fmt	fmt	-	-	-	-	-	-
L2	0.251	88	0.342	+9.0	120	0.378	+12.6	133	fmt	fmt	fmt	-	-	-	-	-	-
S1	0.254	89	0.323	+6.9	113	0.371	+11.7	130	fmt	fmt	fmt	-	-	-	-	-	-
S2	0.272	96	0.358	+8.5	125	0.402	+13.0	141	fmt	fmt	fmt	-	-	-	-	-	-
X1	0.147	51	0.199	+5.3	70	0.231	+8.4	81	fmt	fmt	fmt	-	-	-	-	-	-
C_L1b_S1	0.219	77	0.247	+2.9	87	0.279	+6.0	98	fmt	fmt	fmt	-	-	-	-	-	-
C_L1r_S1	0.204	72	0.266	+6.1	93	0.301	+9.7	106	fmt	fmt	fmt	-	-	-	-	-	-
C_S1_L1r	0.224	79	0.262	+3.8	92	0.293	+6.9	103	fmt	fmt	fmt	-	-	-	-	-	-
C_L2_S4	0.242	85	0.319	+7.7	112	0.351	+10.9	123	fmt	fmt	fmt	-	-	-	-	-	-
C_L1r_S3	0.259	91	0.324	+6.5	114	0.365	+10.6	128	fmt	fmt	fmt	-	-	-	-	-	-
C_S4_S3	0.270	95	0.353	+8.3	124	0.394	+12.4	138	fmt	fmt	fmt	-	-	-	-	-	-
C3_L1r_S3_S4	0.257	90	0.307	+5.0	108	0.348	+9.1	122	fmt	fmt	fmt	-	-	-	-	-	-
C3_S1_S3_S4	0.242	85	0.315	+7.3	111	0.342	+10.1	120	fmt	fmt	fmt	-	-	-	-	-	-
C3_L1r_S1_S4	0.221	78	0.271	+4.9	95	0.287	+6.6	101	fmt	fmt	fmt	-	-	-	-	-	-
C4_L1r_S1_S3_S4	0.203	71	0.251	+4.8	88	0.281	+7.8	99	fmt	fmt	fmt	-	-	-	-	-	-
C_L1r_X1	0.136	48	0.153	+1.7	54	0.173	+3.7	61	fmt	fmt	fmt	-	-	-	-	-	-
C_X1_S1	0.153	54	0.200	+4.7	70	0.244	+9.1	86	fmt	fmt	fmt	-	-	-	-	-	-
C_S2_X1	0.209	73	0.246	+3.7	86	0.272	+6.3	95	fmt	fmt	fmt	-	-	-	-	-	-
C_L1r_X1m	0.158	55	0.173	+1.5	61	0.211	+5.3	74	fmt	fmt	fmt	-	-	-	-	-	-
C_S1_X1s	0.206	72	0.259	+5.3	91	0.293	+8.7	103	fmt	fmt	fmt	-	-	-	-	-	-
C3_L1r_S1_X1	0.167	59	0.204	+3.7	72	0.217	+5.0	76	fmt	fmt	fmt	-	-	-	-	-	-

Table 24. **Merging, Llama-3.1-8B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

Condition	untuned		TIES			DARE-TIES			DARE-linear			L0-anch. TIES			L0-anch. DARE-TIES		
	base		merge_ties			merge_dare_ties			merge_dare_linear			l0merge_ties			l0merge_dare_ties		
	acc	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%	acc	Δ	%
L0	0.259	100	0.398	+14.0	154	0.462	+20.3	179	0.292	+3.4	113	-	-	-	-	-	-
L1b	0.222	86	0.336	+11.4	130	0.386	+16.4	150	0.253	+3.1	98	-	-	-	-	-	-
L1r	0.242	94	0.348	+10.6	135	0.398	+15.6	154	0.234	-0.8	91	-	-	-	-	-	-
L2	0.215	83	0.353	+13.8	137	0.396	+18.1	153	0.246	+3.1	95	-	-	-	-	-	-
S1	0.213	83	0.335	+12.2	130	0.412	+19.9	160	0.256	+4.3	99	-	-	-	-	-	-
S2	0.208	80	0.361	+15.3	140	0.437	+23.0	169	0.256	+4.9	99	-	-	-	-	-	-
X1	0.132	51	0.217	+8.5	84	0.266	+13.4	103	fmt	fmt	fmt	-	-	-	-	-	-
C_L1b_S1	0.152	59	0.252	+10.1	98	0.307	+15.6	119	0.211	+6.0	82	-	-	-	-	-	-
C_L1r_S1	0.162	63	0.273	+11.1	106	0.328	+16.6	127	0.206	+4.4	80	-	-	-	-	-	-
C_S1_L1r	0.159	62	0.247	+8.9	96	0.306	+14.8	119	0.200	+4.2	78	-	-	-	-	-	-
C_L2_S4	0.206	80	0.311	+10.5	120	0.383	+17.8	149	0.226	+2.0	88	-	-	-	-	-	-
C_L1r_S3	0.198	77	0.334	+13.5	129	0.387	+18.9	150	0.222	+2.3	86	-	-	-	-	-	-
C_S4_S3	0.214	83	0.357	+14.3	138	0.437	+22.3	169	0.255	+4.1	99	-	-	-	-	-	-
C3_L1r_S3_S4	0.184	71	0.306	+12.2	119	0.370	+18.6	143	0.199	+1.6	77	-	-	-	-	-	-
C3_S1_S3_S4	0.157	61	0.308	+15.1	119	0.377	+22.0	146	0.220	+6.2	85	-	-	-	-	-	-
C3_L1r_S1_S4	0.166	64	0.260	+9.4	101	0.319	+15.3	124	0.183	+1.7	71	-	-	-	-	-	-
C4_L1r_S1_S3_S4	0.134	52	0.235	+10.1	91	0.310	+17.6	120	0.179	+4.5	69	-	-	-	-	-	-
C_L1r_X1	0.100	39	0.147	+4.6	57	0.184	+8.3	71	fmt	fmt	fmt	-	-	-	-	-	-
C_X1_S1	0.130	50	0.233	+10.3	90	0.296	+16.6	115	fmt	fmt	fmt	-	-	-	-	-	-
C_S2_X1	0.169	66	0.263	+9.4	102	0.317	+14.8	123	fmt	fmt	fmt	-	-	-	-	-	-
C_L1r_X1m	0.114	44	0.162	+4.8	63	0.217	+10.3	84	0.126	+1.2	49	-	-	-	-	-	-
C_S1_X1s	0.198	77	0.287	+8.9	111	0.349	+15.1	135	fmt	fmt	fmt	-	-	-	-	-	-
C3_L1r_S1_X1	0.105	41	0.170	+6.6	66	0.215	+11.0	83	fmt	fmt	fmt	-	-	-	-	-	-

Table 25. **Merging, StarCoder2-15B**. Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

Condition	untuned base		TIES			DARE-TIES			DARE-linear			L0-anch. TIES			L0-anch. DARE-TIES		
	acc	%	merge_ties	Δ	%	merge_dare_ties	Δ	%	merge_dare_linear	Δ	%	l0merge_ties	Δ	%	l0merge_dare_ties	Δ	%
L0	fnt	fnt	0.484	fnt	fnt	0.539	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
L1b	fnt	fnt	0.366	fnt	fnt	0.426	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
L1r	fnt	fnt	0.414	fnt	fnt	0.476	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
L2	fnt	fnt	0.401	fnt	fnt	0.466	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
S1	fnt	fnt	0.451	fnt	fnt	0.487	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
S2	fnt	fnt	0.440	fnt	fnt	0.518	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
X1	fnt	fnt	0.283	fnt	fnt	0.343	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C_L1b_S1	fnt	fnt	0.321	fnt	fnt	0.354	fnt	fnt	0.063	fnt	fnt	fnt	-	-	-	-	-
C_L1r_S1	fnt	fnt	0.344	fnt	fnt	0.389	fnt	fnt	0.047	fnt	fnt	fnt	-	-	-	-	-
C_S1_L1r	fnt	fnt	0.346	fnt	fnt	0.387	fnt	fnt	0.041	fnt	fnt	fnt	-	-	-	-	-
C_L2_S4	fnt	fnt	0.381	fnt	fnt	0.438	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C_L1r_S3	fnt	fnt	0.390	fnt	fnt	0.459	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C_S4_S3	fnt	fnt	0.442	fnt	fnt	0.506	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C3_L1r_S3_S4	fnt	fnt	0.377	fnt	fnt	0.436	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C3_S1_S3_S4	fnt	fnt	0.393	fnt	fnt	0.464	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C3_L1r_S1_S4	fnt	fnt	0.332	fnt	fnt	0.376	fnt	fnt	0.045	fnt	fnt	fnt	-	-	-	-	-
C4_L1r_S1_S3_S4	fnt	fnt	0.301	fnt	fnt	0.358	fnt	fnt	0.026	fnt	fnt	fnt	-	-	-	-	-
C_L1r_X1	fnt	fnt	0.215	fnt	fnt	0.259	fnt	fnt	0.058	fnt	fnt	fnt	-	-	-	-	-
C_X1_S1	fnt	fnt	0.295	fnt	fnt	0.371	fnt	fnt	0.067	fnt	fnt	fnt	-	-	-	-	-
C_S2_X1	fnt	fnt	0.341	fnt	fnt	0.403	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C_L1r_X1m	fnt	fnt	0.252	fnt	fnt	0.317	fnt	fnt	0.065	fnt	fnt	fnt	-	-	-	-	-
C_S1_X1s	fnt	fnt	0.378	fnt	fnt	0.427	fnt	fnt	fnt	fnt	fnt	fnt	-	-	-	-	-
C3_L1r_S1_X1	fnt	fnt	0.241	fnt	fnt	0.286	fnt	fnt	0.058	fnt	fnt	fnt	-	-	-	-	-

Table 26. **RQ1: routing and merging on stacks of seen transforms**. Paired differences in points on the ten stacks (six depth-2, four depth-3/4), CodeLlama-7B, 394 programs common to every cell; the last column repeats the contrasts on the four stacks that contain the unseen family (319 programs; merges other than DARE-TIES have no cells there). Routing is worth about a point over a random gate and nothing over breadth; the mixture itself carries the gain (mole_uniform – base); a hard argmax router is indistinguishable from the soft one. The best merge (DARE-TIES) matches the clean-code control and the other four fall below it. Bold intervals exclude zero. Every arm ran through the mixture engine (obtune.mole.eval_mole); an earlier run through vLLM had written the untuned model under the routing arms’ names and was withdrawn.

Contrast	all ten seen	depth 2	depth 3–4	unseen inside
<i>Routing</i>				
mole_router – mole_random	+1.34 [+0.48, +2.17]	+1.11 [+0.28, +1.93]	+1.67 [+0.47, +2.86]	–0.03 [–0.99, +0.84]
mole_router – mole_uniform	+1.40 [+0.52, +2.25]	+1.17 [+0.31, +2.02]	+1.74 [+0.53, +2.90]	+0.10 [–0.84, +0.99]
mole_hardrouter – mole_router	+0.03 [–0.13, +0.19]	–0.03 [–0.24, +0.18]	+0.13 [–0.11, +0.38]	–0.13 [–0.44, +0.18]
mole_router – mono_all	–0.28 [–1.57, +1.09]	–0.06 [–1.38, +1.30]	–0.61 [–2.10, +0.91]	+2.88 [+1.33, +4.41]
mole_uniform – base	+19.79 [+17.40, +22.23]	+19.16 [+16.77, +21.58]	+20.72 [+18.16, +23.31]	+14.98 [+12.81, +17.29]
<i>Merging</i>				
merge_dare_ties – tuned_L0	+0.52 [–0.15, +1.20]	+0.61 [–0.16, +1.41]	+0.38 [–0.47, +1.25]	+0.86 [–0.05, +1.78]
merge_ties – tuned_L0	–5.50 [–6.86, –4.13]	–5.19 [–6.59, –3.77]	–5.95 [–7.47, –4.44]	–
merge_dare_linear – tuned_L0	–20.56 [–22.88, –18.25]	–20.60 [–23.00, –18.22]	–20.49 [–22.86, –18.15]	–
l0merge_ties – tuned_L0	–3.14 [–4.21, –2.12]	–2.81 [–3.98, –1.67]	–3.64 [–4.91, –2.41]	–
l0merge_dare_ties – tuned_L0	–1.74 [–2.57, –0.91]	–1.40 [–2.30, –0.47]	–2.24 [–3.26, –1.23]	–
<i>Breadth, for reference</i>				
mono_all – tuned_L0	+3.67 [+2.02, +5.24]	+3.34 [+1.61, +5.04]	+4.15 [+2.35, +5.91]	–1.49 [–3.08, +0.08]

Table 27. **Weight-space merging on the panel.** The best merge (DARE-TIES of the clean adapter and the five specialists) against the clean-code control, breadth and the KL-consistency arm, pooled over the six seen stacks and the four stacks containing the unseen family; breadth and anchoring against the control in the last two columns. Points, paired on the programs common to every cell, program-clustered bootstrap. Bold intervals exclude zero. Pre-registered per model: merging does not beat the control on seen stacks (CONFIRMED iff $ci_{hi} \leq +1$) and stays below breadth ($ci_{hi} < 0$).

Stacks	merge-clean	merge-mono	merge-cons	mono-clean	cons-clean
CodeLlama-7B					
seen (6)	+0.83 [+0.01, +1.63]	-2.80 [-4.43, -1.19]	-4.01 [-5.29, -2.70]	+3.63 [+1.96, +5.27]	+4.84 [+3.49, +6.21]
unseen inside (4)	+0.86 [-0.05, +1.78]	+2.35 [+0.63, +3.97]	-1.39 [-2.72, -0.05]	-1.49 [-3.08, +0.08]	+2.25 [+0.86, +3.61]
CodeLlama-13B					
seen (6)	+1.67 [+0.83, +2.51]	-1.40 [-2.98, +0.12]	-2.41 [-3.85, -1.05]	+3.07 [+1.55, +4.63]	+4.08 [+2.72, +5.40]
unseen inside (4)	+1.75 [+0.78, +2.80]	+2.48 [+0.89, +4.08]	+0.08 [-1.23, +1.39]	-0.73 [-2.23, +0.76]	+1.67 [+0.45, +2.82]
Llama-3.1-8B					
seen (6)	+2.31 [+1.27, +3.38]	-0.85 [-2.41, +0.72]	-2.80 [-4.26, -1.35]	+3.16 [+1.52, +4.84]	+5.11 [+3.63, +6.58]
unseen inside (4)	+2.56 [+1.49, +3.61]	+3.09 [+1.54, +4.60]	+0.81 [-0.47, +2.12]	-0.52 [-2.07, +1.02]	+1.75 [+0.39, +3.11]
StarCoder2-15B					
seen (6)	-0.69 [-1.57, +0.13]	-3.27 [-4.67, -1.86]	-5.73 [-7.05, -4.47]	+2.57 [+1.28, +3.86]	+5.04 [+3.82, +6.26]
unseen inside (4)	+1.15 [+0.08, +2.17]	+1.70 [+0.05, +3.32]	+0.39 [-0.92, +1.80]	-0.55 [-2.12, +1.02]	+0.76 [-0.47, +2.02]
Gemma-3-12B					
seen (6)	+1.27 [+0.32, +2.22]	-2.61 [-4.13, -1.11]	-3.91 [-5.28, -2.52]	+3.88 [+2.18, +5.64]	+5.17 [+3.69, +6.64]
unseen inside (4)	+0.42 [-0.55, +1.36]	+1.83 [+0.39, +3.40]	-0.18 [-1.57, +1.20]	-1.41 [-3.01, +0.13]	+0.60 [-0.78, +1.96]
CodeGemma-7B					
seen (6)	+0.44 [-0.48, +1.36]	-3.83 [-5.34, -2.32]	-3.59 [-5.08, -2.24]	+4.27 [+2.83, +5.63]	+4.03 [+2.67, +5.46]
unseen inside (4)	-0.10 [-1.02, +0.81]	-0.13 [-1.67, +1.44]	+0.92 [-0.47, +2.35]	+0.03 [-1.55, +1.57]	-1.02 [-2.38, +0.29]
Granite-3.1-8B					
seen (6)	+1.57 [+0.76, +2.46]	-3.23 [-4.92, -1.52]	-2.32 [-4.01, -0.61]	+4.80 [+3.05, +6.53]	+3.89 [+2.12, +5.67]
unseen inside (4)	+2.09 [+1.10, +3.09]	+2.20 [+0.50, +3.84]	+2.98 [+1.28, +4.65]	-0.10 [-1.78, +1.62]	-0.89 [-2.59, +0.89]

Table 28. **The direction ratio: how much of a forward gain survives the task being run backwards.** Forward is output prediction; backward is input prediction on the same programs, graded by execution. No adapter is trained on the backward task, so any backward gain is transferred program understanding rather than task fitting. $DR < 0$ means the arm bought forward accuracy by *losing* backward competence. Every adaptation method that fits the forward task directly is negative; only anchoring and family exposure are not. Both directions use the same 412 programs and the same six conditions. Bold intervals exclude zero; the ratio is descriptive and is read with its components.

Method	Arm	Forward gain	Backward gain	DR
Format-only control	formatonly	+1.47 [+0.90, +2.10]	-0.44 [-0.81, -0.08]	-0.30
Clean-code tuning	tuned_L0	+19.34 [+16.87, +21.81]	-1.60 [-3.41, +0.35]	-0.08
Training breadth	mono_all	+19.46 [+16.68, +22.01]	-0.89 [-2.86, +1.04]	-0.05
Paired consistency (ours)	cons_lam3	+20.88 [+18.07, +23.49]	+0.62 [-1.56, +2.81]	+0.03
Family exposure	tuned_X1	+17.35 [+14.98, +19.73]	+1.63 [+0.11, +3.20]	+0.09

Table 29. **RQ4: the divergence result across the panel**, on stacks containing the unseen family. cons-mono is anchoring’s advantage over breadth; fam is an arm trained on the unseen family’s trainable sibling, included because family exposure is the largest effect in the table and replicates on every model. Bold intervals exclude zero.

Model	Lineage	cons-mono	cons-clean	fam-clean
CodeLlama-7B	Meta	+3.84 [+1.90, +5.70]	+2.13 [+0.66, +3.60]	+5.93 [+4.10, +7.78]
CodeLlama-13B	Meta	+3.22 [+1.74, +4.65]	+2.02 [+0.58, +3.53]	+5.97 [+4.11, +7.91]
CodeLlama-34B	Meta	+3.95 [+2.13, +5.81]	+3.10 [+1.63, +4.58]	+4.19 [+2.48, +5.94]
Llama-3.1-8B	Meta	+2.87 [+1.20, +4.62]	+1.86 [+0.27, +3.53]	+6.67 [+4.58, +8.71]
StarCoder2-15B	BigCode	+1.36 [−0.08, +2.71]	+0.35 [−1.13, +1.86]	+3.64 [+1.90, +5.35]
Gemma-3-12B	Google	+2.44 [+0.97, +3.99]	−0.08 [−1.67, +1.51]	+5.35 [+3.21, +7.37]
CodeGemma-7B	Google	−1.59 [−3.26, +0.12]	−2.40 [−4.11, −0.85]	+5.23 [+3.25, +7.21]
Granite-3.1-8B	IBM	−1.47 [−2.99, +0.00]	−1.86 [−3.80, +0.12]	+7.29 [+5.19, +9.65]

Table 30. **Every intervention against every kind of obfuscation, across the eight-model panel**. For each system and direction, Δ is the mean change in points against the untuned model and % the mean accuracy as a percentage of the untuned model’s clean-code accuracy, over the cells that exist and pass the format gate (in parentheses: the number of cells; after the semicolon, the number gated as *fmt*). Systems: ICL = base_1shot, clean LoRA = tuned_L0, breadth = mono_all, KL = cons_lam3, family = tuned_X1. *seen* = L1b, L1r, L2, S1, S2; *seen stacks* = the six depth-2 and four depth-3/4 stacks of seen transforms; *unseen stacks* = the three depth-2 and one depth-3 stacks with the X1 family inside. Backward cells survive the gate almost only on CodeLlama-7B, whose grid (Table 13) is the one fully interpretable backward read; the per-model grids for the whole panel are in Appendix B.

System	measure	clean	seen	unseen family	seen stacks	unseen stacks
ICL	fwd Δ pts	+3.8 (3; 5 fmt)	+3.2 (16; 24 fmt)	−	−	−
	fwd % of clean	115% (3; 5 fmt)	97% (16; 24 fmt)	−	−	−
clean LoRA	fwd Δ pts	+19.0 (5; 3 fmt)	+18.5 (30; 10 fmt)	+13.7 (7; 1 fmt)	+16.7 (60; 20 fmt)	+11.5 (28; 4 fmt)
	fwd % of clean	175% (5; 3 fmt)	152% (25; 15 fmt)	102% (5; 3 fmt)	129% (50; 30 fmt)	95% (20; 12 fmt)
	bwd Δ pts	−0.4 (7; 1 fmt)	−0.3 (28; 12 fmt)	+0.2 (5; 3 fmt)	+0.8 (54; 26 fmt)	−0.8 (20; 12 fmt)
	bwd % of clean	101% (7; 1 fmt)	97% (32; 8 fmt)	79% (5; 3 fmt)	90% (63; 17 fmt)	79% (21; 11 fmt)
breadth	fwd Δ pts	+17.0 (5; 3 fmt)	+19.7 (30; 10 fmt)	+10.7 (7; 1 fmt)	+20.9 (60; 20 fmt)	+10.9 (28; 4 fmt)
	fwd % of clean	167% (5; 3 fmt)	157% (25; 15 fmt)	91% (5; 3 fmt)	146% (50; 30 fmt)	93% (20; 12 fmt)
	bwd Δ pts	−2.2 (6; 2 fmt)	−1.6 (26; 14 fmt)	−1.1 (4; 4 fmt)	−0.0 (50; 30 fmt)	−4.3 (15; 17 fmt)
	bwd % of clean	95% (6; 2 fmt)	91% (29; 11 fmt)	73% (4; 4 fmt)	88% (52; 28 fmt)	64% (17; 15 fmt)
KL	fwd Δ pts	+17.6 (5; 3 fmt)	+21.0 (30; 10 fmt)	+13.1 (7; 1 fmt)	+22.0 (60; 20 fmt)	+12.8 (28; 4 fmt)
	fwd % of clean	169% (5; 3 fmt)	161% (25; 15 fmt)	99% (5; 3 fmt)	148% (50; 30 fmt)	98% (20; 12 fmt)
	bwd Δ pts	−4.4 (6; 2 fmt)	−5.1 (22; 18 fmt)	−2.6 (4; 4 fmt)	−4.2 (39; 41 fmt)	−3.4 (9; 23 fmt)
	bwd % of clean	89% (6; 2 fmt)	86% (26; 14 fmt)	75% (5; 3 fmt)	81% (49; 31 fmt)	75% (9; 23 fmt)
family	fwd Δ pts	+18.5 (5; 3 fmt)	+17.8 (30; 10 fmt)	+20.1 (7; 1 fmt)	+17.5 (60; 20 fmt)	+16.8 (28; 4 fmt)
	fwd % of clean	173% (5; 3 fmt)	150% (25; 15 fmt)	128% (5; 3 fmt)	132% (50; 30 fmt)	116% (20; 12 fmt)
	bwd Δ pts	−1.4 (7; 1 fmt)	−2.8 (31; 9 fmt)	+1.6 (5; 3 fmt)	−1.0 (58; 22 fmt)	−0.9 (21; 11 fmt)
	bwd % of clean	96% (7; 1 fmt)	87% (33; 7 fmt)	80% (6; 2 fmt)	84% (59; 21 fmt)	76% (23; 9 fmt)